



ST. MARY'S UNIVERSITY

**Faculty of Informatics
Computer Science Department**

IOT Integrated Smart Urban Farming (MOFER)

Done by:

Group members:

- | | |
|------------------------------|----------------------|
| 1. Abinet Tamiru | RCD/1202/2012 |
| 2. Eyob Desta | RCD/1158/2012 |
| 3. Hiwot Chernet | RCD/2049/2012 |
| 4. Ruth Damtew | RCD/2086/2012 |
| 5. Tsegazeab Befikadu | RCD/2061/2012 |

Advisor: Eyasu Tekle

Date: February 15, 2023



Mofer

lot Integrated Smart
urban farming

CERTIFICATE OF AUTHENTICATED WORK

Authentication of our work

Advisee's Declaration

We the undersigned declare that this senior project paper is our original work. Prepared under the guidance of _____. All sources of materials used for the manuscript have been duly acknowledged.

Name:

Name:

Name:

Name:

Name:

Date of Submission _____

Advisor's Declaration

The paper has been submitted for the examination with my approval as the University advisor.

Name: _____

Signature: _____

Date: - _____

APPROVED BY THE COMMITTEE

CHAIRPERSON SIGNATURE_____ DATE_____

ADVISOR SIGNATURE_____ DATE_____

EXAMINOR SIGNATURE_____ DATE_____

Acknowledgment

We would like to express our sincere gratitude and appreciation to several individuals who have contributed to the success of our project. First and foremost, we would like to thank our advisor Eyasu Tekele for supervising and assisting us through the making of this project his input and feedback have helped us to refine our research and improve our approach, And the Communication Manager of Digital Green Ethiopia for providing us with information regarding their company and how they operate. We would like to extend our heartfelt thanks to all the people who participated in our survey. Their willingness to share their experiences and opinions has been invaluable and has helped us gain valuable insights into the subject matter. We are deeply grateful for their time and efforts. We would also like to express our thanks to our friends and teachers who have provided us with invaluable advice and insights throughout the project.

Lastly, we would like to thank St. Mary University for providing us with the necessary resources and support to conduct this project and for providing working space. We are grateful for their continued commitment to academic excellence and innovation.

Once again, thank you to everyone who has contributed to our project. Your support and encouragement have been invaluable, and we are deeply grateful for all that you have done to help us succeed.

Contents

CERTIFICATE OF AUTHENTICATED WORK	ii
Acknowledgment	iii
List Of Tables.....	ix
List Of Figures	x
List of Algorithms	xii
ACRONYMS.....	xiii
Introduction	1
1.1 Background of the Project	1
1.2 Statement of the Problem.....	2
1.3 Objective of the Project	3
1.3.1 General Objective	3
1.4 Significance of the Project.....	4
1.5 Feasibility Study.....	4
1.5.1 Technical Feasibility	4
1.5.2 Economic Feasibility	5
1.5.3 Operational Feasibility	5
1.5.4 Legal Feasibility.....	6
1.5.5 Schedule Feasibility.....	6
1.6 Beneficiaries of the Project	6
1.6.1 Direct Beneficiaries.....	6
1.6.2 Indirect Beneficiaries	6
1.7 Methodology	7
1.7.1 Data collection	7
1.7.1.1 Interview.....	7

1.7.1.2 Survey	7
1.7.1.3 Observation	7
1.7.2 System Design and Analysis Tools.....	8
1.7.3 Development Environment and Programming Tool.....	8
1.7.3.1 Software Development Process Model.....	8
1.7.3.2 Development Environment and Devices.....	8
1.8 Scope and Limitation of the Project.....	12
1.8.1 Scope of the Project.....	12
1.8.2 Limitation of the Project.....	12
1.9 Risk, Assumption, and Constraints.....	12
1.9.1 Risk	12
1.9.2 Assumptions.....	13
1.9.3 Constraints.....	14
1.10 Organization of the Project.....	14
1.11 Work Plan.....	14
1.12. Budget Allocation.....	16
Business area Analysis and requirement definition	17
2.1. Introduction.....	17
2.2. Business Area Analysis.....	17
2.2.2. Current system	18
2.2.3 Player of the Existing System.....	20
2.2.4. Proposed System.....	21
2.3 Requirement Gathering.....	24
2.3.1 Requirement Gathering Techniques.....	24
2.4 Method of Communication	25

2.4.1 Communication Techniques.....	25
2.5 Requirement Definition	25
2.5.1. Functional Requirement	25
2.5.1.1. Essential Use Case Diagram	27
2.5.1.2 Essential User Interface Prototyping.....	28
2.5.2 Collaboration Modeling.....	30
2.5.3. Non-Functional Requirement.....	32
System Modeling.....	34
3.1 Introduction.....	34
3.2 UI Identification	34
3.3 Business Rules Identification	35
3.4 Actor Identification.....	37
3.5 Actor Description	37
3.6 Use Case Identification	38
3.7 Use Case Diagram	39
3.8 Use Case Description	41
3.9 Activity Diagram.....	48
3.10 Sequence Diagram	54
3.11 Class Diagram	57
3.12 State Chart Diagram.....	58
3.13 User Interface Prototyping.....	60
System Design	65
4.1. System Architecture.....	65
4.2. ER Diagram.....	70
4.3. Database Normalization	71

4.3.1. First Normal Form (1NF)	72
4.3.2. Second Normal Form (2NF)	72
4.3.3. Third Normal Form (3NF)	72
4.4 Database Table Structure.....	73
4.5 Deployment Diagram.....	75
4.6. High Fidelity Prototype	76
Implementation.....	82
5.1.Introduction.....	82
5.2 Development tools	85
5.3 prototype of the system.....	86
5.3.1 Farmers	87
5.3.2 New sellers.....	91
5.3.3 Admin	92
5.4 Testing.....	93
5.4.1 User Accepting Testing	93
5.4.1.1 Blackbox testing.....	93
5.4.1.1.1 System test	93
5.4.1.2 Whitebox testing	96
5.4.1.2.1 Unit test	96
5.4.1.2.2 Integration test	97
Conclusion and Recommendation.....	99
6.1. Conclusion	99
6.2. Recommendation.....	99
Appendices	101
Appendix A: Code Sample	101

Appendix B: User questions	104
Appendix C: Interview	109
References.....	110

List Of Tables

TABLE 1.1: DEVELOPMENT ENVIRONMENT USED IN THE PROJECT.	8
TABLE 1.2: COMPUTING DEVICES USED IN THE PROJECT.	9
TABLE 1.3: TOOLS USED IN THE PROJECT.	10
TABLE 1.4: WORK PLAN OF THE PROJECT	14
TABLE 1.5: BUDGET ALLOCATION OF THE PROJECT.....	16
TABLE 3.1: UI IDENTIFICATION	34
TABLE 3.2: BUSINESS RULE IDENTIFICATION	35
TABLE 3.3: ACTOR IDENTIFICATION.....	37
TABLE 3.4: USE CASE IDENTIFICATION	38
TABLE 3.5: USE CASE DESCRIPTION FOR REGISTRATION.....	41
TABLE 3.6: USE CASE DESCRIPTION FOR LOGIN	41
TABLE 3.7: USE CASE DESCRIPTION FOR MANAGE PROFILE.....	42
TABLE 3.8: USE CASE DESCRIPTION FOR TRACK PLANT.....	42
TABLE 3.9: USE CASE DESCRIPTION FOR PAY FOR SERVICE	43
TABLE 3.10: USE CASE DESCRIPTION FOR GENERATE REPORT	44
TABLE 3.11: USE CASE DESCRIPTION FOR VIEW PRODUCT	44
TABLE 3.12: USE CASE DESCRIPTION FOR MANAGE PRODUCT	45
TABLE 3.13: USE CASE DESCRIPTION FOR USER PASSWORD RECOVERY	45
TABLE 3.14: USE CASE DESCRIPTION FOR USER MANAGE ADMIN	46
TABLE 3.15: USE CASE DESCRIPTION FOR BAN USERS.....	47
TABLE 3.16: USE CASE DESCRIPTION FOR DELETE USER PRODUCT	47
TABLE 5.1: SIGN UP UNIT TEST.....	94
TABLE 5.2: PASSWORD RECOVERY UNIT TEST.....	95
TABLE 5.3 INTEGRATION TEST	97

List Of Figures

FIG 1.1: WORK PLAN OF THE PROJECT	16
FIG 2.1: URBAN AGRICULTURAL FARMING PROCESS DIAGRAM.	19
FIG 2.2: ESSENTIAL USE CASE DIAGRAM OF THE SYSTEM.....	27
FIG 2.3: ESSENTIAL USER INTERFACE PROTOTYPING FOR WEB.....	28
FIG 2.4: ESSENTIAL USER INTERFACE PROTOTYPING FOR ANDROID	29
FIG 2.5: COLLABORATION MODEL FOR MANAGE PRODUCTS.	30
FIG 2.6: COLLABORATION MODEL FOR REGISTRATION.....	30
FIG 2.7: COLLABORATION MODEL FOR LOGIN	31
FIG 2.8: COLLABORATION MODEL FOR MANAGE PROFILE.....	31
FIG 2.9: COLLABORATION MODEL FOR PAY FOR SERVICE.	32
FIG 3.1: USE CASE DIAGRAM FOR ADMIN	39
FIG 3.2: USE CASE DIAGRAM FOR USER (BUYERS)	39
FIG 3.3: USE CASE DIAGRAM FOR USER (FARMER).....	40
FIG 3.4: ACTIVITY DIAGRAM FOR REGISTRATION.....	48
FIG 3.5: ACTIVITY DIAGRAM FOR LOGIN	49
FIG 3.6: ACTIVITY DIAGRAM FOR PASSWORD RECOVERY	50
FIG 3.7: ACTIVITY DIAGRAM FOR PAY FOR SERVICE	51
FIG 3.8: ACTIVITY DIAGRAM FOR MARKETPLACE VIEW.....	52
FIG 3.9: ACTIVITY DIAGRAM FOR MARKETPLACE MANAGE PRODUCT	53
FIG 3.10: REGISTER SEQUENCE DIAGRAM.....	54
FIG 3.11: LOGIN SEQUENCE DIAGRAM	54
FIG 3.12: FORGET PASSWORD SEQUENCE DIAGRAM.	55
FIG 3.13: MANAGE PRODUCT SEQUENCE DIAGRAM.	55
FIG 3.14: MANAGE ACCOUNT SEQUENCE DIAGRAM.....	56
FIG 3.15: PAY FOR THE SERVICE SEQUENCE DIAGRAM.	56
FIG 3.16: CLASS DIAGRAM OF THE SYSTEM	57
FIG 3.17: STATE CHART DIAGRAM FOR LOGIN AND REGISTRATION.....	58
FIG 3.18: STATE CHART DIAGRAM FOR PAY FOR THE SERVICES.	59
FIG 3.19: LOGIN PAGE.....	60
FIG 3.20: USER INTERFACE PROTOTYPING FOR THE SIGN-UP PAGES.....	61

FIG 3.21: USER INTERFACE PROTOTYPING FOR THE VERIFICATION, LOADING, SUCCESSFUL MESSAGE SHOWER PAGES.....	61
FIG 3.22: USER INTERFACE PROTOTYPING FOR THE HOME, DATA, AND MARKETPLACE PAGES	62
FIG 3.23: USER INTERFACE PROTOTYPING FOR THE CALENDAR AND MANAGE ACCOUNT PAGES.	62
FIG 3.24: MARKETPLACE WEB PAGE.....	63
FIG 3.25: MARKETPLACE ADMIN PANEL WEB PAGE	64
FIG 4.1 FOLDER STRUCTURE OF THE MOFER BACKEND.....	67
FIG 4.2 FOLDER STRUCTURE OF THE MOFER FRONTEND ANDROID	67
FIG 4.3: ER DIAGRAM OF THE SYSTEM.....	70
FIG 4.4: TABLE SCHEMA OF THE NORMALIZED DATABASE	73
FIG 4.5: FIREBASE AUTH TABLE.....	74
FIG 4.6: DEPLOYMENT DIAGRAM OF THE SYSTEM.....	75
FIG 4.7: HIGH FIDELITY PROTOTYPE OF THE LOGIN PAGE.....	76
FIG 4.8: HIGH FIDELITY PROTOTYPE FOR THE SIGN-UP PAGES.	77
FIG 4.9: HIGH FIDELITY PROTOTYPE FOR THE VERIFICATION, LOADING, SUCCESSFUL MESSAGE SHOWER PAGES.....	77
FIG 4.10: HIGH FIDELITY PROTOTYPE FOR THE HOME, DATA AND MARKETPLACE PAGES.	78
FIG 4.11: HIGH FIDELITY PROTOTYPE FOR THE CALENDAR AND MANAGE ACCOUNT PAGES.	78
FIG 4.12: MARKETPLACE LOGIN PAGE	79
FIG 4.13: MARKETPLACE HOMEPAGE.....	80
FIG 4.14: ADMIN PANEL	81
FIG 5.1 SIGN IN, LIST ALL SUBSCRIPTION PACKAGES, BANK LOGIN PAGES SCREENSHOT.	87
FIG 5.2: HOME, VIEW PLANTS AND TRANSFER PLANT PAGES SCREENSHOT.....	88
FIG 5.3: DATA, PLANT ACTIVITY AND PLANT RECOMMENDATION PAGES SCREENSHOT.	89
FIG 5.4: APP MARKETPLACE, MY PRODUCTS, AND SETTINGS PAGES SCREENSHOT.	90

FIG 5.5: MOFER MARKETPLACE WEB SCREENSHOT	91
FIG 5.6: MOFER MARKETPLACE PRODUCT PAGE SCREENSHOT.	91
FIG 5.7: MOFER MARKETPLACE MANAGE MY ADS PAGE SCREENSHOT.	92
FIG 5.8 ADMIN PANEL MANAGES MY ADMIN PAGE SCREENSHOT.	92
FIG 5.9: SPECIFIC USER PRODUCT ENDPOINT API TESTING.....	97
FIG 5.10: BANK ORDER ENDPOINT API TESTING.....	98
FIG A.1: ARDUINO C CODE SNIPPET	101
FIG A.3: MOFER PLANT STATUS TRACKING APP DART CODE SNIPPET.	103
FIG A.4: MOFER MARKETPLACE REACT.JS CODE SNIPPET	103
FIG B.1: QUESTIONNAIRE 1.....	104
FIG B.2: QUESTIONNAIRE 2.....	104
FIG B.3: QUESTIONNAIRE 3.....	105
FIG B.4: QUESTIONNAIRE 4.....	105
FIG B.5: QUESTIONNAIRE 5.....	106
FIG B.6: QUESTIONNAIRE 6.....	106
FIG B.7: QUESTIONNAIRE 7.....	107
FIG B.8: QUESTIONNAIRE 8.....	107
FIG B.9: QUESTIONNAIRE 9.....	108
FIG B.10: QUESTIONNAIRE 10.....	108
FIG B.11: QUESTIONNAIRE 11.....	108

List of Algorithms

ALGORITHM 1: DYNAMIC TOKEN GENERATION ALGORITHM.....	83
ALGORITHM 2: DATA ANALYTICS READ FROM MONTHLY DATA AND PLANT STATE CONVERSION.....	84
ALGORITHM 3: PLANT RECOMMENDATION BASED ON THE DATA ANALYTICS.....	85

ACRONYMS

AI: Artificial Intelligence

API: Application Program Interface

APK: Android Application

APP: Application

Auth: Authentication

ER: Entity Relationship

HW: Hardware

IOT: Internet of things

IDE: Integrated development environment

IOS: iPhone Operating System

JSON: JavaScript Object Notation

Mb/s: Megabit per second

NOSQL: not only SQL

OS: Operating System

OTP: One-time password

PNG: Portal graphics format

REST: Representational state transfer

SQL: Structured Query language

SW: Software

UI: User interface

UX: User experience

VS code: Visual studio code

XXS: Cross-site Scripting

Chapter One

Introduction

1.1 Background of the Project

Agriculture is a major source of income and food for humans and animals around the world. With 9 billion people expected by 2050, agricultural food production must double to meet demand. Given limited land, water, and labor, agricultural productivity must increase by at least 25% to meet that goal while limiting environmental pressure [1]. Global food production faces many uncertainties and hurdles, such as maximizing resource use for maximum yield while minimizing waste for sustainable growth. Various technologies and management tools must be integrated to achieve long-term, site-specific production efficiencies while minimizing unintended consequences and impacts.

Smart farming uses modern technology to increase crop yield and quality. 21st-century farmers have GPS, soil scanning, data management, and IoT. In IOT-based smart agriculture, sensors monitor the field and automate the irrigation system. IOT (Internet of Things) in agriculture involves using sensors and other devices to turn farming into data.

Ethiopia's 67–80% agricultural sector has little or no technology. Many farmers use old or traditional methods, which lowers productivity and unbalanced demand and supply. Despite company and government action, "urban agriculture" is a new term.

This IOT-integrated system solves problems. It's a platform where individuals and groups get help with farming and sell their harvested product, making it cheaper on the market. In other words, it benefits both producers and buyers.

This system includes many vegetables. Anyone can grow a plant and sell its products. It makes planting and growing vegetables easier and more efficient by providing harvest data.

1.2 Statement of the Problem

Urban growth, urban poverty, food price hikes, climate change, consumption patterns, and diet-related health problems all call for more attention to how cities feed their residents. Urban growth increases demand for food and ecosystem services from land and water. Urban and rural populations need integrated territorial development and balanced urban-rural linkages.

Ancient agriculture involved producing food in cultivated lands for human survival and animal breeding [2] and was called the traditional agricultural era 1.0. Manpower and animals were used. Agricultural steam engines appeared in the 19th century. The widespread use of agricultural machinery and chemicals by farmers marked the start of era 2.0 and improved farmers' effectiveness and productivity. Due to computing and electronics, Agricultural Era 3.0 emerged in the 20th century. Robotics, programmed farm equipment, and other technologies improved agriculture. Work distribution, precise irrigation, reduced chemical use, site-specific nutrient application, efficient pest control technologies, etc. were used to adapt policies to agricultural era 3.0. The next agricultural era, 4.0, involves the Internet of Things, big data analysis, artificial intelligence, cloud computing, remote sensing, etc. New technologies have improved agricultural activities by developing low-cost sensor and network platforms to optimize production efficiency and reduce water and energy use with minimal environmental effects [3]. Smart farming uses big data to provide extrapolated overviews of real-time agricultural situations [4]. Real-time AI programming embedded in IoT devices helps farmers make the best decisions [5]. Smart farming helps harvesting and crop yields by automating sensors and machinery [6]. The Internet of Things has transformed traditional farming techniques [7].

Ethiopia's food security is a big problem. This is due to rapid population growth, high inflation, urbanization, climate change, depletion of natural resources, and inflation of basic needs. Governments, NGOs, and other organizations must ensure basic needs and promote urban agriculture to overcome these challenges.

No platform provides these services separately, let alone in one system for urban agriculture, making it difficult for farmers to grow it. Even if they grow and harvest it, they must sell it at a lower price due to their market connection.

Generally, these are the reasons why this system is needed:

- Farmers do not get a better yield from their plants and product estimations are not accurate.
- Small-scale farmers are not provided with a good marketplace.
- Data analysis is not done for the products that are harvested.
- Time and consumers waste time and resources.
- The difficulty of getting different services on one platform
- Products are not affordable because of their scarcity.
- Consumers don't have a variety of choices to meet their needs.

current issues This alternative system is proposed. It allows farmers and consumers to quickly and easily communicate from anywhere. This system saves time, money, effort, and energy and helps everyone succeed.

1.3 Objective of the Project

1.3.1 General Objective

The prime objective of this project is to develop IOT based urban agriculture service provider applications for different platform devices.

1.3.2 Specific Objective

The specific objectives of the proposed system are listed below:

- Identify and specify the current urban farming problems in the country.
- Understand the functional and non-functional requirements of the problem.
- Design alternative solutions to address the current problems.
- Promote awareness of the use of technology in city farming
- Develop a platform for urban farming kit providers.
- Develop a marketplace platform for urban farmers where they can connect with interested buyers through our marketplace.
- Develop an app where users can keep track of their cultivation for producing high quality

products.

- Testing and checking the application that is developed and analyzing the data collected from the farmers to get a better yield for another farmer.

1.4 Significance of the Project

The project aims to develop an app that helps users track and control the farm's environment, leading to better use of IOT (internet of things) in a well-controlled environment that can sustain low waste and does not require daily human monitoring.

The significance of this project is:

- Farmers will get a better yield from their plants and harvested products will be consistent.
- The product estimations will get more accurate while the quality of harvested goods will increase.
- Small-scale farmers will be provided with a good marketplace.
- Data analysis is done for the products that are harvested.
- Time and resources wasted by both the farmers and the consumers will decrease.
- The users will get different services on one platform.
- Products will be more affordable because the scarcity of products decreases.
- Awareness of urban agriculture and technology will be created.
- Leads the country by introducing new technology development and adaptation.
- Provide economic growth to Ethiopia through the market and the creation of jobs.

1.5 Feasibility Study

A feasibility study is simply an assessment of the practicality of a proposed project plan or method. This is done by analyzing technical, economic, legal, operational and time feasibility factors. As the name suggests, is designed to reveal whether a project is feasible.

1.5.1 Technical Feasibility

The application's usability, accuracy, dependability, and efficiency, as well as the hardware's capacity to carry out the required work, are what determine the technical viability of our project. Therefore, it can be claimed that the suggested system is theoretically possible for the following

reasons:

- Our system is cross platform and can operate on different devices. Android or IOS.
- It can offer a user-friendly interface.
- Has simple functions.
- Easy-to-use hardware
- Wireless and seamless communication of hardware with software.
- Built with it-experts and self-taught programmers.
- Secure user data.
- Use the application in any place and environment.
- Connect users and customers from different locations.
- We also provide Marketplace via web.

Generally, the system can be said to be technically feasible because of the reasons listed above.

1.5.2 Economic Feasibility

The proposed system is a hardware integrated system, so users must pay a subscription fee in order to access the tracking system. This subscription fee enables the proposed system to pay for server and equipment costs, allowing it to run effectively and, in turn, providing the user with dependable service that leads to better outcomes and helps the customer also generate income.

- Hardware costs are intermediate considering the service.
- Server costs are covered by the money generated from subscriptions and advertising.
- It economically benefits urban consumers by sufficing their own consumption.
- It can be an income generation method by selling and buying products on the system.

1.5.3 Operational Feasibility

Each member of the "MOFER" project team has a different experience and level of programming expertise, supporting the system's operation.

"Agile software development technique" will determine the project's operation mode. This makes the system flexible and changes requirements based on need and problem. And we can say the proposed system is operationally feasible because of the reasons below:

- The project follows agile software methodologies that suit the requirements and complexity of the proposed system.
- Operates on all devices (cross platform)

1.5.4 Legal Feasibility

Writing this documentation, the proposed system has no legal implementation restrictions, such as data protection acts or social media regulations, a project certificate, a license, or copyright. The proposed project is lawful and ethical.

1.5.5 Schedule Feasibility

This document ensures that the project will be completed within the specified time frame. If possible, this project will be finished in the allotted time at the end of this chapter.

1.6 Beneficiaries of the Project

1.6.1 Direct Beneficiaries

The proposed system will benefit customers, farmers, and the team directly. The team members are the first direct beneficiaries because the system is subscription based. The team members will have economic and job opportunities. thus, making them economically.

The next beneficiary are farmers. It allows people to market their cultivated products and benefit economically. They can also cultivate their own food, reducing their expenditure on items.

The other direct beneficiaries of this system will be the customers, who will have the opportunity to choose the product they want freely and access it with less time and energy loss.

1.6.2 Indirect Beneficiaries

The proposed system will benefit farmers' and team members' families. When a family member's employment supports them financially, they'll profit.

The country will also benefit from the proposed system through the job opportunities that are created. The other way the country could benefit is through the telecom company, which, by providing the internet infrastructure, will generate more revenue from system users buying internet packages.

1.7 Methodology

This system uses applied research to identify a problem and produce a solution. It helps us identify issues and needs, which helps us develop hardware and software solutions and strategies.

It helps us judge by revealing unaddressed system gaps. The proposed method collects qualitative and quantitative data for experiments.

1.7.1 Data collection

The data collection methods that will be used by the system are:

- Interview
- Survey
- observation

Prior research and applications of smart agriculture in urban Ethiopia or other countries are analyzed to determine the project's pros and cons. We study agricultural service providers and institutions like the Ministry of Agriculture, which is modernizing the system with vertical farming, city farming, and technological integration.

1.7.1.1 Interview

Agriculture, as a qualitative research method, we ask, "What is agriculture's biggest problem?" Why is Ethiopian city farming rare? Integrating technology is difficult. Also: The proposed app uses semi-structured interviews with a general plan for what to ask and no specific phrasing order. Semi-structured interviews offer freedom while following a predetermined theme framework. We rely on previous outcomes to ensure the generalizability and validity of our results.

1.7.1.2 Survey

In order to collect information about agriculture services and products, we will conduct surveys with a small number of carefully chosen organizations and groups that only benefit the proposed system.

1.7.1.3 Observation

We also use observation as a data collection method by observing the behavior, events, and current situation of the agriculture sector in Ethiopia, particularly in cities.

1.7.2 System Design and Analysis Tools

We'd use context diagrams, E-R diagrams, use case diagrams, flow charts, and more for this project. As system analysis tools, we'd use data flow diagrams, bubble charts, and simulations.

System design and analysis for this project improves previous systems or creates new systems by evaluating urban farming methods to find an optimal solution.

1.7.3 Development Environment and Programming Tool

1.7.3.1 Software Development Process Model

Agile methodology is a software development process framework that adopts the iterative approach, open collaboration, and process adaptability throughout the life cycle of the project. This iterative agile approach is more flexible and its short time-span iterations seek improvement for the project in small release, with minimal planning, rather than plan at length. This helps to minimize the overall risk and allows the project to adapt to changes more quickly [8]. Agile methodology is an alternative method for traditional project management, typically used in software development. It helps teams respond to unpredictability through incremental, iterative work cadences, known as sprints. Agile methodologies are an alternative to waterfall, or traditional sequential development [9].

1.7.3.2 Development Environment and Devices

The development environment is listed in table below.

Table 1.1: Development environment used in the project.

WORKING ENVIRONMENT AND PLATFORM	
Location/Platform	Description
School	Free Classrooms, labs, and school Garden for Meeting, sharing ideas and programming.
Coworking space	Coworking areas Creative Hub, Home for programming and sharing ideas.
GitHub	It's a platform that can we use for version control and collaborating with team members easily

Google Meeting	It's a platform that we can use for remote meetings.
Slack	It's a platform that can we use for sharing messages and documents between team members easily

Table 1.2: Computing Devices used in the project.

COMPUTING DEVICES			
Devices	Description	Qty	Specification
Laptop	For application development and sensor configuration purpose	6x	Mac OS, win 11, Win 10 above core i5 and each contains a minimum 8 GB Ram.
Desktop	For Internet and File server	1x	Win 10 Dual core with 12GB Ram.
Smart Phone	For testing and deploy the applications	2x	Android and IOS Operating system based smartphones
IOT	For data collection purposes.	1x	Arduino Uno microcontroller Board with sensors like Humidity Sensor, Temperature sensor, Moisture sensor, water level sensor etc....
Router	For internet connection	1x	D-LINK DWR-933
Backup devices	For safety purpose we backup our projects often	1x	1TB HDD Hard disk
DOCUMENTATION APP			
APP	Description		
MS-Word	Used for writing the proposal and documentation		

Google Doc	Used for writing the proposal and documentation
DESIGNING SOFTWARES AND PLATFORM	
FIGMA	For designing the application UI.
Draw.IO	For designing the UML Diagrams

Programming tools

Paul Solt, an iPhone app expert, said, "Apps that weren't built for cross-platform use would require a ton of re-engineering." What are native and cross-platform apps? Native apps target a specific OS. To develop an Android native apps, we might use Kotlin or Java. To develop an iPhone native app, we could use Swift or Object C. For both mobile operating systems, we may need to re-engineer and develop separate apps. Cross-platform app development creates apps from a single code base. Native apps are fast and responsive because they're designed for one OS and compiled with its language and APIs. Machine dependability limits their reach. High-performance apps that don't require re-engineering can reach a large audience quickly.

Our system is going to use the following tools:

Table 1.3: Tools used in the project.

DATABASE	
Database	Description
MYSQL	MySQL is a relational database management system based on the Structured Query Language, which is the popular language for accessing and managing the records in the database. It is supported by Oracle Company.
Firebase (NoSQL)	Firebase is a scalable cloud solution provided by Google, where data is saved on a collection into Cloud Storage or Real Time database
FRAMEWORK	
Framework	Description

Express JS	Express JS is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It is used on the backend. It uses a programming language called JavaScript
Flutter	Flutter is an open-source framework by Google for building beautiful, natively compiled, multi-platform applications from a single codebase by using a programming language called Dart.
React JS	React is an open-source JavaScript library developed by Facebook. It is mainly used for building user interfaces for single-page Web applications and mobile applications.
RUNTIME ENVIRONMENT	
Runtime Environment	Description
Node JS	Node.js is an open-source server environment. Node.js allows you to run. JavaScript on the server.
API ARCHITECTURE AND DATA INTERCHANGE FORMAT	
Name	Description
REST API	A REST API is a way for two computer systems to communicate using the HTTP technologies found in web browsers and servers.
JSON	JSON is an open standard file format and data interchange format that uses human-readable text to store and transmit data objects consisting of attribute–value pairs and arrays.
INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)	
IDE	Description
Arduino IDE	Its Arduino Integrated Development Environment designed for configuring the Arduino easily.
Android Studio	Android Studio is the official integrated development environment for Google's Android operating system, built on JetBrains' IntelliJ IDEA software and designed specifically for Android development.

VS code	Visual Studio Code is a code editor redefined and optimized for building and debugging modern web, mobile apps and cloud applications
---------	---

1.8 Scope and Limitation of the Project

1.8.1 Scope of the Project

The proposed system aims to create an all-in-one service platform for smart and automated city farming for any individual or community with smartphone skills. The system is an easy online platform that works in urban Ethiopia to create smart and automated city farming for any individual or community.

It will be cross-platform, integrated with IOT devices, and allow automated data manipulation.

The system scope focuses on:

- The customers register into the system with their smartphones online.
- The customers can access the two parts, which are the tracking part and the marketplace.
- The tracking part will have the PH value, humidity sensors, water level sensors, temperature, and other tasks.
- They can track the progress online and get notifications about the plant's condition if they need any treatment.
- In a marketplace, they can buy and sell products, so users of our app will be able to sell their products on the marketplace.

1.8.2 Limitation of the Project

- The system interactivity will not be accessible for blind persons.
- Not able to implement machine learning.
- Not able to implement a learning platform for the users.

1.9 Risk, Assumption, and Constraints

1.9.1 Risk

The project's languages, systems, concepts, and components lack a common ground for clear communication. Ergo, we must build a middleman API communication system for our client-side product to talk to our "less footprint" hardware. Predictions based on data sets, risk management,

and assumptions not driven by ambition but by measurable steps adding constraints or holdbacks with possible solutions are expected from the team.

1) Failure in the communication protocol

As mentioned above, our application needs to communicate via API. Miscommunication will result in our vision of the system and product severely failing.

These unfortunate miscommunications could be.

- Misconfigured data

This is a risk that could be caused if the wrong plant has been configured on the hardware or if the client plants the wrong seeds and follows procedure, which could lead to

- Wrong sensory reading

Also, a possible risk to account for is that our system needs accurate readings from sensors to decide what to do next. Getting the wrong readings will cause problems.

- Having the wrong data set for a plant

Due to time constraints, it will use online critics and botany research to gather plant data (how much water, sunlight, etc.). The transitive property states that if two quantities are equal to a third quantity, then all quantities are equal.

In simpler terms, our product is only as good as the latest data and research we are referring to. A mistake made by our source is a mistake in our system, which is a risk in our scope of risks.

2) Maintaining the integrity of data during transmission (security risk)

"Man in the middle" is a serious security issue. After spoofing, XSS could send fake data signed by a trusted key (listening to the communication between the SW and HW).

3) Not being able to afford the materials.

1.9.2 Assumptions

- All team members have a clear understanding of the goal and possible vision of the product.
- All members of the team will deliver their fair share of the load on time.
- A good internet connection during development
- Clear team communication to avoid conflict.
- Team meetings, including our advisor. to discuss the overall progress of the project.
- Loss of power from the Arduino during transmission will not happen.

1.9.3 Constraints

- Time
- Budget
- Scope
- Not able to get the data set.

1.10 Organization of the Project

The study's introduction is covered in Chapter 1. This covers the background of MOFER, statement of the problem, alternative solutions, objectives of the problem, significance of the project, beneficiaries of the project, scope and limitations of the project and methodology & environmental tools of the project.

Chapter two describes business area analysis about the current system and requirement definitions like identifying functional requirement, non-functional requirement and actors.

Chapter three incorporates object-oriented analysis and design. It covers UML diagrams like structural diagrams and behavioral diagrams. Those are system use cases, collaboration modeling, conceptual modeling, deployment diagram, and sequence diagrams. It also consists of system designs like architectural designs, data model and user interface prototyping.

1.11 Work Plan

Table 1.4: Work Plan of the project

#	Activity	Duration	Deliverable
1	Project Proposal		Feasible project proposal (Hard and soft copy)
	Project Planning and Identification	5	
	Feasibility study	3	
2	Requirement Analysis		Analysis documentation and questionnaires. (Hard and soft copy)
	Identify major functions and process	7	
	Primary source gathering	10	
	Analyzing the proposed system	8	
3	Object oriented analysis		System Models (Hard copy and PNG)
	System use case modeling	9	

	Low and high-fidelity UI prototype	10	
	Collaboration Modeling	3	
	Sequence diagram	4	
	Class diagram	3	
	Data model	3	
	ER diagram	1	
	Activity diagram	3	
4	Object oriented implementation		Database • SQL, Firebase (NOSQL) and JSON API • REST API using Node.JS. Arduino configuration • C++ Mobile Application • ANDROID APP • IOS APP
	Coding • Database • API • Arduino configuration • Mobile applications	45	
	Deployment	10	
	Testing • Unit and migrate testing. • User testing	15	
	Orientation and documentation	5	

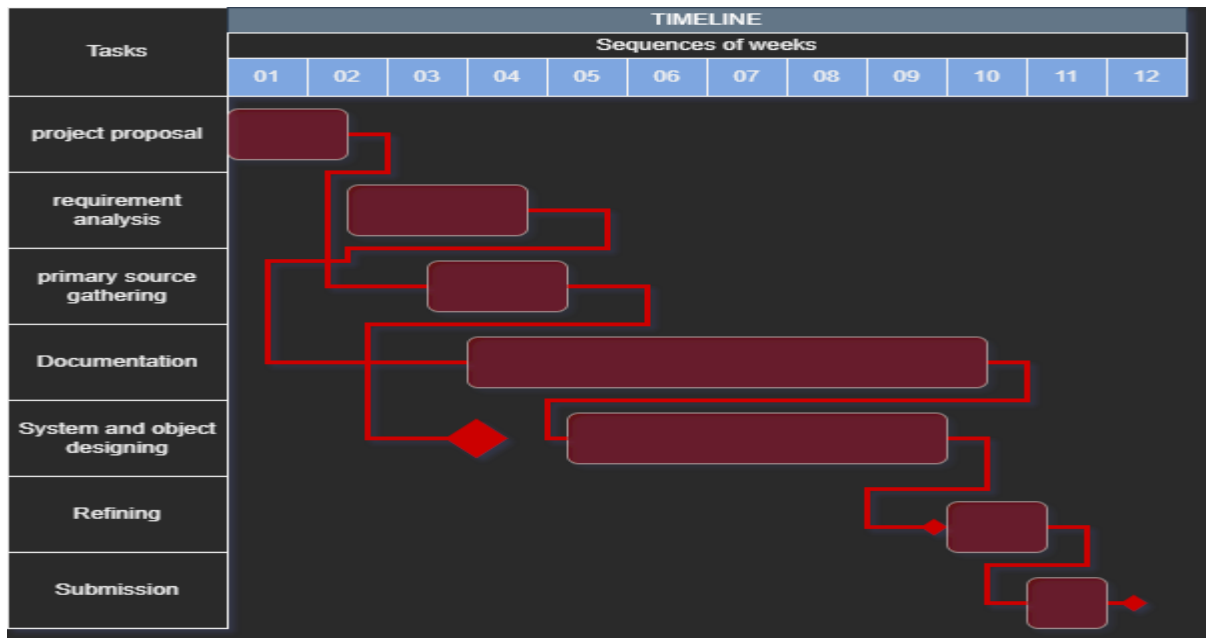


Fig 1.1: Work plan of the project

1.12. Budget Allocation

Table 1.5: budget allocation of the project

List Of Items	Price
Breadboard	300
Jump Wire	100
Node MCU	850
DHT22 Sensor	600
Soil Moisture Sensor	400
Paper and CD	2000
Transport and Internet fee	1000
Sheard Host Server	2000
TOTAL	7250 Birr

Chapter Two

Business area Analysis and requirement definition

2.1. Introduction

In this chapter, a business requirement analysis and requirement definition of the existing system as well as a comparison between the existing system and the proposed system will be briefly explained, along with documentation of each step and procedure.

The prior chapter provides some insight on which method of requirement gathering is used to collect data from a variety of users and organizations. In addition, within this chapter, we provide a concise explanation of the rationale behind our decision to use the method of requirement gathering that was outlined previously, as well as an explanation of and documentation for the proposed system's functional and non-functional requirements, use case models, and actor and communication descriptions.

2.2. Business Area Analysis

The proposed system could potentially be used in various sectors of the agriculture industry, including small-scale farms, large-scale commercial farms, and greenhouses. This type of app could be used to track the status of various types of crops, including fruits, vegetables, herbs, and ornamental plants and our main focus is on small-scale farms.

By providing farmers and growers with real-time data on the status of their crops, the proposed system could help to improve efficiency, productivity, and profitability. It could also be a useful tool for researchers and educators, helping to improve our understanding of plant growth and development in the future.

2.2.1. Detailed Analysis

We attempted to conduct a thorough study of the proposed system's demand and problem-solving capacity to get a detail analysis of the proposed system and its significance, in order to alter or drop elements that were not essential. And as mentioned in chapter one we used different data collection technique such as questionnaires, interviews and surveys. We used to google forms to collect our surveys and we have questioned 96 people, 83.3% men, 16.7% women with age range of 18-40 which 24% are farmers. According to the survey we were able to identify that 60.4% used

to track the status of their plant by visually inspecting them, 39.6% by using paper journals. We have asked what type of information the user would like to track about his/her plant and got varying response mostly 99% of users choose to get the watering schedule information of their plant while moderately 41.7% choose to get climate conditions. We asked if there are any specific challenges, they face in terms of tracking the status of their plants currently and got a response that 37.5% are facing challenges on soil and nutrient management while 26% face challenge on tracking weather conditions. 96 of the people on our survey choose to receive watering reminders, 95% choose fertilization reminder, 94% choose pest control reminder and 95 choose to receive general plant care tips. And 53.3% from 96 responses choose to receive a weekly status update of their plant while 26% choose to receive Daily updates. We have also asked if they are interested on selling plants or gardening supplies through the app and got a 100% response saying yes which assures the use of our marketplace. We have tried to ask if there any other features users would like to see in the app for tracking the status of their plants and according to the data, we got users requested to have an automated irrigation system.

We also tried to reach out to the office of agriculture ministry of Ethiopia but unfortunately could not get a proper meeting schedule, so we instead contacted Digital green Ethiopia which is a global development organization that empowers smallholder farmers to lift themselves out of poverty by harnessing the collective power of technology and grassroots-level partnerships. We were able to contact the communications manager of DGE she was kind enough to brief us about their work and allowed us to ask some questions and interview.

2.2.2. Current system

Even though we are concentrating on urban farming, it is arguable that there is only a small amount of implementation due to the many different factors and reasons. The current method of agriculture that is being utilized in Ethiopia is primarily manual and requires a significant amount of labor. Additionally, the farmers have limited access to data that could improve their crop yields.

For instance, in Addis Ababa, the implementation of urban farming is low due to a lack of vast fertile area and land for farming, as well as a lack of awareness of the use of urban agriculture. However, it is now common to see vegetables such as salads being planted near rivers using the available space despite the inefficiency [10]. But in our nation, policies for the application of

various agricultural techniques such as vertical farming, hydroponic farming, greenhouse farming, and aqua phonic farming have only recently been developed in order to meet the needs of the population that resides in urban areas. These policies were developed in order to meet the demand for food that is generated by the growing number of people who are choosing to live in cities. Vertical farming is gaining popularity in cities all over the world as a method to achieve self-sufficiency in food production and even create cash.

The majority of farmers who do not use modern crop monitoring methods do not have access to analytical data. This is due to the fact that only a minority of farmers employ modern crop monitoring methods.

The following are some of the problems inherent in the existing system:

- There is a significant amount of manual labor involved.
- There is no regulated method for ensuring the product's quality during the growing process.
- There is no utilization of any cutting-edge technology.
- Lack of Agricultural Marketing
- Unknown soil nutrient composition and PH scale

The Process of the current agriculture system in urban and ruler areas in Ethiopia is more of traditional technique and lacks efficiency.

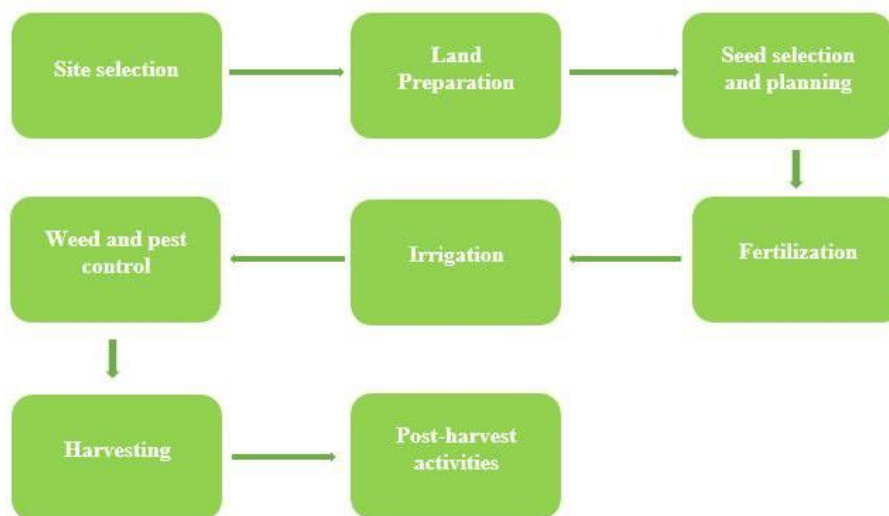


Fig 2.1: Urban agricultural farming process diagram.

The above diagram depicts the present urban agricultural farming process in Ethiopia. This method is labor-intensive and subject to numerous limits, and its current execution is plagued by a number of negative aspects that have a negative impact on productivity, health, and yield.

1. Land preparation: Farmers may use hand tools or animal power to till the soil, remove weeds, and prepare the land for planting.
2. Planting: Seeds are sown by hand or with the use of simple tools such as hoes. In some cases, farmers may also use traditional techniques such as seed balls or seed pouches to plant their seeds.
3. Weeding: As the crops grow, farmers may need to remove weeds by hand or with the use of simple tools such as hoes.
4. Irrigation: Depending on the location and climate, farmers may need to water their crops manually or with the use of simple irrigation systems such as bucket chains or treadle pumps.
5. Fertilization: Farmers may use natural fertilizers such as compost or animal manure to enrich the soil and improve crop yields.
6. Pest and disease control: Farmers may use natural methods such as crop rotation and intercropping to help control pests and diseases, or they may use traditional remedies made from plant extracts or animal products.
7. Harvesting: Crops are typically harvested by hand, using tools such as sickles or scythes.
8. Post-harvest activities: After the crops are harvested, farmers may need to dry, thresh, and winnow the grains to remove the husks and chaff. The grains may then be stored for later use or sold at local markets.

2.2.3 Player of the Existing System

The primary participants in the system that is now in place are **Government, Community organizations, Farmer, Middleman, Traders and Consumer**. All of these individuals are a part of the agricultural supply chain in Ethiopia, and they all play an important and significant role.

As the primary actor, **the government** provides farmers with improved production methods and support services. Furthermore, the government assists farmers by allocating funds and delivering low-cost fertilizer.

In the agriculture supply chain, **the farmer** is responsible for producing the raw materials that are used to create food and other products. This includes crops such as grains, fruits, and vegetables, as well as livestock such as cows, sheep, and chickens. The farmer sells these products to wholesalers and distributors who then distribute to retailers, who in turn sell them to consumers. The farmer plays a vital role in the agriculture supply chain as they are responsible for producing the raw materials that are used to create the food that we eat and other products that we use on a daily basis.

The presence of **the middleman** in the supply chain has an effect on the price variance of particular commodities, which in turn contributes to inflation. On the other hand, the role of the middleman in connecting smallholder farmers with dealers and the end consumer market is a very significant one. And as a link in the distribution system, **the trader** is responsible for acquiring goods from the intermediary or suppliers and reselling them to end users.

The consumer is the end user of manufactured goods and services. This means that consumers are extremely important to any country's economic structure. Without consumer demand, manufacturers would have less of a reason to create goods or yield products.

Community organizations are groups of individuals who come together to engage in urban agriculture activities, either for personal consumption or to sell their products. These organizations may be informal or formally registered.

2.2.4. Proposed System

The proposed system places its primary emphasis on agricultural endeavors, more specifically on urban farming and the application of technology in the service of producing superior goods. The purpose of this system is to provide a workable solution by building a hybrid system composed of both software and hardware components. The goal of this system is to facilitate enhancements to the current business model and the farmers' operational procedure.

The agricultural sector is the primary contributor to the economy of Ethiopia, and in order to keep up with the growing demand in this sector, there needs to be a move toward the digitalization of

agricultural methods and techniques, with an emphasis on urban agriculture. This move is necessary in order to keep up with the growing demand in this sector. The implementation of the solution that we have suggested makes a contribution to the digitization of data analysis and marketing in the agricultural sector by:

- Improving crop yield
- Improving soil quality
- Increasing agricultural input control
- Increasing data quality
- Maximizing the use of small spaces while incorporating IoT
- Obtaining real-time data and information

In the previous section, we demonstrated that the proposed system makes data analysis easier. Additionally, the system that is being suggested will create a user- and publicly accessible marketplace in which farmers will be able to trade their goods. As a consequence of this, the distribution chain might reap the benefits of a reduction in the number of middlemen that are involved in the transaction. On the other hand, the system that we propose will only function as a platform on which vendors can post advertisements of their items for sale and buyers can make purchases.

The proposed system is going to be integrated with a variety of hardware devices, including sensors and other monitoring equipment. These devices are being used to collect data on plant conditions such as temperature, humidity, soil moisture, and PH value. This data is critical for understanding the health and well-being of plants, and it is used to make informed decisions about irrigation, fertilization, and other management practices.

The hardware is being integrated with the plant status tracking system through wired and wireless connections. Data collected by the hardware is transmitted to the system in real-time or near-real-time, and it is processed and analyzed by the system to provide insights and recommendations to users. The system is also configured to provide alerts or notifications if certain thresholds or conditions are met, such as if soil moisture levels drop below a certain point or if high soil PH value is detected.

The integration of hardware with the plant status tracking system provides a number of benefits. It allows for the collection of more accurate and reliable data on plant conditions, as the hardware is able to collect data from a larger number of plants or locations. It also enables real-time or near-real-time monitoring of plant conditions, which allows for timely intervention if issues arise. Additionally, the hardware integration is expected to provide cost savings and other financial benefits, as it reduces the need for manual inspections and other labor-intensive tasks.

Overall, the integration of hardware with the plant status tracking system is an important development that is expected to greatly improve the accuracy and reliability of the data collected, as well as the efficiency and effectiveness of plant management. Below is the process and step that the proposed system takes to perform its intended task:

1. Setup and configuration: To use the plant status tracking app, users will need to install the app on their device and create an account. They may also need to configure the app by entering their personal and plant-specific information, such as the types of plants they are growing.
2. Data collection: The plant status tracking app uses various methods to collect data on plant conditions, such as sensors and manual input. This data is then transmitted to the app and stored in the user's account.
3. Data analysis and recommendations: The plant status tracking app processes and analyzes the data collected to provide insights and recommendations to users. These might include recommendations for irrigation, fertilization, pest control, or other management practices.
4. Monitoring and alerts: Users can use the plant status tracking app to monitor their plants and receive alerts if certain thresholds or conditions are met. For example, the app might alert users if soil moisture levels drop below a certain point.
5. Record-keeping and reporting: The plant status tracking app provides users with a record of their plant data and management activities, which can be useful for tracking progress and identifying trends over time. Users can also use the app to generate reports or charts that provide an overview of their plant data.

Overall, the proposed system is useful for collecting, analyzing, and managing data on plant conditions and management practices. By using the app, users can make more informed decisions about how to care for their plants, and they can track their progress and identify areas for improvement. The marketplace on the other hand can be used as a place to sell those products that have been produced with the help of our system to customers and the demanding population serving as a platform where our users can sell their urban grown products and also participate users who do not use our system.

2.3 Requirement Gathering

2.3.1 Requirement Gathering Techniques

Techniques our team used for obtaining requirements included both qualitative and quantitative research to learn as much as possible about users' goals, obstacles, and preferences.

Questionnaires: Potential app users were surveyed using questionnaires created by our team. Users were asked about the kinds of features they'd want to see in the app, the difficulties they've encountered so far, and the reminders they'd find most helpful. And a total of 92 individuals filled out the survey after it was circulated using Google Forms.

Interviews: Our team also conducted an interview with Helen Gebregiorgis, the communications manager of Digital Green Ethiopia, in order to get insights on their work and determine how the suggested app may be useful in the agricultural industry.

Surveys: Our team was able to gather data on the preferred way of monitoring plant status, the sort of information users would want to track, and the obstacles they presently encounter in tracking the state of their plants.

Our team also conducted market research to identify the competition and market trends in the plant tracking and gardening supply marketplace.

2.4 Method of Communication

2.4.1 Communication Techniques

When it comes to the techniques of communication, our team tried to conduct a face-to-face meeting with the agriculture ministry to make an interview but could not manage due to the tight schedule of the office and our team used the following communication methods to gather information from potential user's and markets: Emails and voice call interviews.

And while doing this project the team have used many communication methods to analyze assets and implement ideas. For example, we used video conferencing and online collaboration tools to work remotely and facilitate communication between our team. We also had face-to-face sessions in order to work and collaborate with each other.

2.5 Requirement Definition

2.5.1. Functional Requirement

User (farmers)

- ✓ Login
- ✓ Register
- ✓ Manage Account
- ✓ Pay for service.
- ✓ Generate report.
- ✓ View product (for marketplace)
- ✓ Upload product (for marketplace)
- ✓ Manage product (for marketplace)
- ✓ User's password recovery
- ✓ Logout

User (new sellers)

- ✓ Register
- ✓ Login

- ✓ Manage Account
- ✓ View product
- ✓ Upload product
- ✓ Manage product.
- ✓ User's password recovery
- ✓ logout

User (buyers)

- ✓ View product
- ✓ View seller information

Bank

- ✓ Make transfer payment.
- ✓ Check transfer payment.
- ✓ Approve payment.
- ✓ Disapprove payment.
- ✓ Generate error.

Administrator

- ✓ Login
- ✓ Create admin.
- ✓ Manage admin information.
- ✓ Ban/terminate users.
- ✓ Admin's password recovery
- ✓ Delete user's product.

2.5.1.1. Essential Use Case Diagram

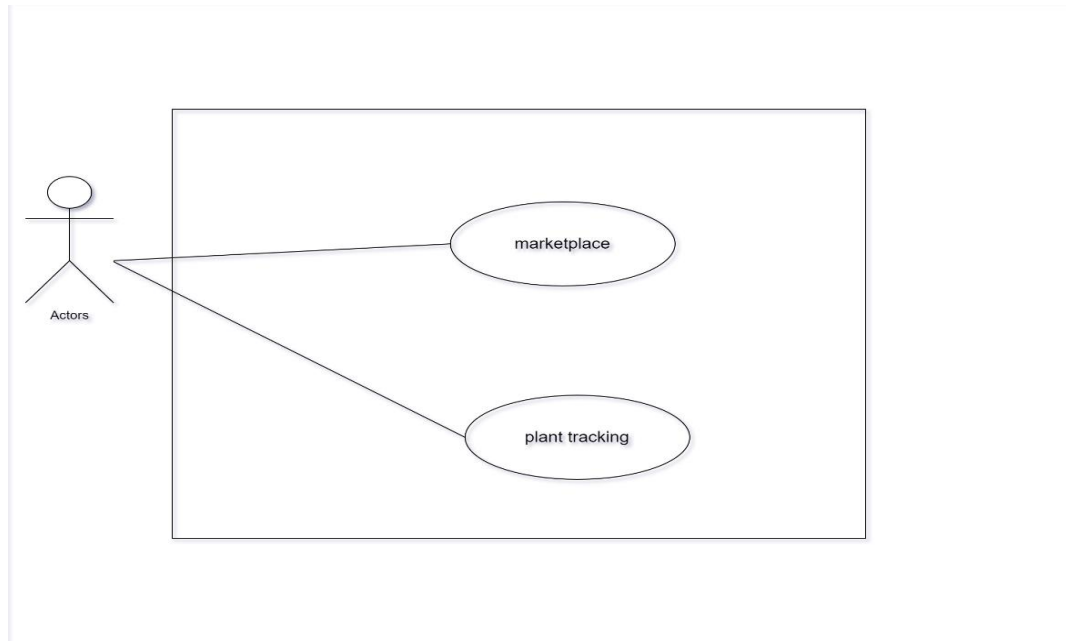


Fig 2.2: Essential use case diagram of the system

In the above diagram, the Plant tracking use case consists of different functionalities such as plant tracking, data analytics about the tracked plant, pay for services, manage account, manage product and so many. Marketplace use cases consist of different functionalities such as post product, manage product, manage account and so many.

2.5.1.2 Essential User Interface Prototyping

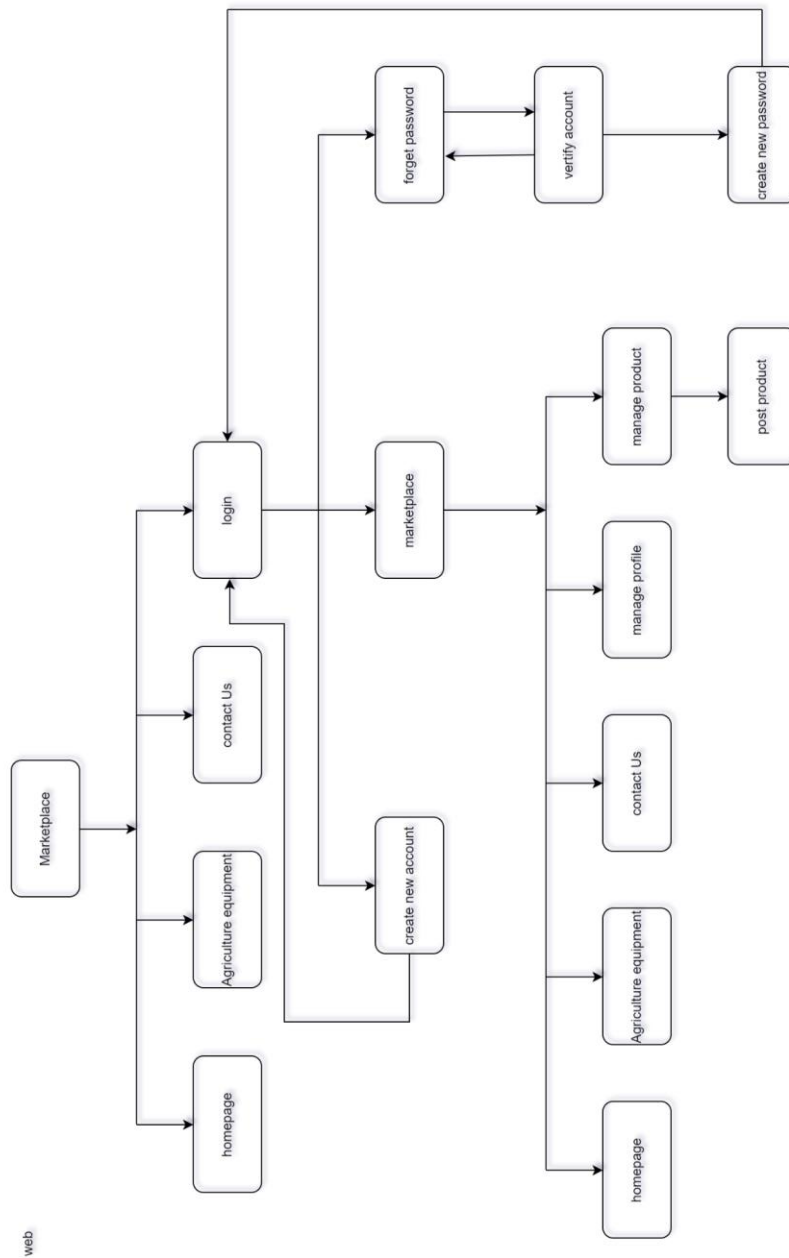


Fig 2.3: Essential user interface prototyping for web

android

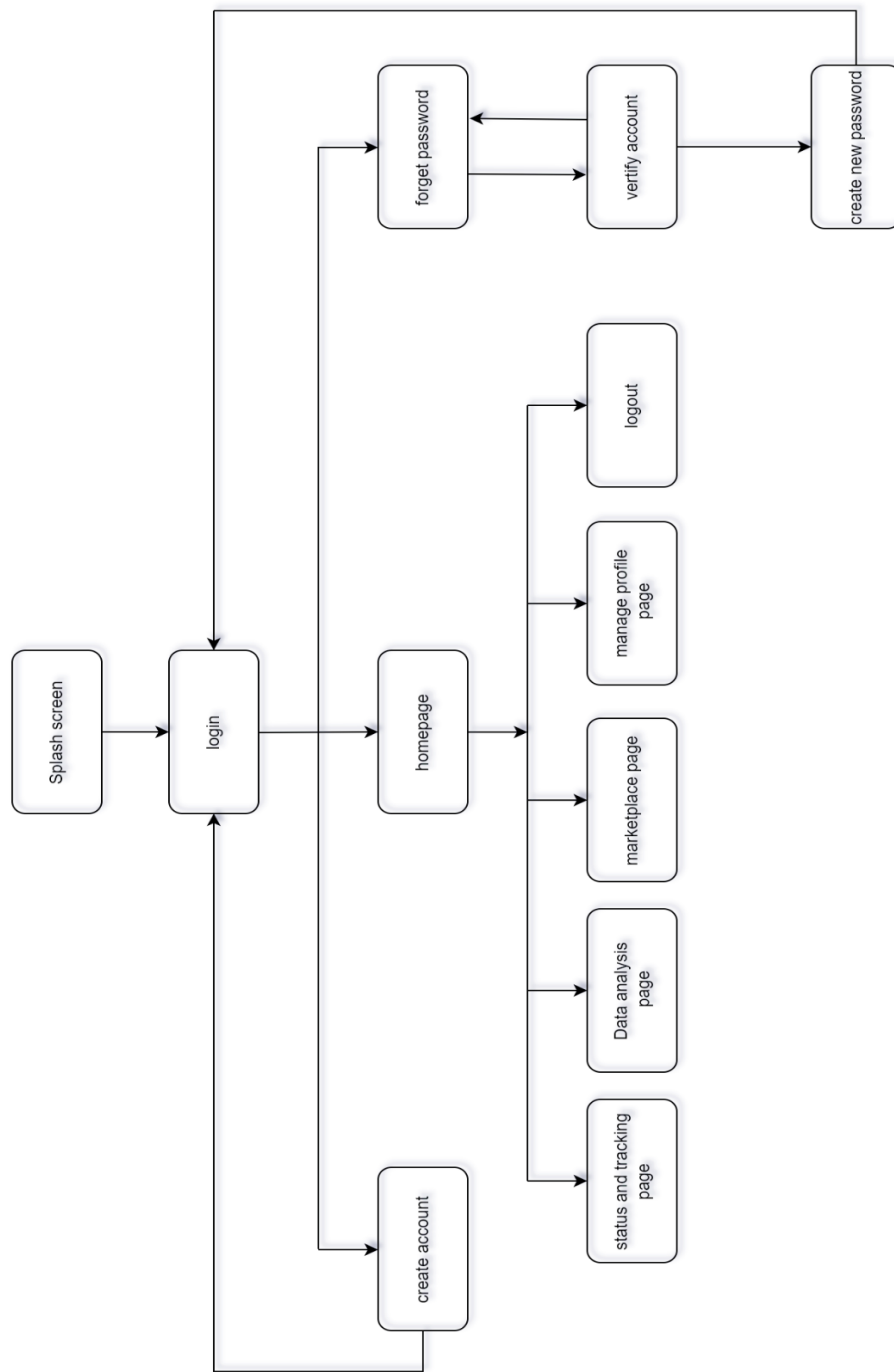


Fig 2.4: Essential user interface prototyping for android

2.5.2 Collaboration Modeling

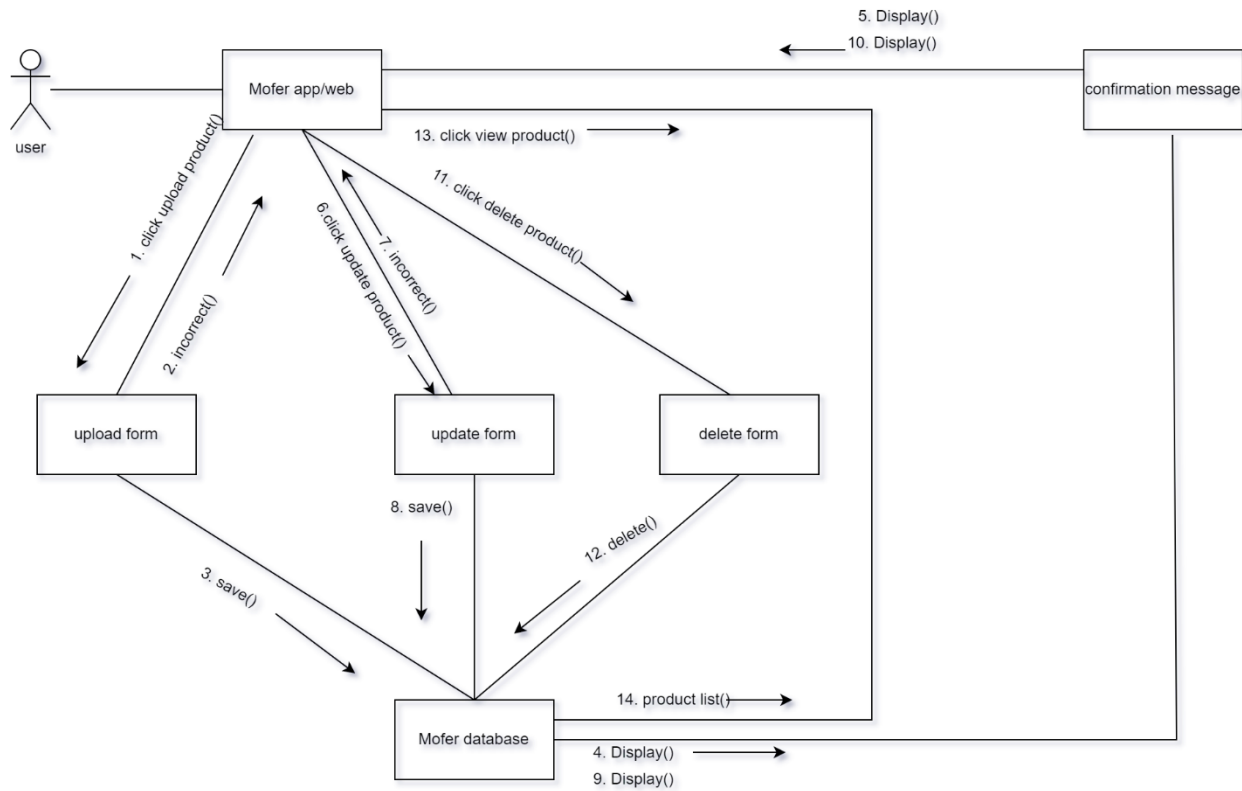


Fig 2.5: collaboration model for manage products.

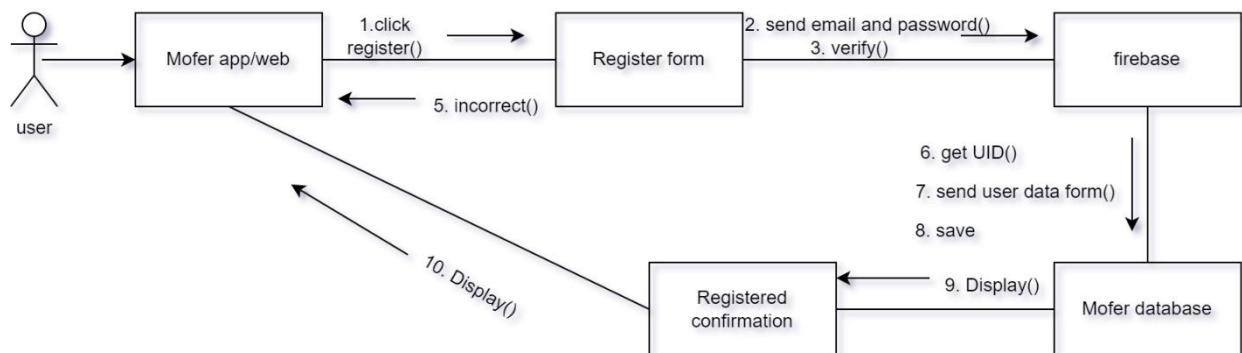


Fig 2.6: collaboration model for Registration

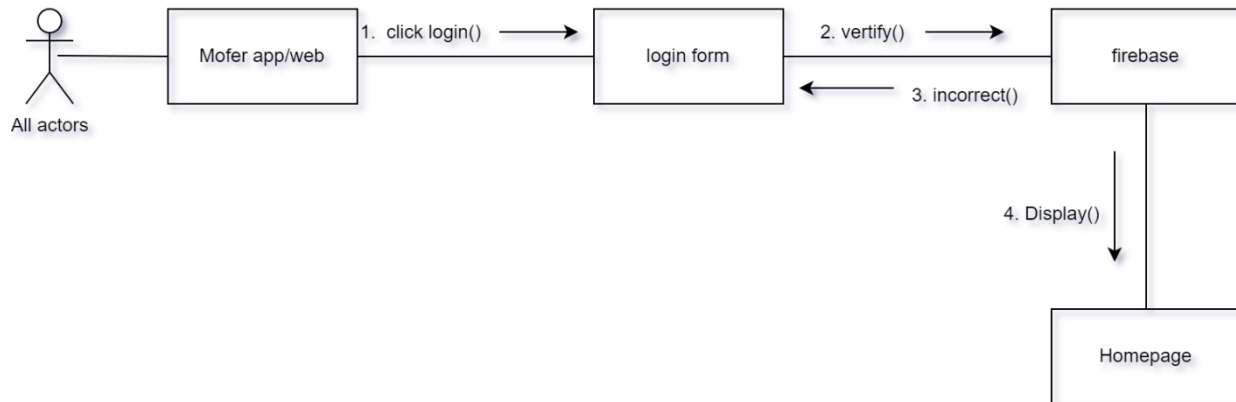


Fig 2.7: collaboration model for Login

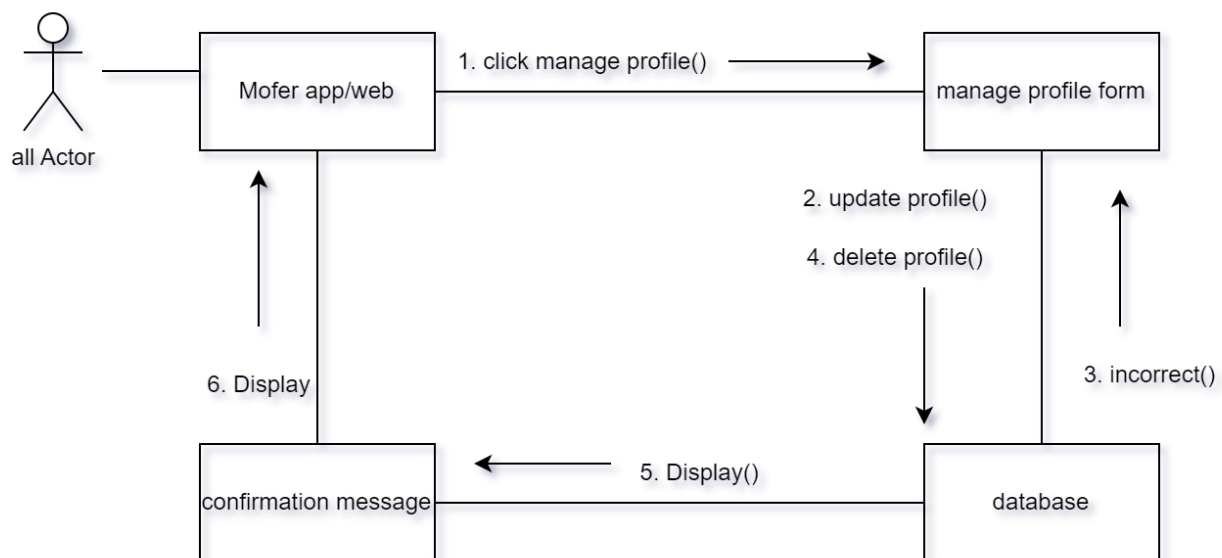


Fig 2.8: collaboration model for Manage profile.

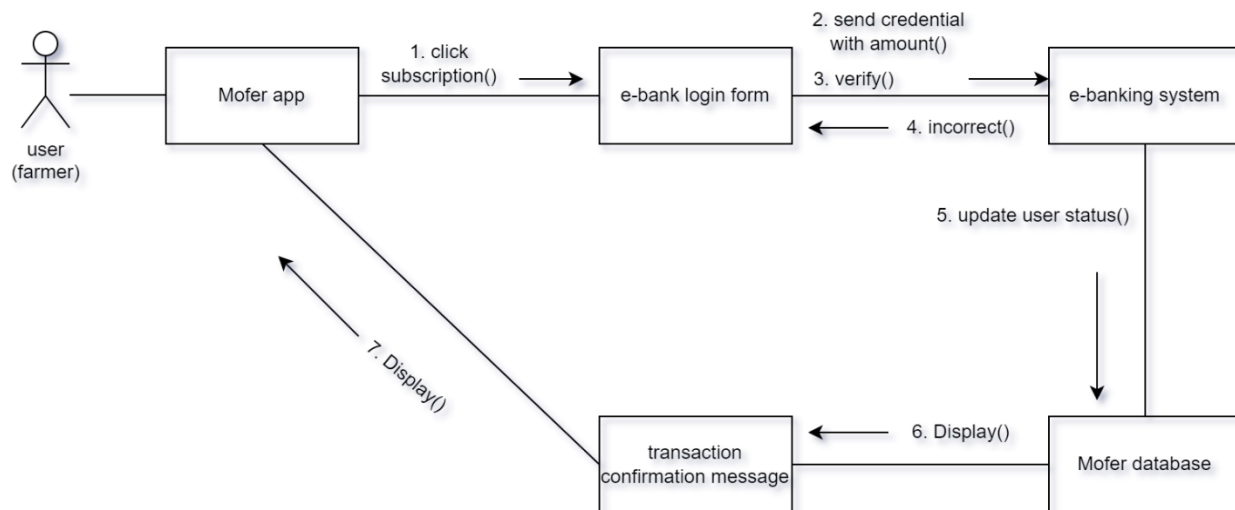


Fig 2.9: collaboration model for Pay for service.

2.5.3. Non-Functional Requirement

Scalability: the number of concurrent users in the system can be greater than two so that multiple users at the same time can order service.

Portability & Responsiveness: the system can work on devices supporting either Android device which version is above marshmallow (API 23) or OS device.

Security: All the user data like personal contact, payment methods should be encrypted and should not be accessible to unauthorized personals like locking a way for users to manipulate the app by bypassing. Also, it should use two types of authentications, email & OTP.

Extensibility: Features can be enriched, hence ensuring version control by carrying out several development iterations to add new service categories at a later stage.

Performance: the system will use proficient algorithms for each task, requiring less storage and fast response. When the user opens the app, the app should load the services menu within 5 seconds with all vector icons.

UX Interface: The system should have an easily navigable, device considerable, and understandable interface that users can interact with the system easily.

Accuracy: The system should give only a valid result the system should not give an invalid response.

Maintainability: The system can be managed most efficiently by using object-oriented software development techniques. If there raise additional requirements, the system is flexible to change on the iteration phases.

Availability: users can install the app from the official website or redirect to the store (App Store or Play Store) to download, rate the app, and contact the developers.

Error Handling: The system will check user inputs to the system to handle the error. Every error will be handled and returned with appropriate error messages.

Chapter Three

System Modeling

3.1 Introduction

This chapter covers the technical aspects of the proposed plant status tracking system and online marketplace, including the identification of the user interface (UI) and business rules. It discusses the design and implementation of the Android and Web UI, as well as the layout, navigation, and visual elements used to create an intuitive and user-friendly interface. Additionally, it highlights the guidelines and requirements that have been established to govern how the system operates and makes decisions, including scalability and flexibility, integration with other systems, and cost-effectiveness. It also covers more technical aspects of the system such as Data modeling, use case modeling and Sequence diagrams.

Overall, this chapter provides an in-depth examination of the system modeling process for the proposed plant status tracking system and online marketplace. By understanding the above stated elements and how they interact with one another, it ensures that the system is designed to meet the needs of users and operates effectively and efficiently.

3.2 UI Identification

Table 3.1: UI Identification

Identifier	Interface Name
UI-01	Splash Screen
UI-02	Registration
UI-03	Login
UI-04	Status and Tracking page
UI-05	Data analysis page
UI-06	Marketplace page
UI-07	Manage profile
UI-08	Forget password

UI-09	View product
UI-10	Admin Dashboard
UI-11	Manage product
UI-12	Post product
UI-13	Agriculture equipment
UI-14	Contact Us
UI-15	Admin Dashboard
UI-16	Administrators
UI-17	Users
UI-18	Logout

3.3 Business Rules Identification

Table 3.2: Business rule Identification

BR-01	The user must provide correct and legit information during registration.
BR-02	If the user registered either in MOFER plant status tracking app or MOFER marketplace, then the user can access the other platform by using the credential of the other platform.
BR-03	If the user registered in the MOFER plant status tracking app first, they would get a special badge icon on their name when they post product on marketplace.
BR-04	Users can track 2 plant species Maximum. We will update the maximum limit in the future updates
BR-05	For the time being the MOFER plant status tracking, app will track 2 plant species (tomato and potato). We will add the capability to track more and different species in the future update. till then stay tuned

BR-06	For each plant the customer must buy an Arduino and its sensor from our store that will be configured based on the plant species that he/she wants to track. so generally, the customer needs to buy Arduino and its sensor for each plant species that he/she wants to track
BR-07	We offer a 1-month free trial to use the MOFER plant status tracking app for any new customer that will register to the MOFER plant status tracking app. After 1 month the customer must pay a subscription fee based on his/her package preference to use the MOFER plant status tracking app.
BR-08	If the customer wants to change the Arduino to a different plant species, the customer must bring the Arduino to our store so that the Arduino can be configured. We will provide these services without any extra cost.
BR-09	The Arduino needs an internet connection to track the plant/s so it's preferable to use stable Wi-Fi (minimum 100Kb/s). also, its preferable to add an additional battery so that the tracking process will not be interrupted by power shortage.
BR-10	MOFER Marketplace is an open market. We don't add extra fees for the customers that want to buy plants or sell plants.
BR-11	We do not control the transactions that are done between sellers and buyers. we only offer a place where the seller can show his/her product and the buyer can search, view product that he/she wants
BR-12	We don't take responsibility for any scamming activity that can be done in the marketplace. but we can take necessary measures based on the user feedbacks such from terminating for temporary till banning the scammer from the platform
BR-13	On the Marketplace, the sellers must post only plant species farming product only. we will take measures for illegal post starting from removing the post till banning the user

3.4 Actor Identification

Table 3.3: Actor Identification

ID	Actor name	Description
A01	User (farmer)	Track their plants and also upload products that they want to sell
A02	User (new seller)	Upload agriculture products
A03	User (buyer)	View products and if they are interested, they view seller information so that they can buy the product
A04	Administrator	Control and manage the overall system
A05	Bank	It's an external system that provides a system for money transaction

3.5 Actor Description

User (farmers): in the MOFER system, users (farmers) are the one that register and sign to the MOFER mobile application in order to use the MOFER services. After signing in, MOFER app provides different services such as view product, upload product, pay for the service, update product, get notification, tracking, generate report etc. after signing in, the user can update or delete his profile too.

User (new sellers): in the MOFER system, users (new sellers) are the one that register and sign to the MOFER marketplace (web). After signing in, the user can post products that can be sold, update product information that are posted on the marketplace, delete products that are posted on the marketplace, view products that are posted. The user also can update or deactivate his/her account. It offers the user's password recovery just in case the user forgets his/her password.

User (buyers): these users are the one that buy plant products from the MOFER marketplace. They can view different products from the marketplace, and they can view seller information for the product that they want to buy.

Banks: banks are an external actor that works with the MOFER system. The main tasks of banks are to verify and take care of transactions from user (farmers) e-banking accounts to MOFER's account to pay for the services fee.

Administrator: these actors are the one who control and manage the overall system. They can add, update, and delete admins. They can delete a user's product that is posted on the marketplace if they feel an inappropriate product, they also can search and terminate/ban the user if he/she posted inappropriate products repeatedly. The system offers admins password recovery just in case the admin forgets his/her password.

3.6 Use Case Identification

Table 3.4: Use Case Identification

ID	Use case name	Include
UC-01	Register	None
UC-02	Login	UC-01
UC-03	manage profile	UC-02
UC-04	Track plant	UC-01
UC-05	Pay for service	UC-02
UC-06	Generate report	UC-02
UC-07	View product	None
UC-08	manage product	UC-02
UC-09	User's password recovery	UC-01
UC-10	Logout	UC-02
UC-11	View seller information	None
UC-12	Manage admin	UC-02
UC-13	Ban Users	UC-02
UC-14	Delete users' product	UC-02

3.7 Use Case Diagram

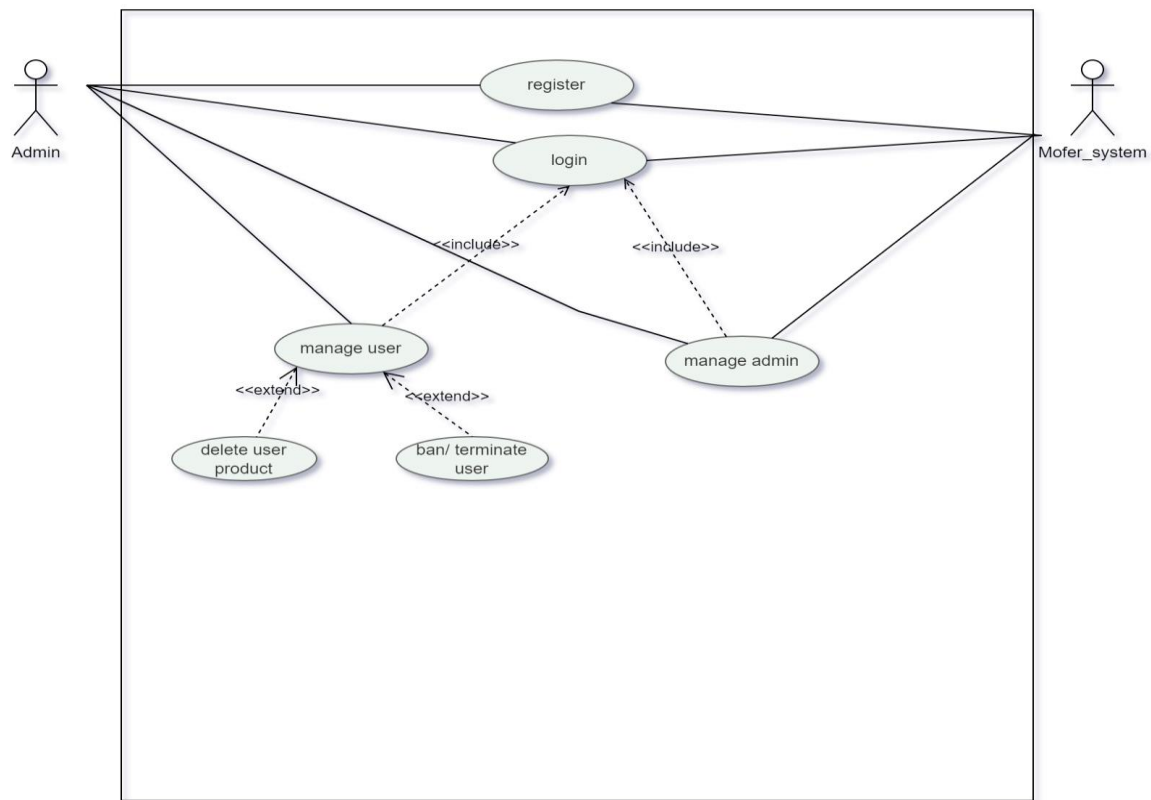


Fig 3.1: Use Case Diagram for Admin

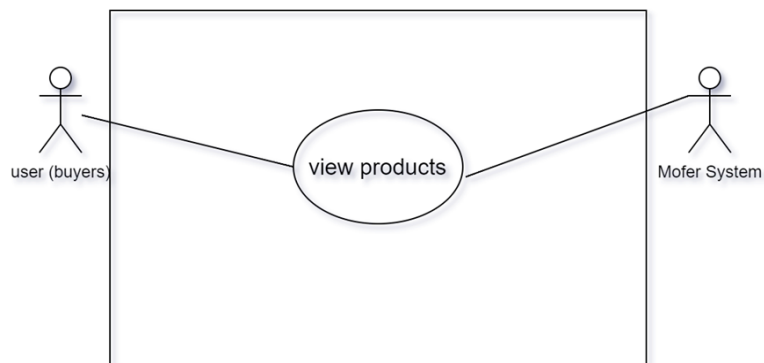


Fig 3.2: Use Case Diagram for User (Buyers)

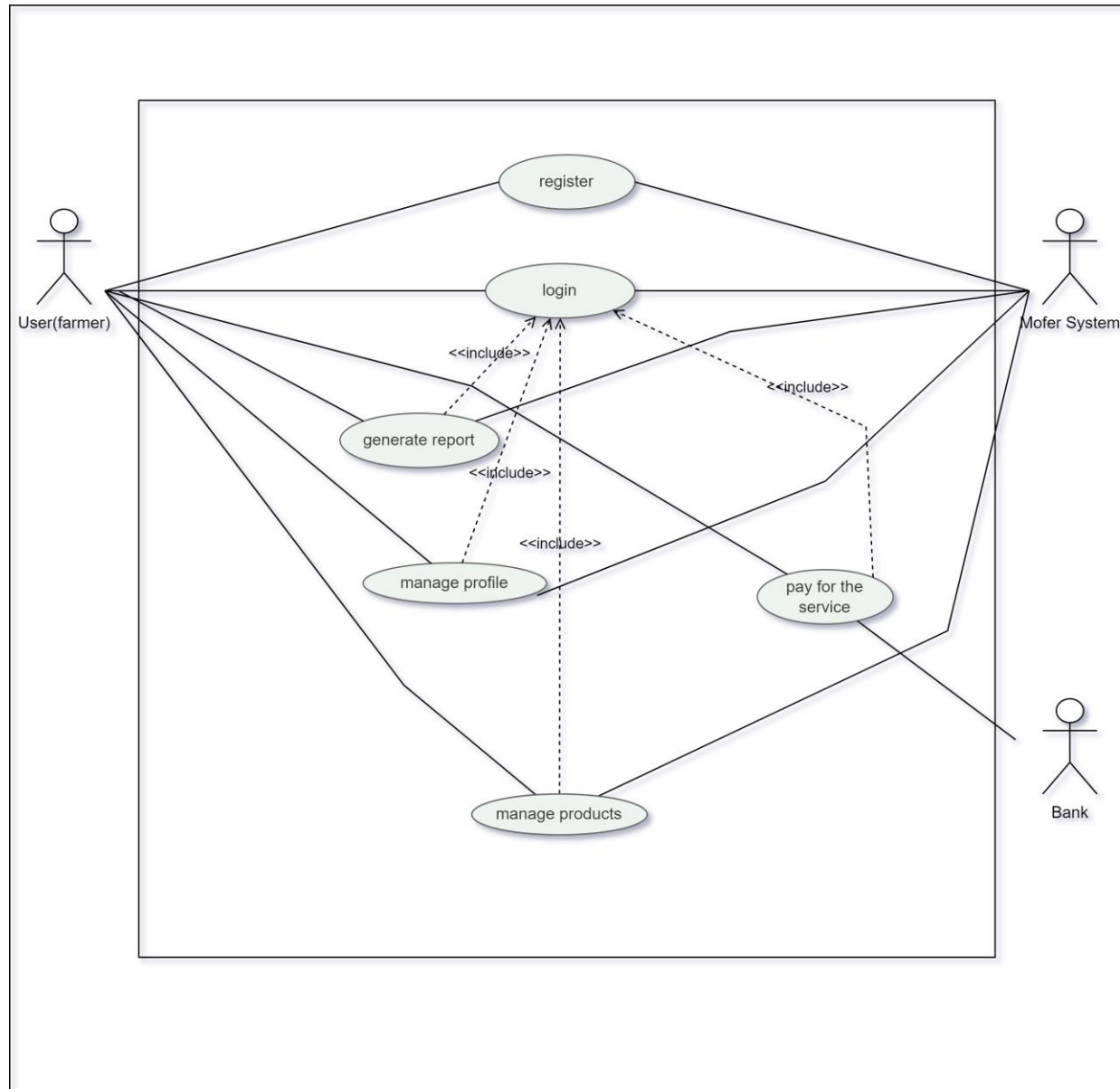


Fig 3.3: Use Case Diagram for User (Farmer)

3.8 Use Case Description

Table 3.5: Use Case Description for Registration

Use case id	UC1
Use case name	Register
Actor	User (Farmer), User (Seller)
Pre-condition	None
Sequential row	1. The user selects registration. 2. The system display registration interface 3. The user enters data to the form and submits it. 4. The system takes the data and inserts it into the database. 5. The system will show success message. 6. Use case ends
Alternative	1. If the user enters incorrect data 1.1. The system suggests the user to enter correct data
Post condition	Shows registered successfully message

Table 3.6: Use Case Description for Login

Use case id	UC2
Use case name	Login
Actor	User (Farmer), User (Seller), Admin
Pre-condition	Register
Sequential row	1. The user opens the login interface. 2. The user enters email and password and login. 3. The system will validate ate the email and password. 4. The system display home interface 5. Use case ends

Alternative	1. If the user enters incorrect data 1.1. The system suggests the user to enter correct data. 2. If the user exists but the password does not match 2.1. The system suggests the user to enter correct password. 2.2. The system suggests user password recovery
Post condition	Renders home page

Table 3.7: Use Case Description for Manage Profile

Use case id	UC3
Use case name	Manage Profile
Actor	User (Farmer), User (Seller), Admin
Pre-condition	Login
Sequential row	1. The user selects manage profile. 2. The system display manages profile interface. 3. The user manages profile. 4. The system manages profile on the database side and saves the changes. 5. Use Case ends
Alternative	1. If the user enters incomplete data 1.1. The system suggests the user to complete the data
Post condition	Updated or Deleted or Added successfully message

Table 3.8: Use Case Description for Track Plant

Use case id	UC4
Use case name	Track Plant
Actor	User (Farmer)
Pre-condition	Login, Pay for service

Sequential row	1. The user selects tracing plant. 2. The system gets data (soil moisture, PH level, weather, humidity) 3. The system passes the data it fetches to the tracing plant interface and displays it. 4. Use Case ends
Alternative	1. The system could not access the database. 1.1. The system displays unsuccessful message
Post condition	The result of the plant states page

Table 3.9: Use Case Description for Pay for Service

Use case id	UC5
Use case name	Pay for services
Actor	Bank, User(farmer)
Pre-condition	Login
Sequential row	1. The user selects pay for service. 2. The system gets packages from the database and display package interface. 3. The user selects their preferable package. 4. The system takes the selected package and passes it to the Bank. 5. The system display payment form interface 6. The user fill bank credential 7. The bank check credential and amount 8. The bank gives successful message to the system. 9. The system saves the package to database and gives access to the user. 10. Use Case ends
Alternative	1. The user does not hold enough amount in their account. 1.1. The bank display unsuccessful message to the system 1.2. The system displays unsuccessful message to the user
Post condition	Successfully subscribed message

Table 3.10: Use Case Description for Generate Report

Use case id	UC6
Use case name	Generate Report
Actor	User (Farmer)
Pre-condition	Login, Pay for service
Sequential row	1. The user selects plant report. 2. The system fetches the data about the user plant from database. 3. The system will compute the data and compare it with previous data from other users. 4. The system displays the computed data on plant report interface. 5. Use Case ends
Alternative	1. The system could not access the database. 1.1. The system displays unsuccessful message. 2. The system could not compute the data. 1.1. The system suggests trying again
Post condition	Report on the plants page displayed

Table 3.11: Use Case Description for View Product

Use case id	UC7
Use case name	View Product
Actor	User (Customer)
Pre-condition	None
Sequential row	1. The user selects MOFER web market. 2. The system display marketplace 3. The user selects and search for the product they want/need. 4. The system displays the detailed information interface. 5. Use Case ends
Alternative	1. The user search for product that do not exist. 1.1. The system display product not found interface

Post condition	Redirects marketplace page
----------------	----------------------------

Table 3.12: Use Case Description for Manage Product

Use case id	UC8
Use case name	Manage Product
Actor	Admin
Pre-condition	Login
Sequential row	1. The admin selects manage product. 2. The system display manages product interface. 3. The user manage product (add, update, delete) 4. The system manages product on the database side and saves the changes. 5. Use Case ends
Alternative	1. If the user enters incomplete data 1.1. The system suggests the user to complete the data. 2. If the product is not found 2.1. The system display product not found page
Post condition	Updated or Deleted or Added successfully message

Table 3.13: Use Case Description for User Password Recovery

Use case id	UC9
Use case name	User Password Recovery
Actor	User (Customer), User (Farmer), Admin
Pre-condition	Register
Sequential row	1. The user selects password recovery. 2. The system display forgot password interface. 3. The user enters their email.

	4. Firebases validates the email and generates verification email. 5. The user clicks on the verification link on their mail. 6. The system display password generation interface 7. The user inserts a new password and confirmation. 8. The system validates and updates the password. 9. Use Case ends
Alternative	1. The user email does not exist. 1.1. The system displays a message email does not exist. 2. If the password does not match 1.1. The system message new password and confirm password does not match
Post condition	Password changed successfully message

Table 3.14: Use Case Description for User Manage Admin

Use case id	UC10
Use case name	Manage Admin
Actor	Admin
Pre-condition	Login
Sequential row	1. The admin selects manage admin. 2. The system display manages admin interface. 3. The user manages admin (add, delete). 4. The system manages admin on the database side and saves the changes. 5. Use Case ends
Alternative	1. If the admin enters incomplete data 1.1. The system suggests the user to complete the data. 2. If the product is not found 2.1. The system display product not found page
Post condition	Updated or Deleted or Added successfully message.

Table 3.15: Use Case Description for Ban Users

Use case id	UC11
Use case name	Ban Users
Actor	Admin
Pre-condition	Login
Sequential row	1. The admin selects ban users. 2. The system display ban user's interface 3. The admin search for the user 4. The admin bans the selected user. 5. The system saves the changes on database. 6. Use Case ends
Alternative	1. If the admin enters non existing user 1.1. The system suggests the user does not exist
Post condition	The user is banned message.

Table 3.16: Use Case Description for Delete User Product

Use case id	UC12
Use case name	Delete User Product
Actor	Admin
Pre-condition	Login
Sequential row	1. The admin selects product delete. 2. The system display deletes product interface. 3. The admin search for the product 4. The admin deletes the selected user. 5. The system saves the changes on database. 6. Use Case ends
Alternative	1. If the admin enters non existing product 1.1. The system suggests the product does not exist
Post condition	The product is removed message

3.9 Activity Diagram

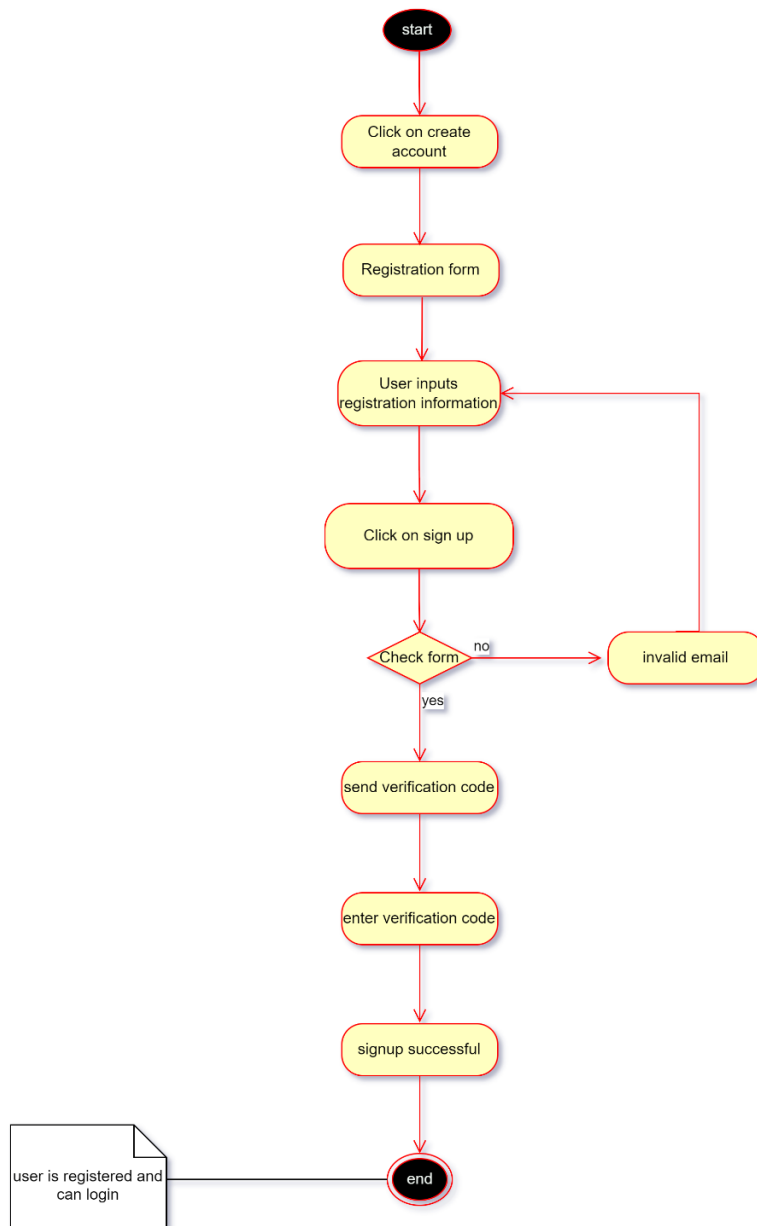


Fig 3.4: Activity diagram for Registration

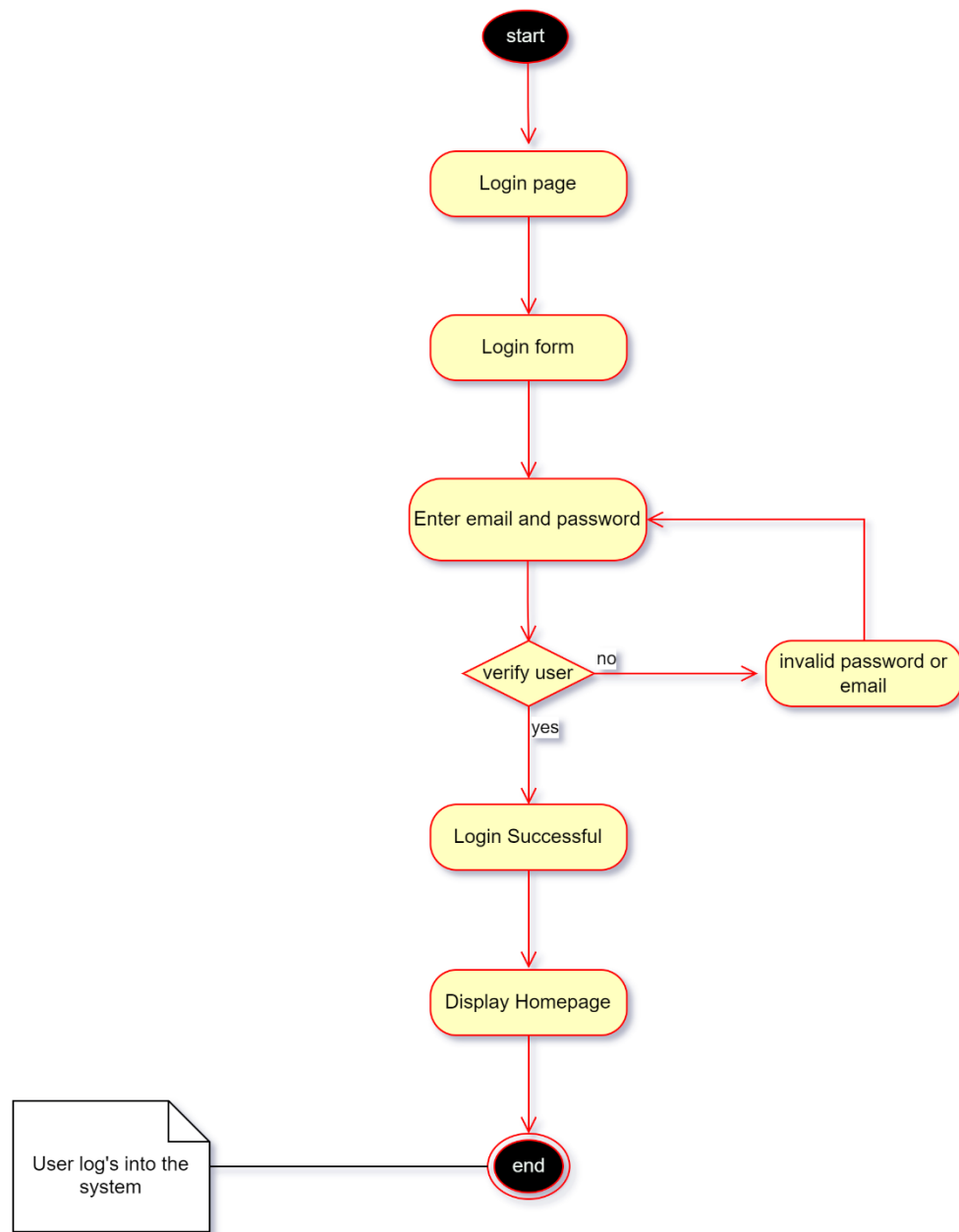


Fig 3.5: Activity diagram for Login

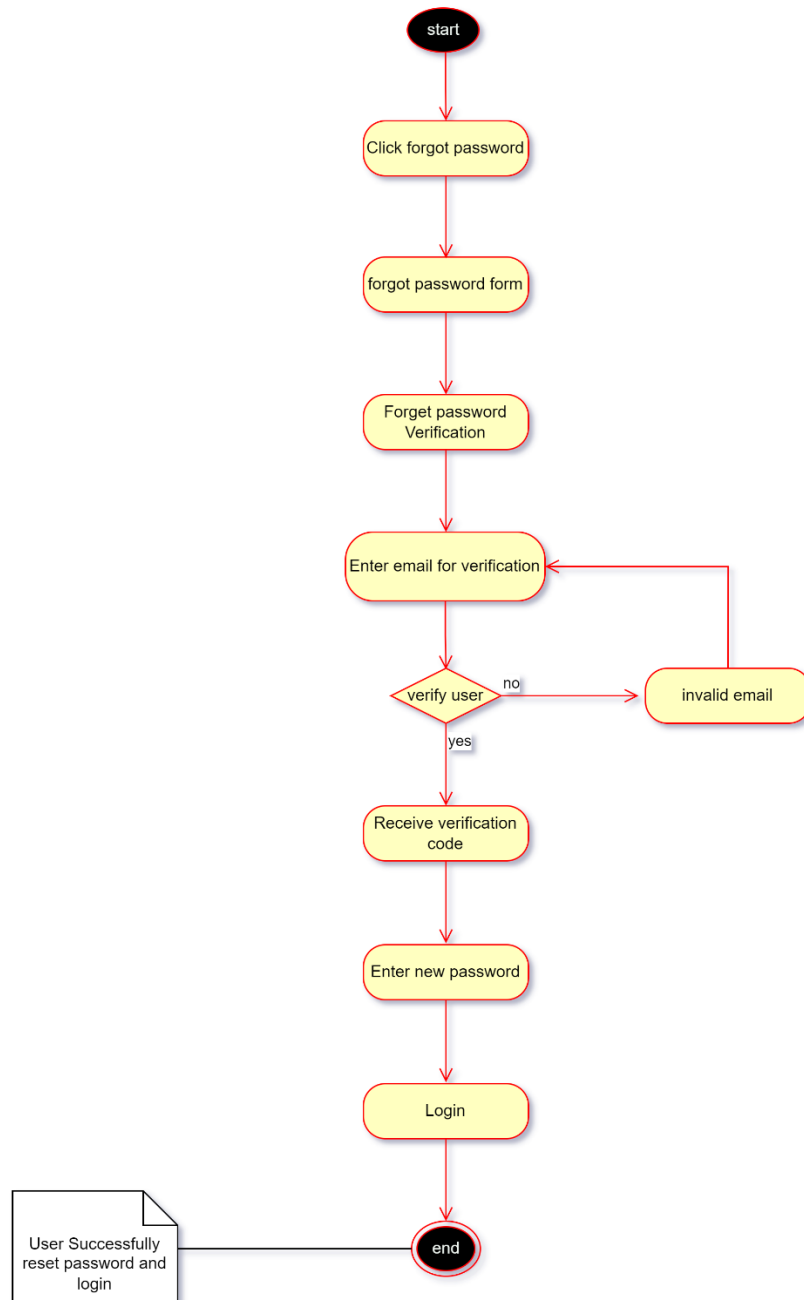


Fig 3.6: Activity diagram for Password Recovery

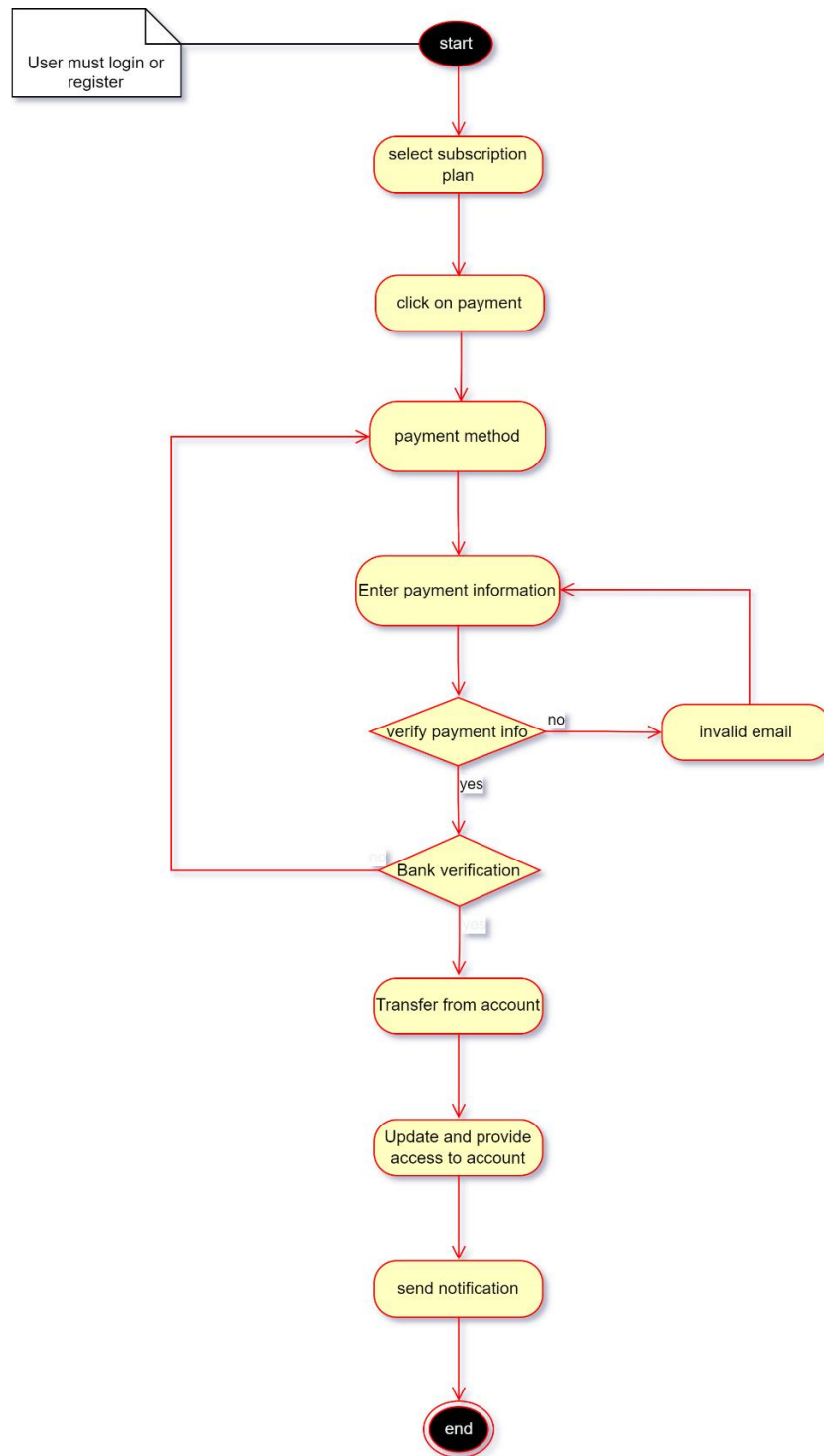


Fig 3.7: Activity diagram for Pay for Service

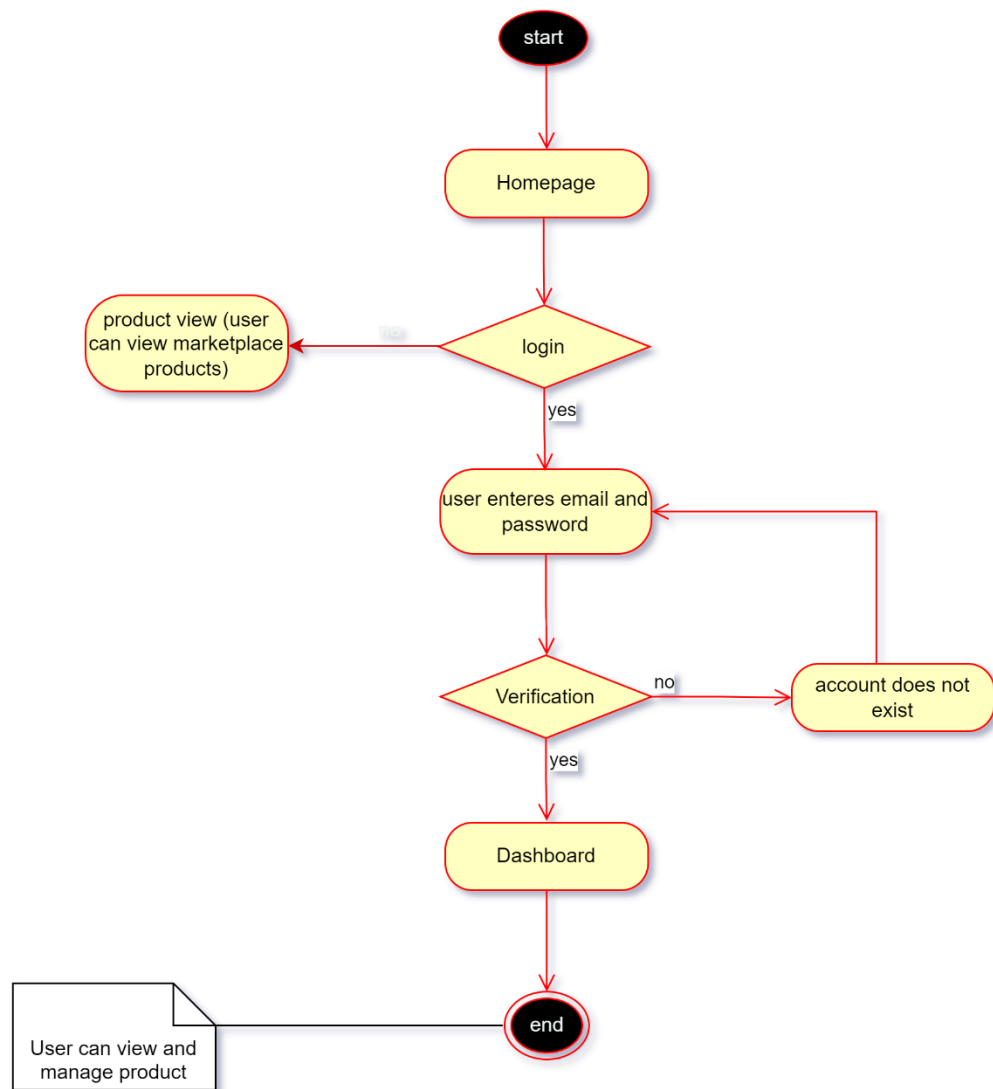


Fig 3.8: Activity diagram for Marketplace view

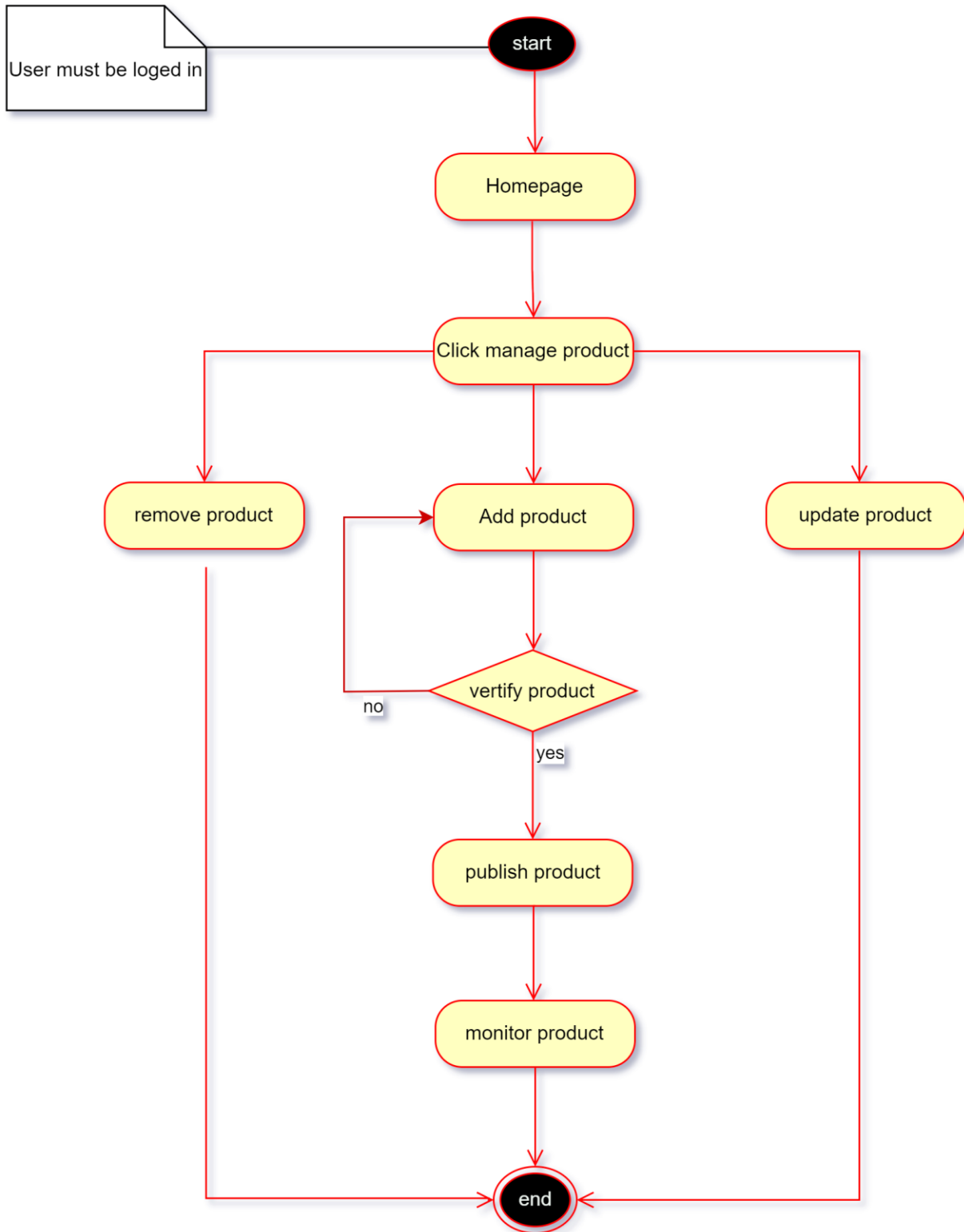


Fig 3.9: Activity diagram for Marketplace Manage Product

3.10 Sequence Diagram

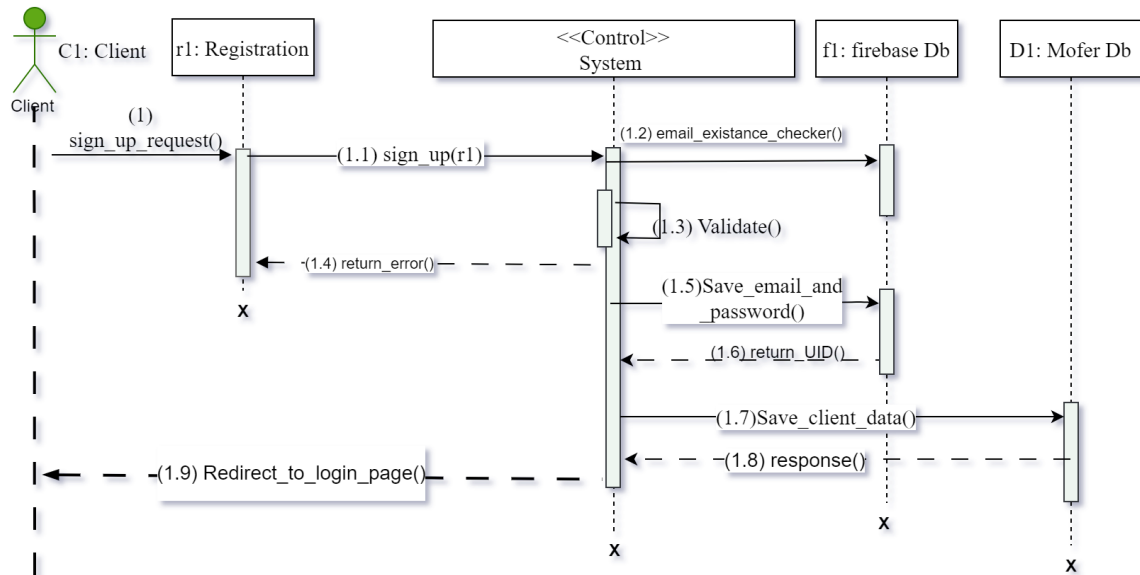


Fig 3.10: register sequence diagram.

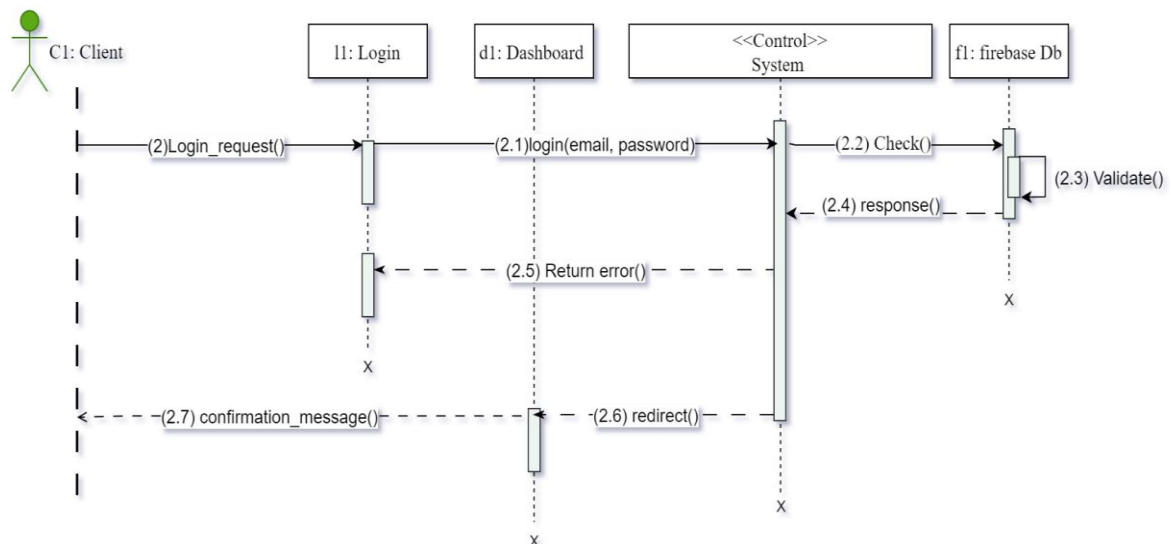


Fig 3.11: Login sequence diagram

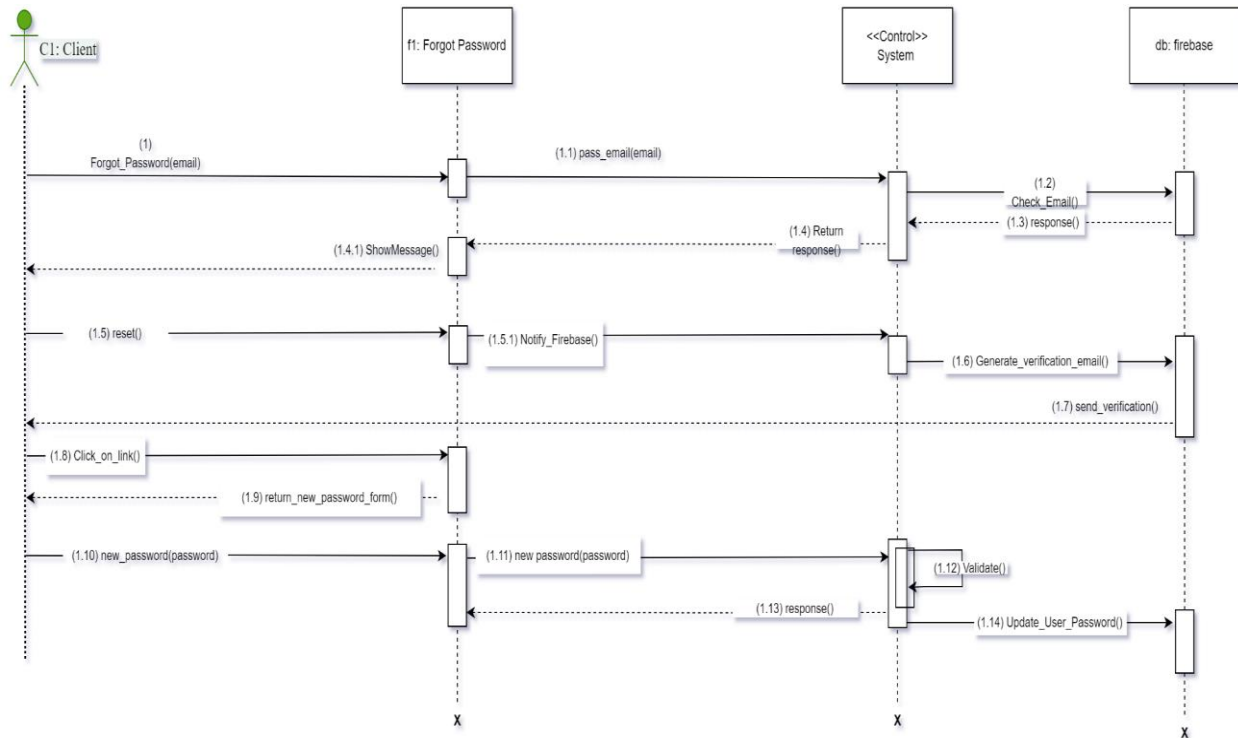


Fig 3.12: Forget password sequence diagram.

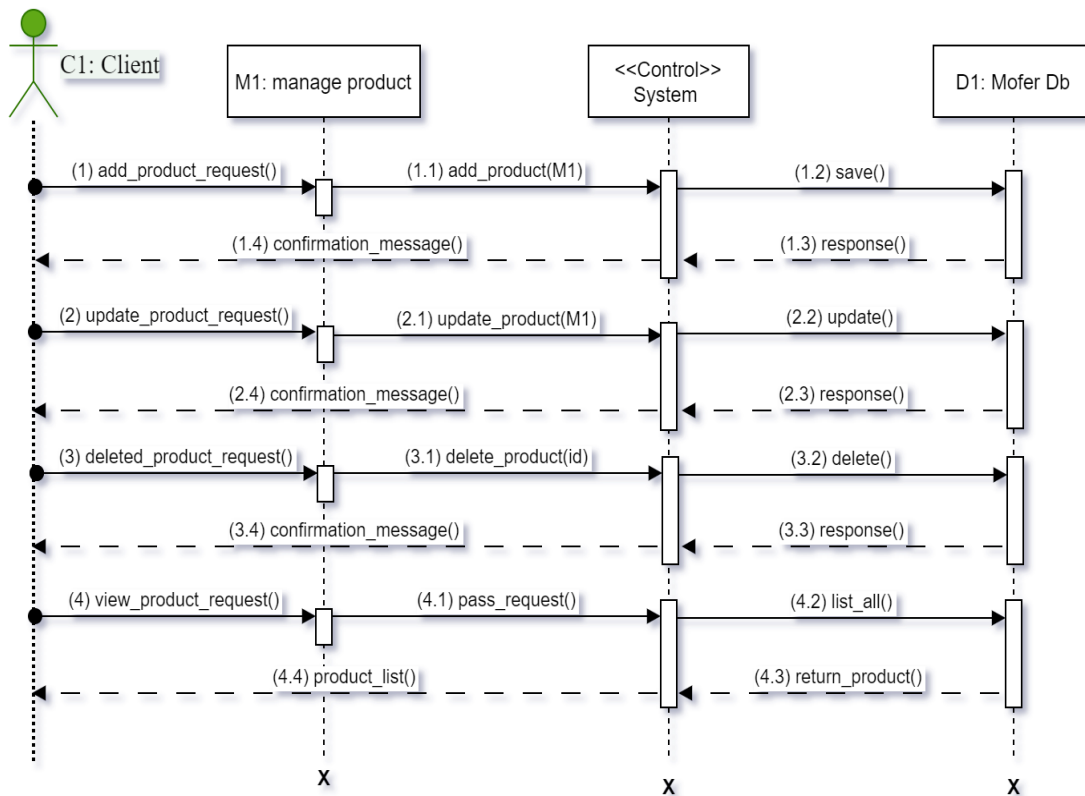


Fig 3.13: Manage product sequence diagram.

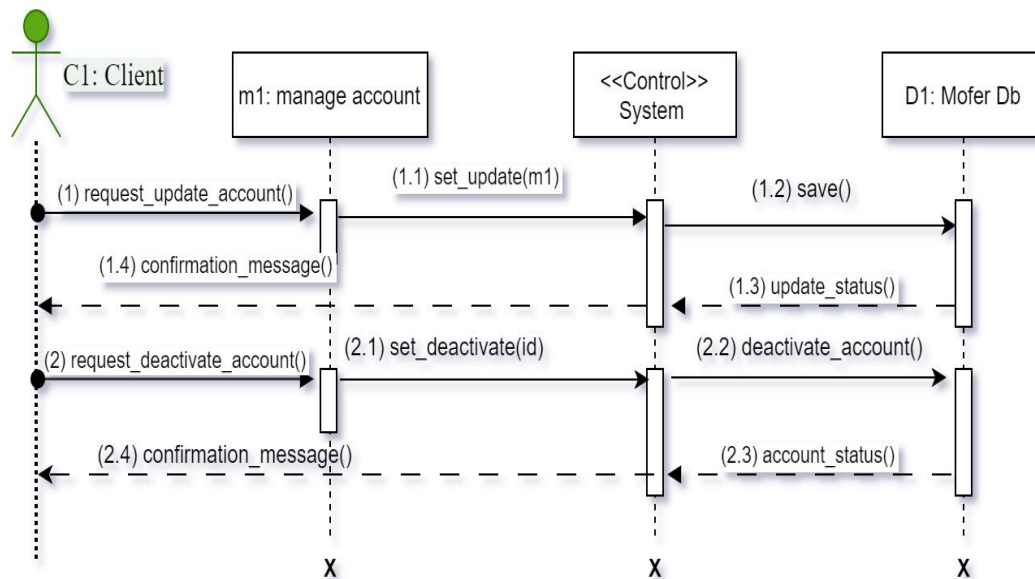


Fig 3.14: Manage account sequence diagram.

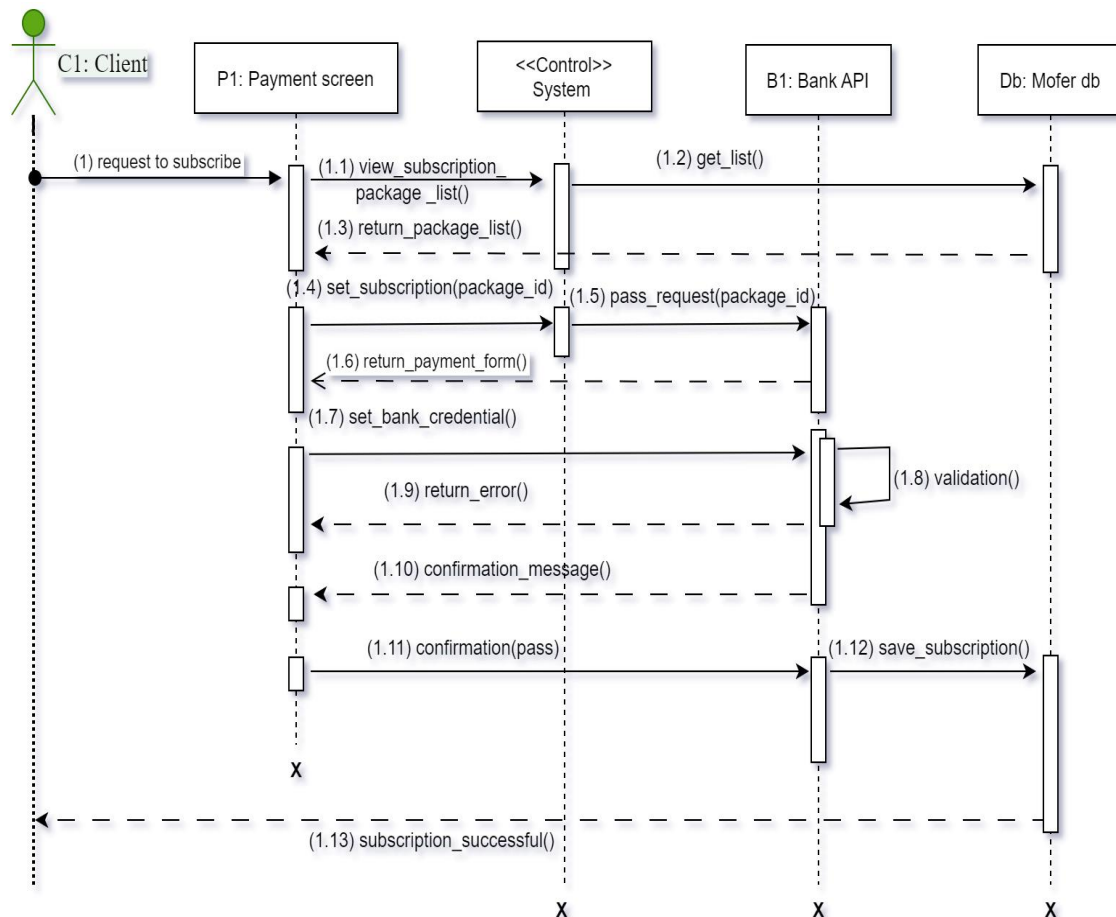


Fig 3.15: Pay for the service sequence diagram.

3.11 Class Diagram

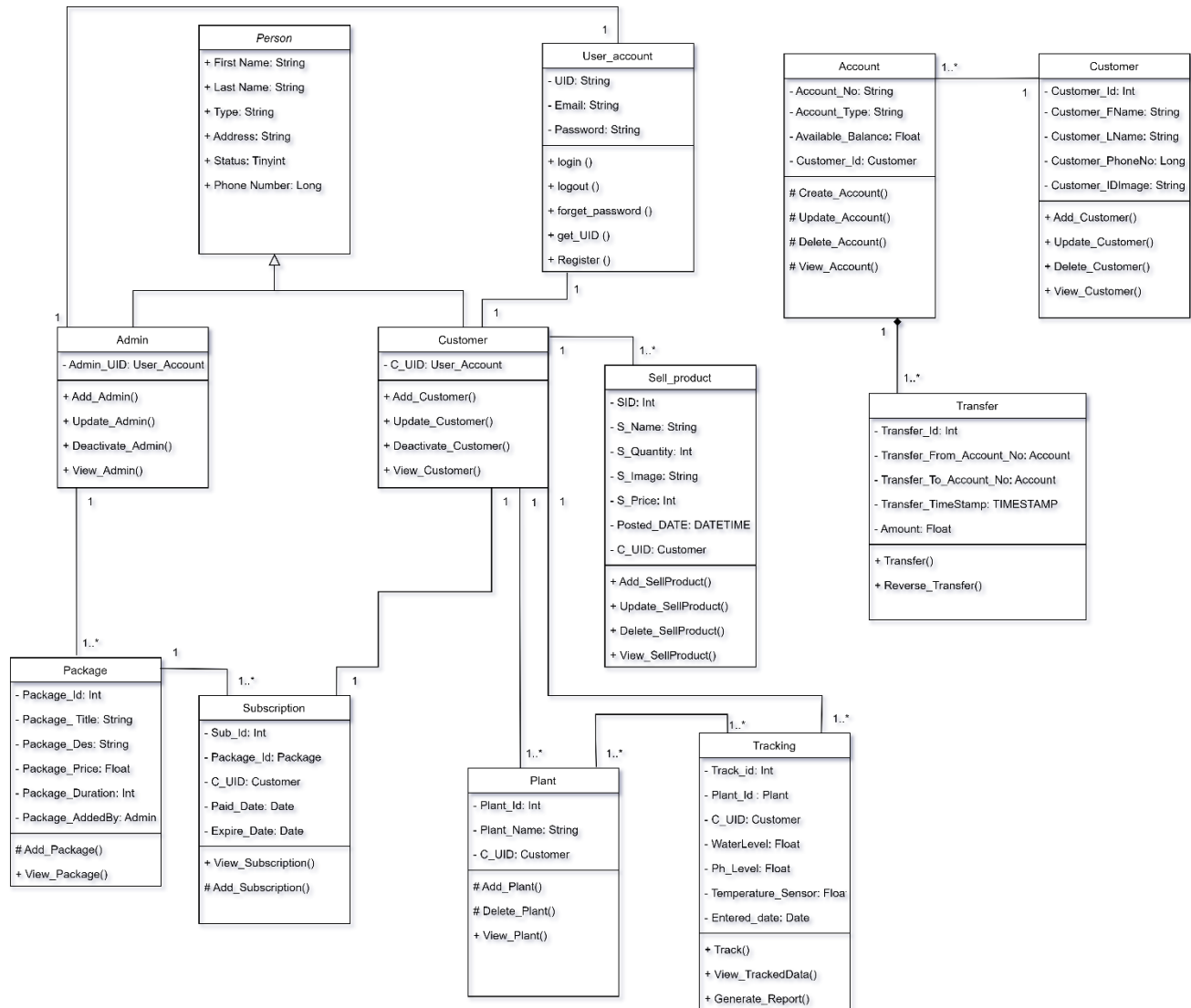


Fig 3.16: Class Diagram of The System

3.12 State Chart Diagram

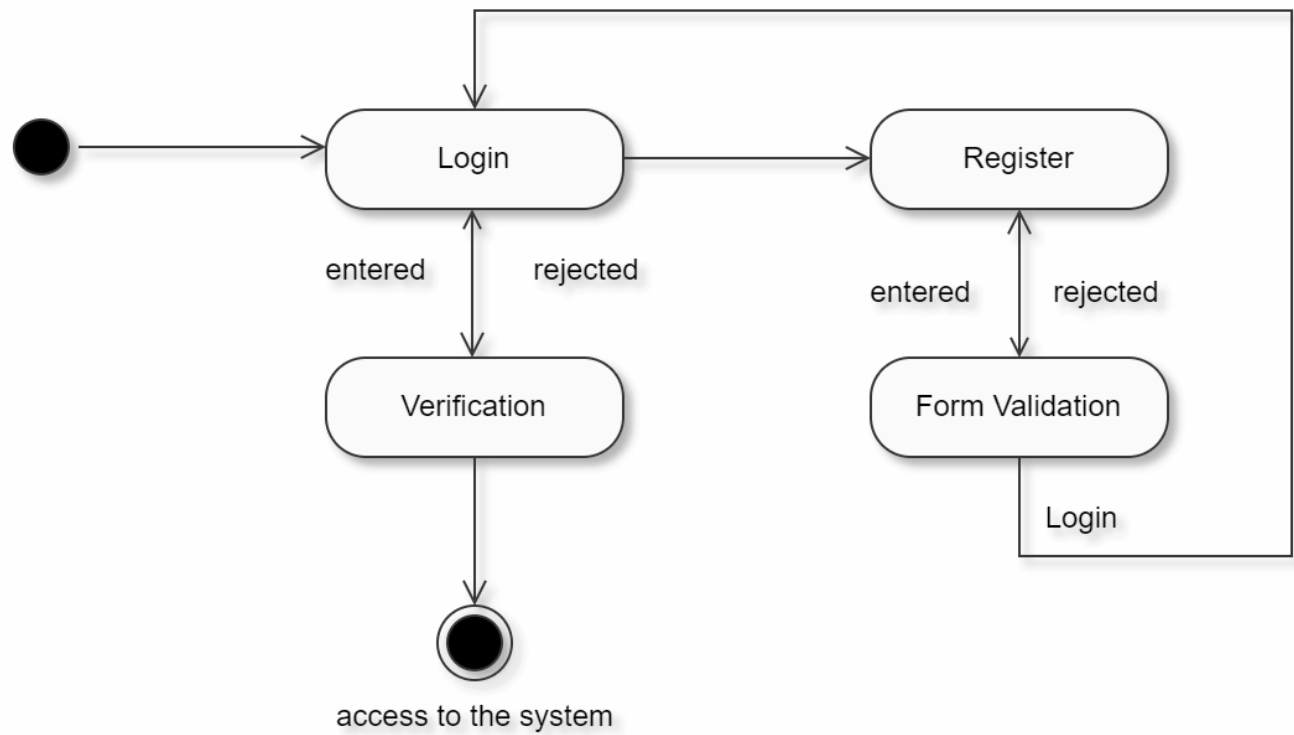


Fig 3.17: State chart diagram for Login and Registration

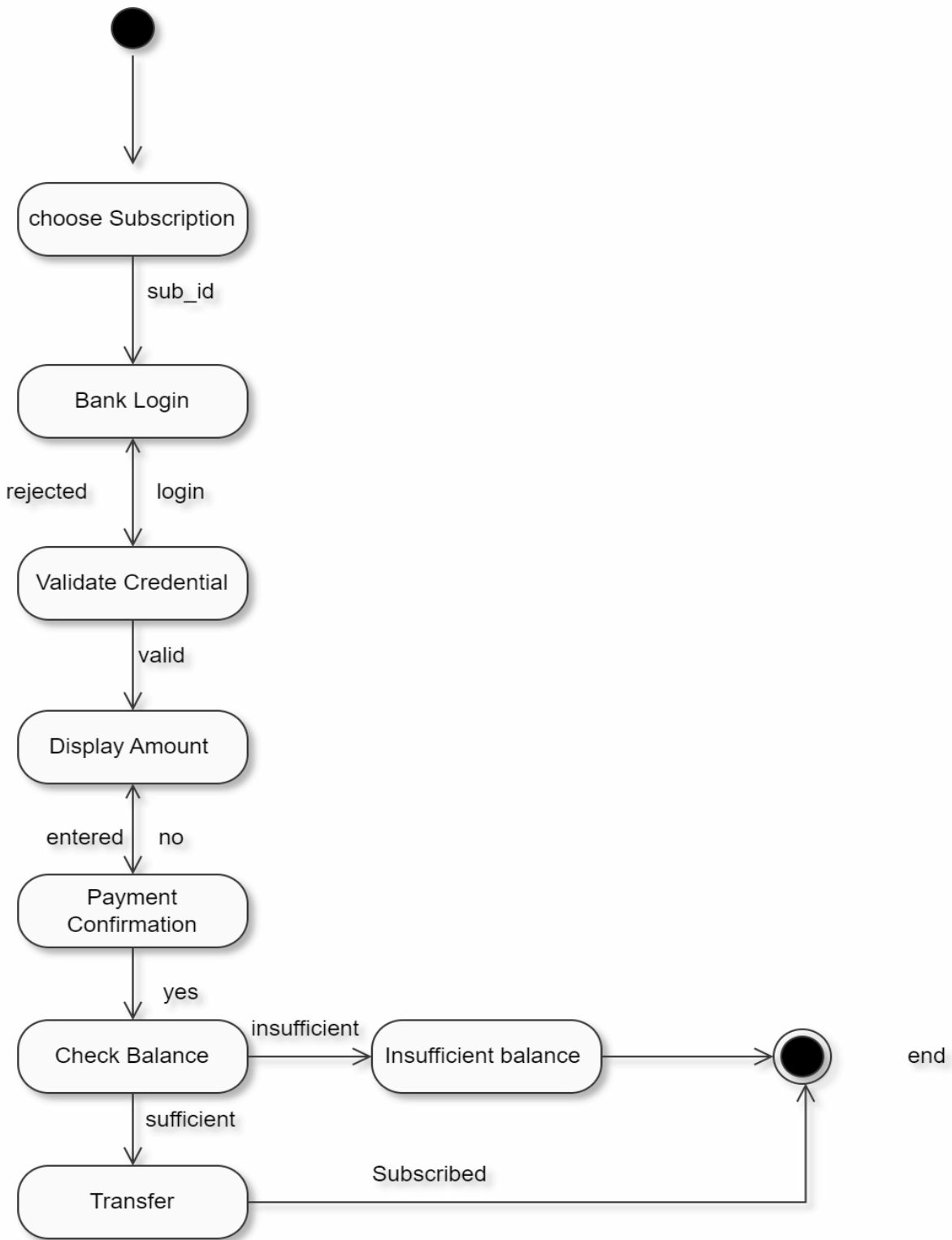
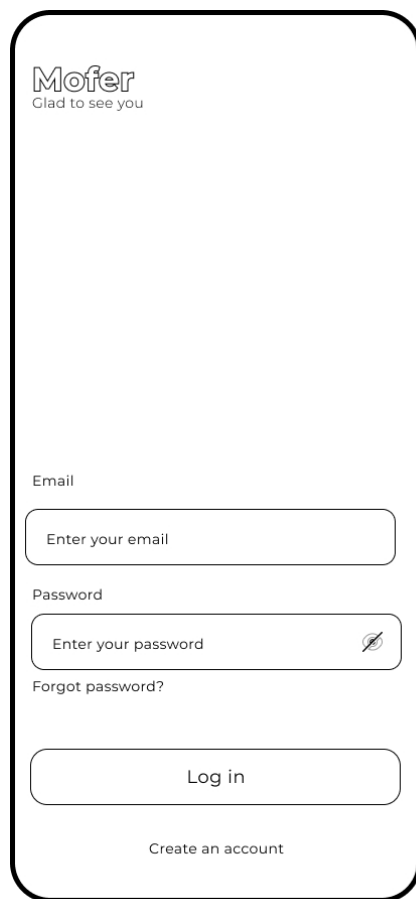


Fig 3.18: state chart diagram for pay for the services.

3.13 User Interface Prototyping



The image shows a mobile application login screen. At the top left, the logo 'Mofer' is displayed with the tagline 'Glad to see you' underneath. The main area contains two input fields: one for 'Email' with the placeholder text 'Enter your email', and one for 'Password' with the placeholder text 'Enter your password' and a toggle icon (an eye with a diagonal line) on the right. Below the password field is a link that says 'Forgot password?'. At the bottom of the form area is a large 'Log in' button. Below the button is a link that says 'Create an account'.

Mofer
Glad to see you

Email

Enter your email

Password

Enter your password

[Forgot password?](#)

Log in

[Create an account](#)

Fig 3.19: Login page

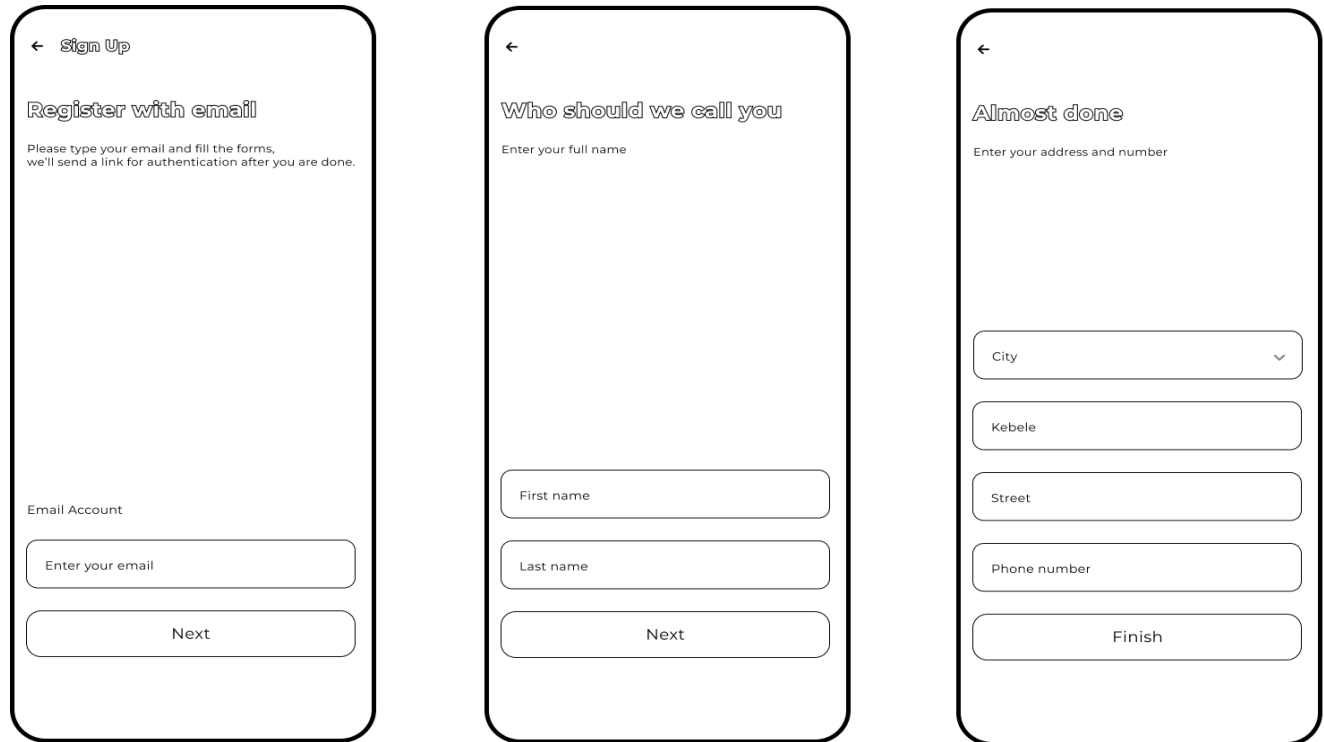


Fig 3.20: User interface prototyping for the sign-up pages

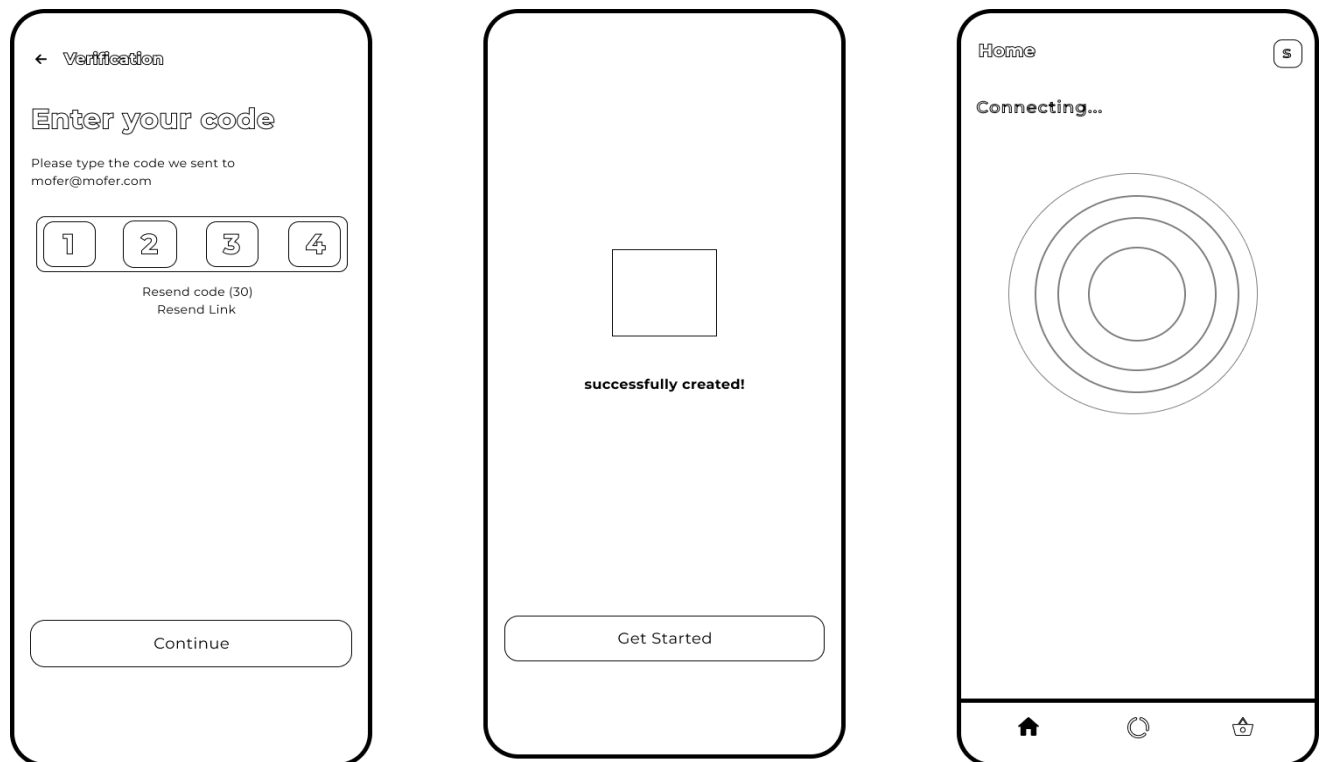


Fig 3.21: User interface prototyping for the verification, loading, successful message shower pages.

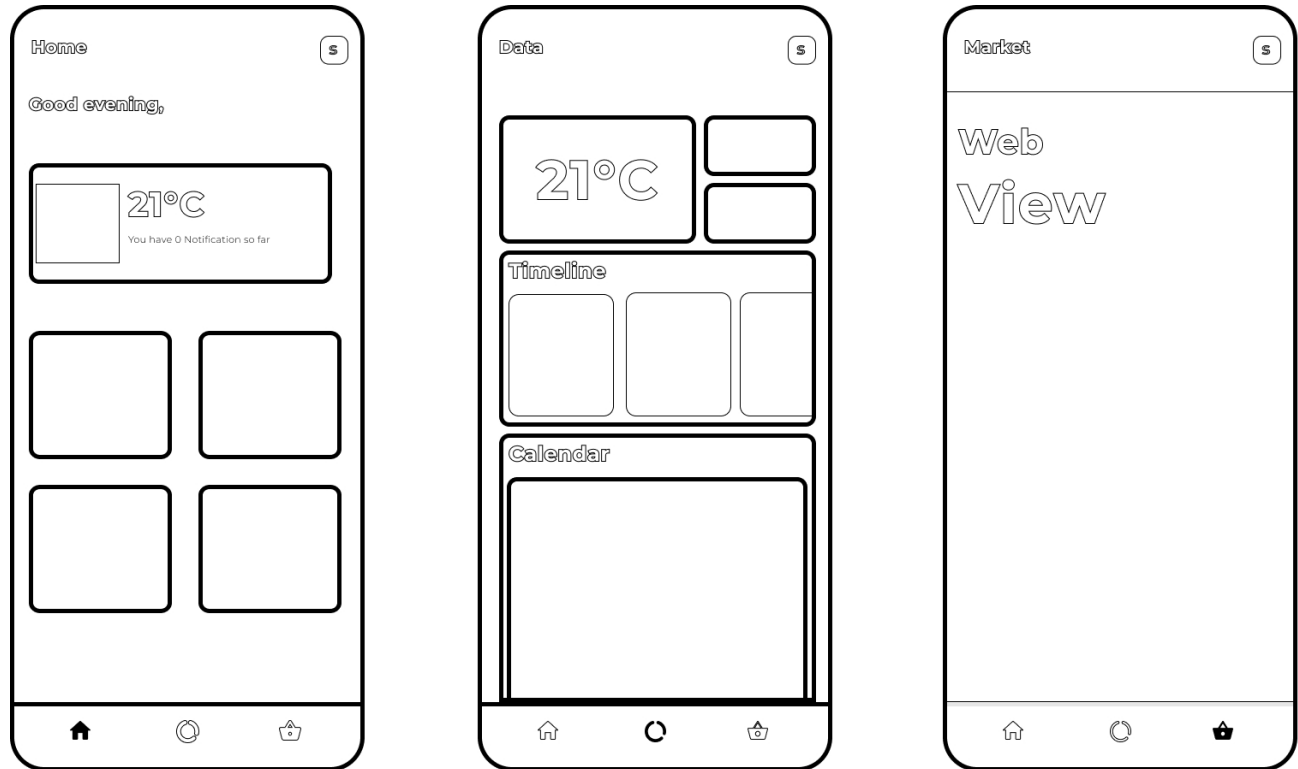


Fig 3.22: User interface prototyping for the Home, data, and marketplace pages

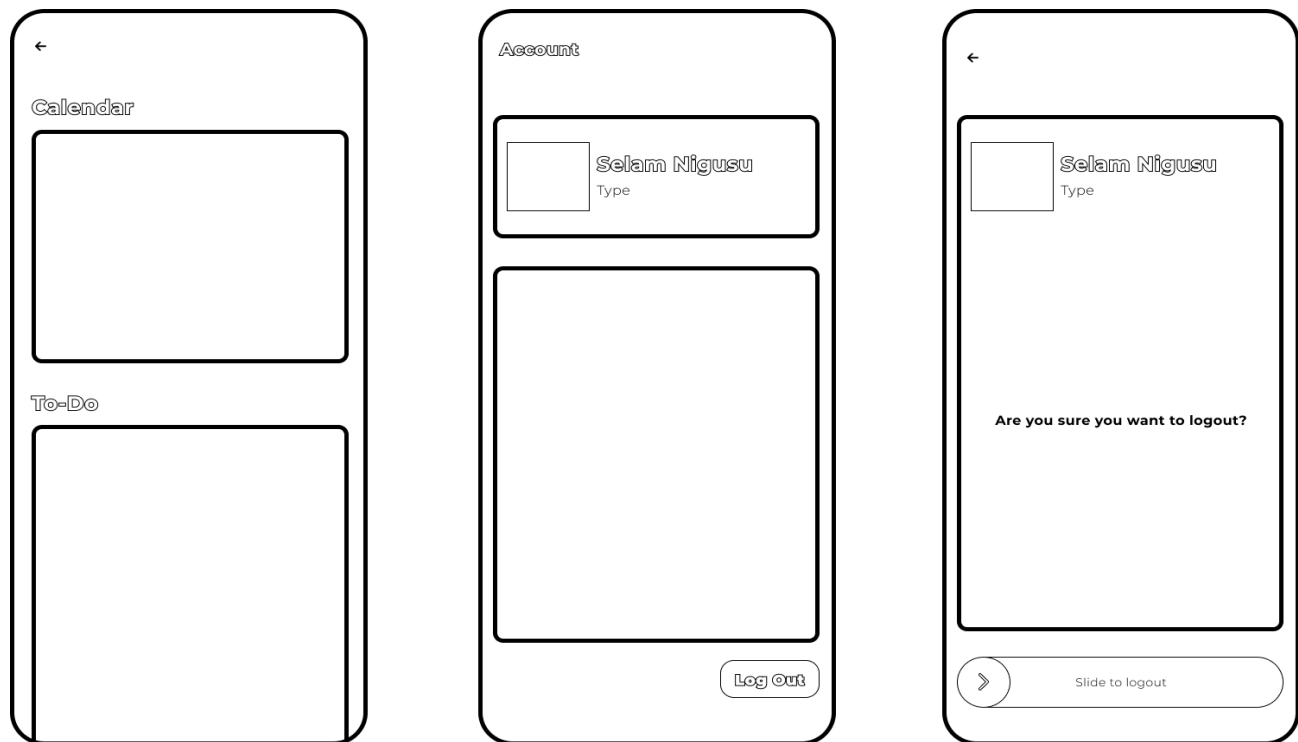


Fig 3.23: User interface prototyping for the calendar and manage account pages.

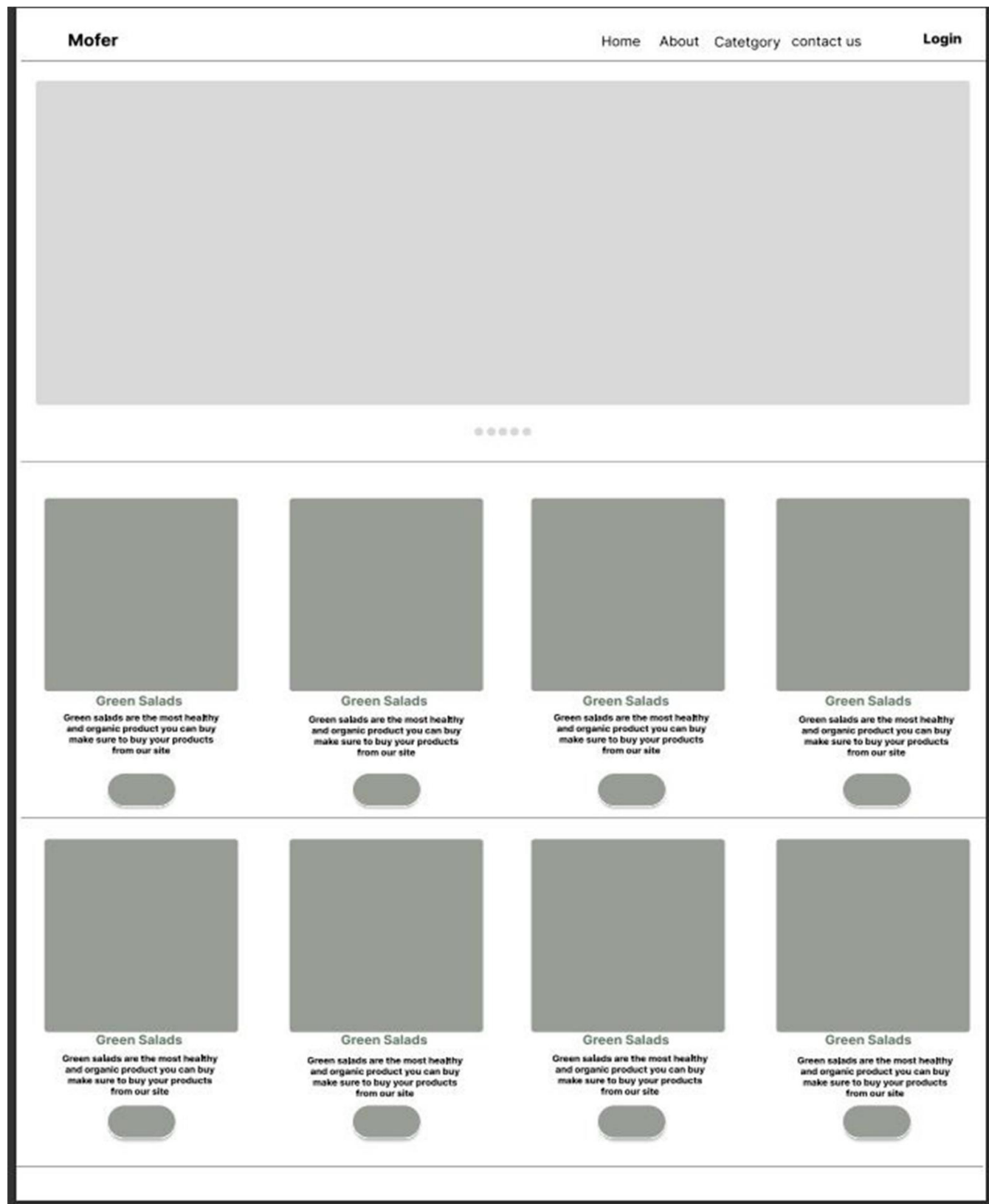


Fig 3.24: Marketplace web page

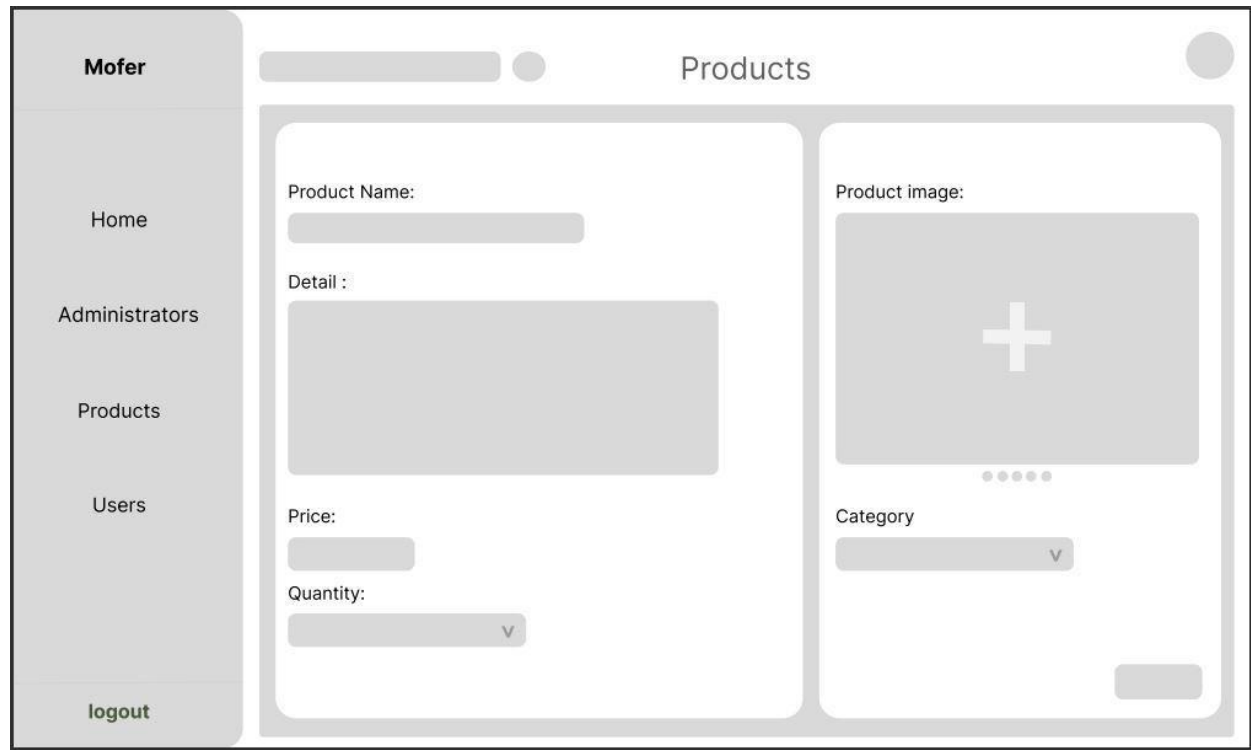


Fig 3.25: marketplace admin panel web page

Chapter Four

System Design

In this chapter, we will delve into the technical details of how the app will be designed and built to ensure that it meets the requirements specified in the previous chapters. In this chapter, we will discuss the architecture of the app, including the backend and frontend components. We will explore the various the database design, normalization, state chart and the security features implemented to ensure the safety of user data.

The chapter will also cover the user interface design of the app, including the visual design, user experience, and user flow. We will discuss how the app's features will be organized and presented to the user to ensure ease of use and an intuitive user experience. Overall, this chapter provides an in-depth look at the technical aspects of designing the proposed system. The system design is a critical component of the development process, and this chapter outlines the key features and considerations needed to ensure that the app is functional, secure, and user-friendly.

4.1. System Architecture

The architectural design pattern known as MVC, or model, view, controller, divides an application into three primary logical components: the model, the view, and the controller. Each of these parts is designed to handle particular application development facets.

It is an approach to designing software projects. It is mostly used by Node developers, while C#, Ruby, and PHP framework users also utilize it. The three interconnected components make up an application and its development based on the MVC pattern. The benefit of this is that it aids in concentrating on a certain aspect of the application name, the methods by which information is provided to and received from the user. It aids in enabling effective code reuse and concurrent application development.

The component known as the controller is in charge of handling client requests, including handling HTTP requests and responding to them. If you call an API, the response can be a JSON.

Model is a database interface that you can use to work with the database API and build various entity schemas for your app on the database (MySQL, MongoDB). Model is called from the

controller based on the client's request; if the client requests data that is stored, the controller will ask the model interface to provide the requested data.

The view needs to use a Template Engine for carrying out the rendering process where the controller feeds it with necessary data and the view renders and converts everything into plain HTML. The view is what compiles and renders into plain HTML and what the client will typically receive back as a response to what he requested.

Explanation in Depth

Model: Data structure, format, and storage constraints are all represented by models. It keeps the application's data up to date. In essence, it is the application's database component. You can find files that represent each table and an index in that folder. The database creation is completed in a JS file, and each table and its properties are stored in another file. When the index file is run, the database and its tables are built.

View: View is what the user is shown. Views make use of the Model and display data in the way the user desires. The data that is shown to the user may also be subject to modification. They are made up of rendered or transmitted static and dynamic pages that the user requests.

Controller: The controller manages the user's requests before producing the necessary responses and feeding them to the viewer. The user typically interacts with the View, which then generates the necessary request, which is then dealt with by a controller. As a response, the controller renders the relevant view using the model data. The files in this folder are separated by table consideration.

So, to sum it up:

- Model is the data part.
- View is the User Interface part.
- Controller is request-response handler.

MVC architecture

The project structure looks like this.

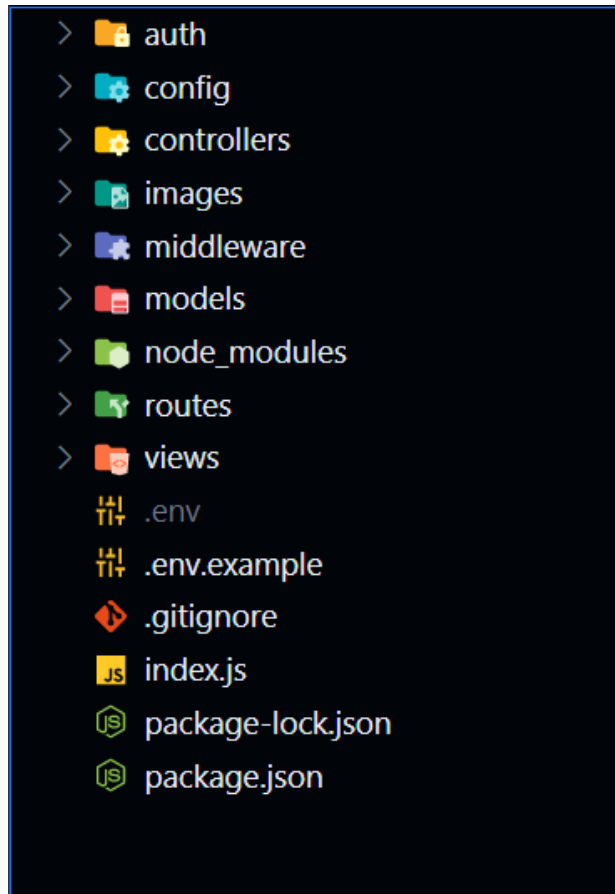


Fig 4.1 Folder structure of the MOFER Backend

Android

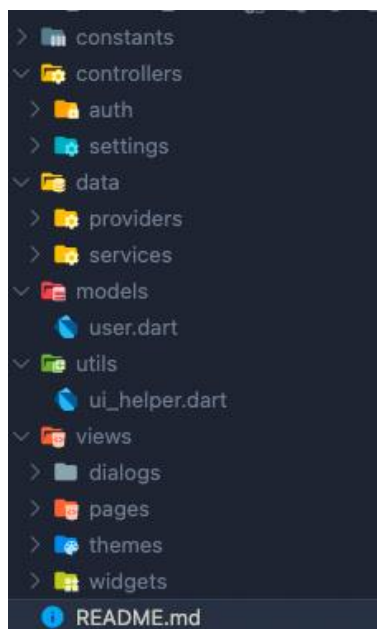


Fig 4.2 Folder structure of the MOFER frontend Android

Controller: The Controller folder contains classes that handle user input and interact with the Model classes. In addition to validating data, controllers may also perform other tasks such as converting data to the appropriate format, managing user authentication and authorization, and handling errors that occur during data processing. Controllers may also interact with other services, such as email or messaging services.

In an MVC (Model-View-Controller) architecture, the Controller is responsible for receiving user input from the View, processing that input, and updating the Model accordingly. For example, if a user fills out a form to create a new account, the Controller would be responsible for validating and processing the form data, creating a new user object in the Model layer, and then redirecting the user to a confirmation page.

Model: The Model folder contains classes that manage application data and perform operations on that data. In addition to working with databases, Models may also interact with external services such as APIs and Firebase. Models may perform tasks such as data validation, data retrieval, data processing, and data storage.

In an MVC architecture, the Model represents the data and the business logic associated with that data. For example, if an application manages customer orders, the Model layer would be responsible for managing the customer data, order data, and the rules associated with placing and fulfilling orders.

View: The View folder contains user interface components such as widgets, animations, and templates. The View layer is responsible for presenting data to the user in a visually appealing and interactive way. Views may also interact with Controllers to update the Model layer based on user input or to retrieve data for display.

In an MVC architecture, the View is responsible for rendering data from the Model layer and presenting it to the user. For example, if an application displays a list of products, the View would be responsible for rendering the product information and presenting it to the user in a format that is easy to read and interact with.

Utils: The Utils folder contains utility functions, helper classes, and other reusable components that can be used across the application. Utils may perform tasks such as data formatting, input validation, and error handling. By separating commonly used functions into the Utils layer, developers can avoid duplicating code and improve the maintainability of the application.

Services: The Services folder contains classes and functions that manage external services such as

notification systems, payment gateways, and email providers. Services may also handle tasks such as data synchronization, background processing, and file storage. By separating external service interactions into the Services layer, developers can keep the Controller and Model layers focused on their core responsibilities.

Const: The Const file contains constants, global configuration parameters, and other application-wide settings. By storing these values in a separate file, developers can easily update them without modifying code throughout the application. Const values may include settings such as application colors, fonts, and themes, as well as database connection strings, API keys, and other configuration parameters.

4.2. ER Diagram.

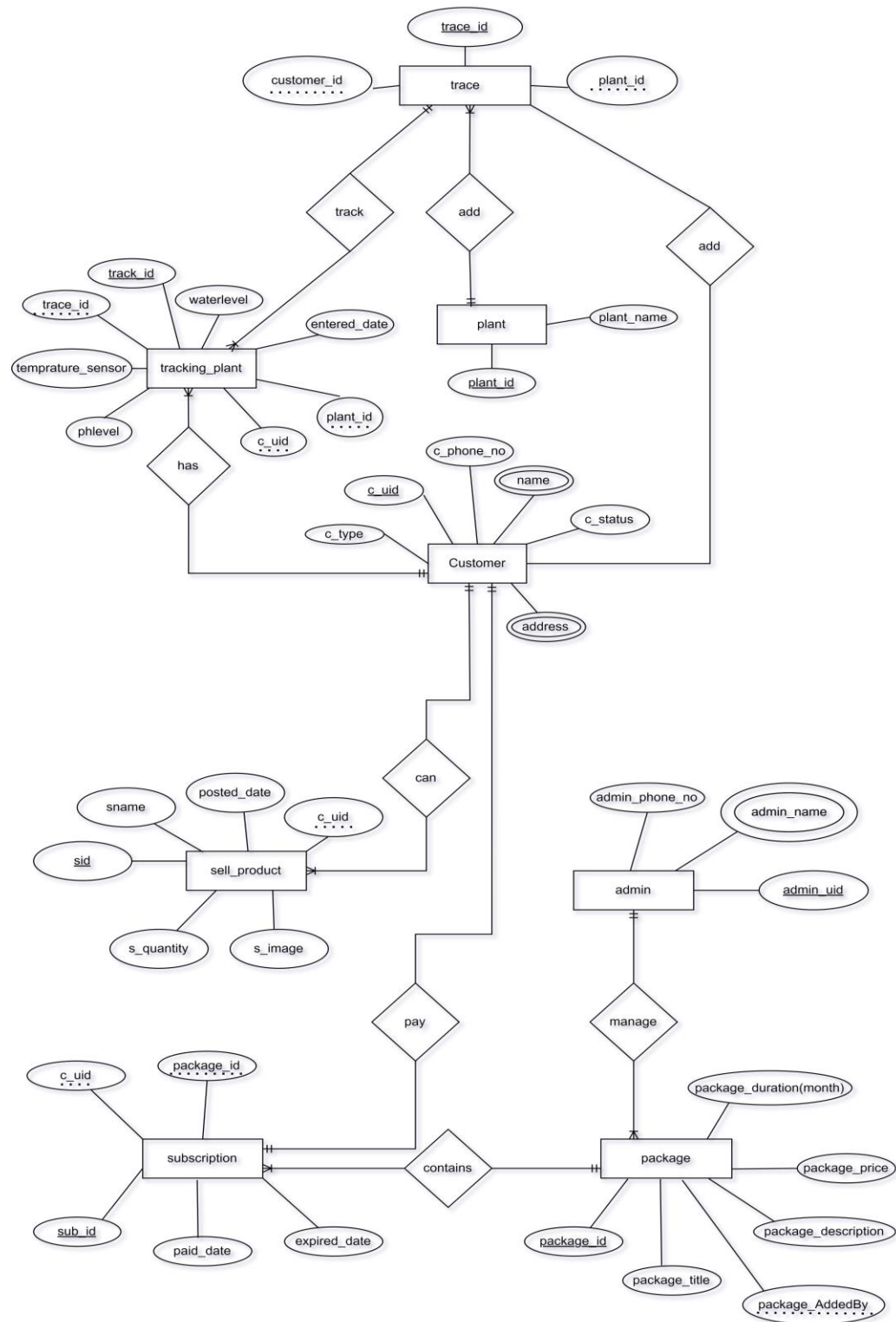


Fig 4.3: ER Diagram of The System

4.3. Database Normalization

MOFER employs two varieties of databases. Both SQL and NOSQL are supported. A SQL database, often known as a relational database, is a form of database that employs SQL (Structured Query Language) as its primary language for organizing and manipulating data. SQL databases are intended to store and handle huge volumes of structured data, and they organize the data using tables. NoSQL (short for "not only SQL") databases, also known as non-relational databases, are a form of database that does not employ the table-based relational model employed by SQL databases. In lieu of this, they employ a number of data models, such as document, key-value, graph, and column-family.

We improve SQL databases using a normalization strategy based on the above specification. Normalization of a database is the act of organizing a database into tables in a way that decreases data redundancy and increases data integrity. It entails partitioning a database into two or more tables and establishing their relationships. Normalization aims to reduce data duplication and guarantee that data is stored in a single location. This facilitates database maintenance, the addition of new data, and information searches.

We do not employ the same normalization procedures for NOSQL databases as for relational databases. But first, why do we need to use the NOSQL database? That is because we use Firebase Authentication feature. Firebase is a well-known mobile and online application development platform that offers a number of backend services, including real-time databases, hosting, authentication, and cloud messaging, among others. Firebase uses a NOSQL database. NOSQL databases are designed to manage vast amounts of unstructured or semi-structured data, and they frequently employ a unique data model, such as document, key-value, graph, or column-family. Instead of normalizing data, NOSQL databases may employ performance-enhancing strategies such as denormalization or duplication. Denormalization is the act of adding redundant data to a database in order to make it easier to query, whereas duplication is the process of keeping several copies of data in different locations in order to enhance read performance. Utilizing mostly three standard forms, we improve the SQL database. Every normal form has its own set of regulations.

4.3.1. First Normal Form (1NF)

According to 1NF (first normal form), each table column must have a single value and each record must be unique. Therefore, based on the preceding rule, we must separate the customer's name column from the customer table into customer fname and customer lname columns, the admin name column from the admin table into admin fname and admin lname columns, and the address column from the customer table into city, kebele, and street columns.

4.3.2. Second Normal Form (2NF)

All non-key columns in a table must be functionally reliant on the primary key, according to 2NF (second normal form). This means that each non-key column must have a direct relationship with the primary key and be independent of other non-key columns. Therefore, we must determine if each non-key property is dependent on their primary key. Therefore, based on the preceding rule, we must delete and replace the city, kebele, and street columns from the customer table with its own address table and associate the customer table with the address table using the address's primary key, address id.

4.3.3. Third Normal Form (3NF)

3NF (third normal form) stipulates that first, the table must be 2NF and second, the table must not have any transitive dependencies. When one non-key column is dependent on another non-key column, this is known as a transitive dependency. To reach 3NF, these dependencies must be eliminated by constructing independent tables. Therefore, the tables are optimized for 3NF.

4.4 Database Table Structure

The normalized SQL database appears as follows.

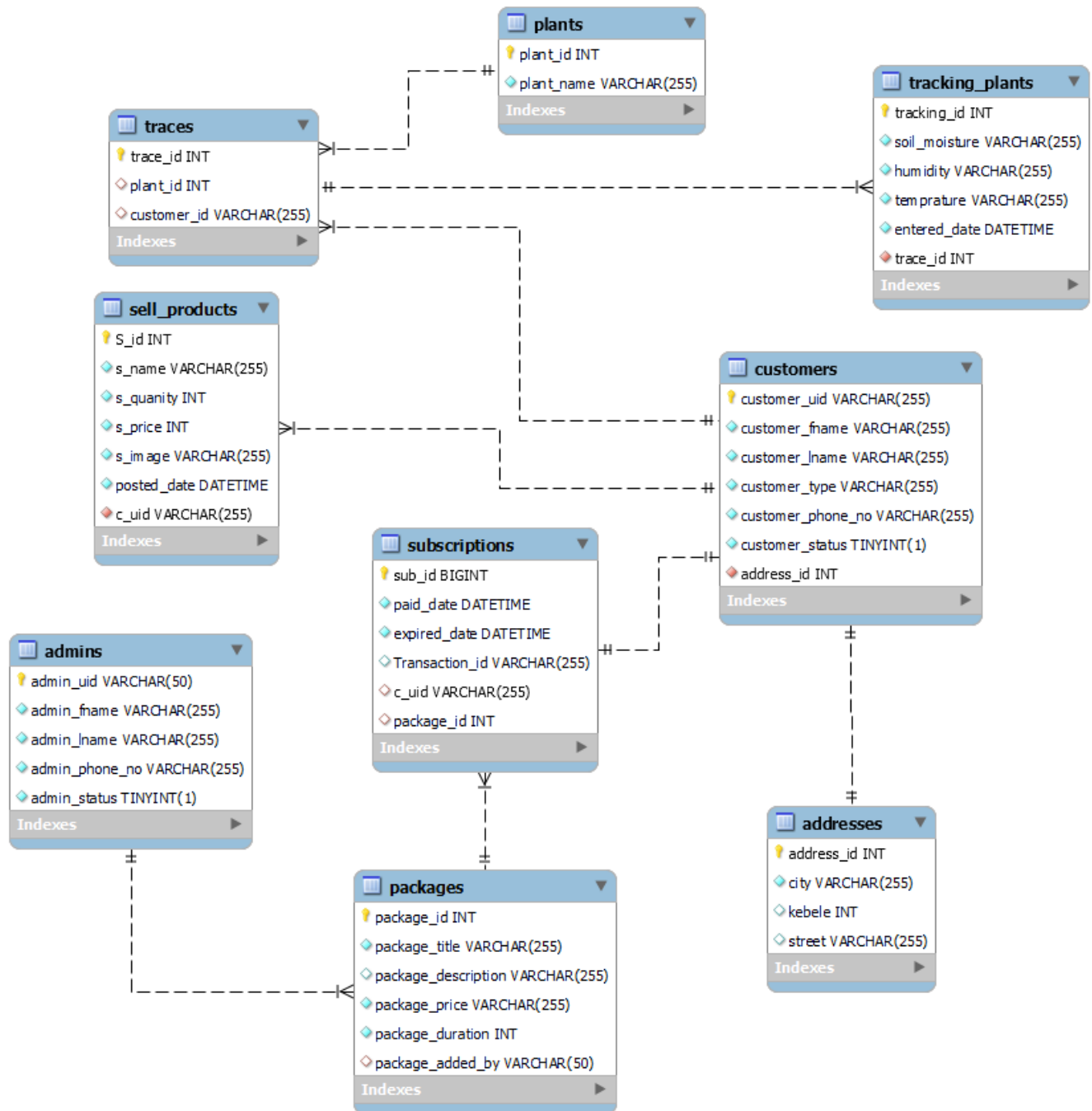


Fig 4.4: Table Schema of The Normalized Database

The firebase (NOSQL) user auth collection appears as follows.

```
{  
  email,  
  password,  
  User UID  
}
```



Identifier	Providers	Created ↓	Signed In	User UID
------------	-----------	-----------	-----------	----------

Fig 4.5: Firebase auth table

4.5 Deployment Diagram

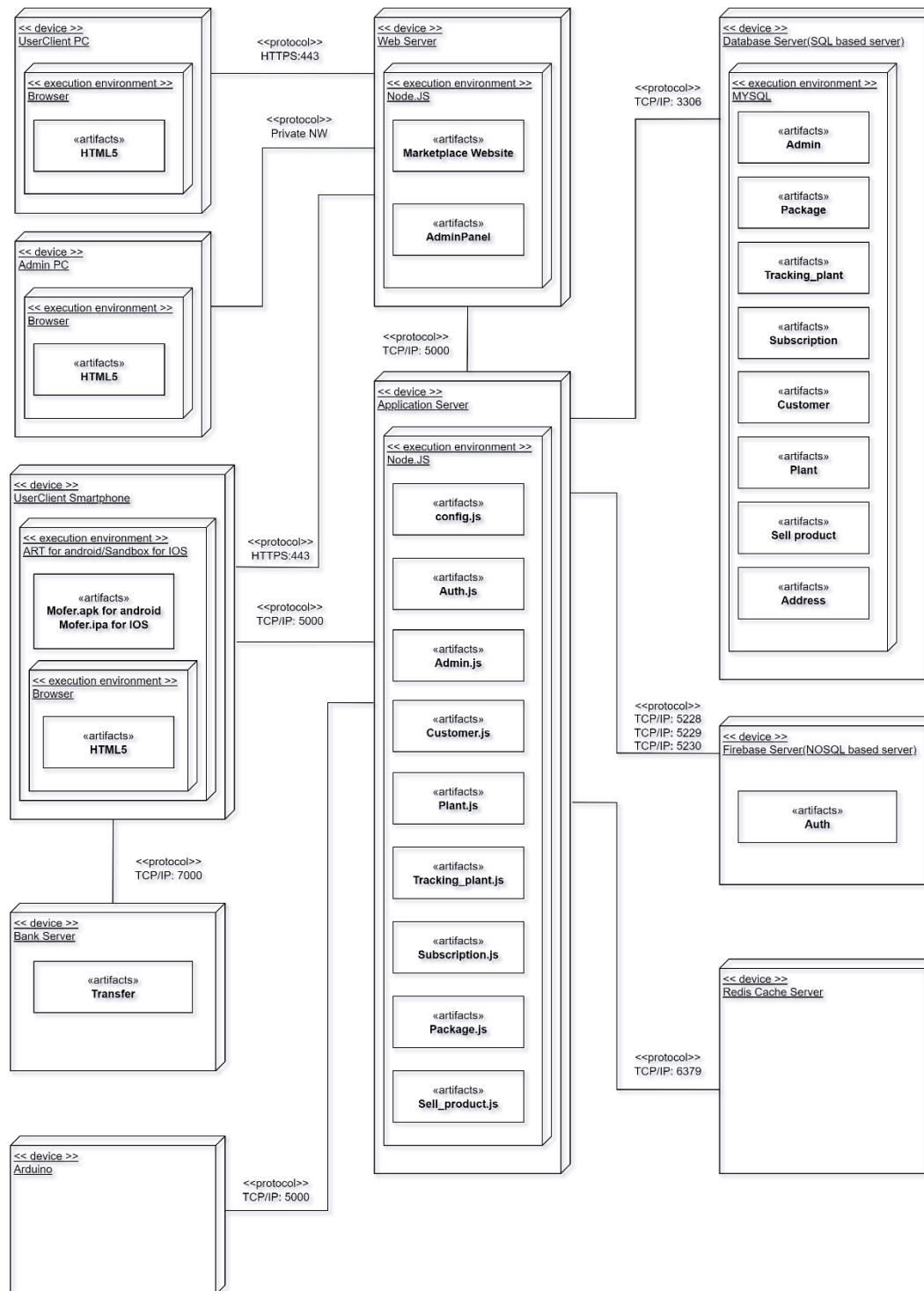
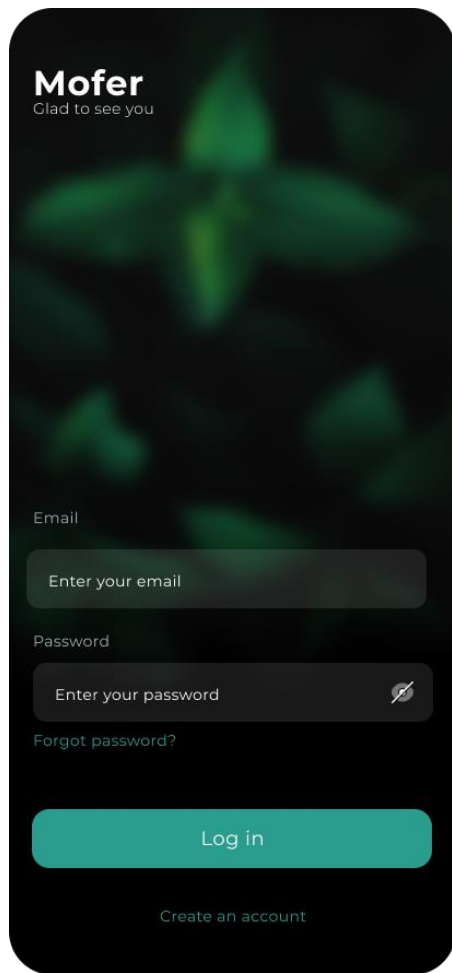


Fig 4.6: Deployment Diagram of The System

4.6. High Fidelity Prototype



The image shows a high-fidelity prototype of a login page for an app called "Mofer". The background is dark with a faint, abstract green pattern. At the top left, the app name "Mofer" is displayed in white, with the tagline "Glad to see you" underneath. Below this, there are two input fields: one for "Email" and one for "Password". The email field has a placeholder text "Enter your email". The password field has a placeholder text "Enter your password" and a small icon of a crossed-out eye to its right. Below the password field, there is a link that says "Forgot password?". At the bottom of the form, there is a large, rounded rectangular button with a teal gradient and the text "Log in". Below the button, there is a link that says "Create an account".

Fig 4.7: High Fidelity Prototype of the login page

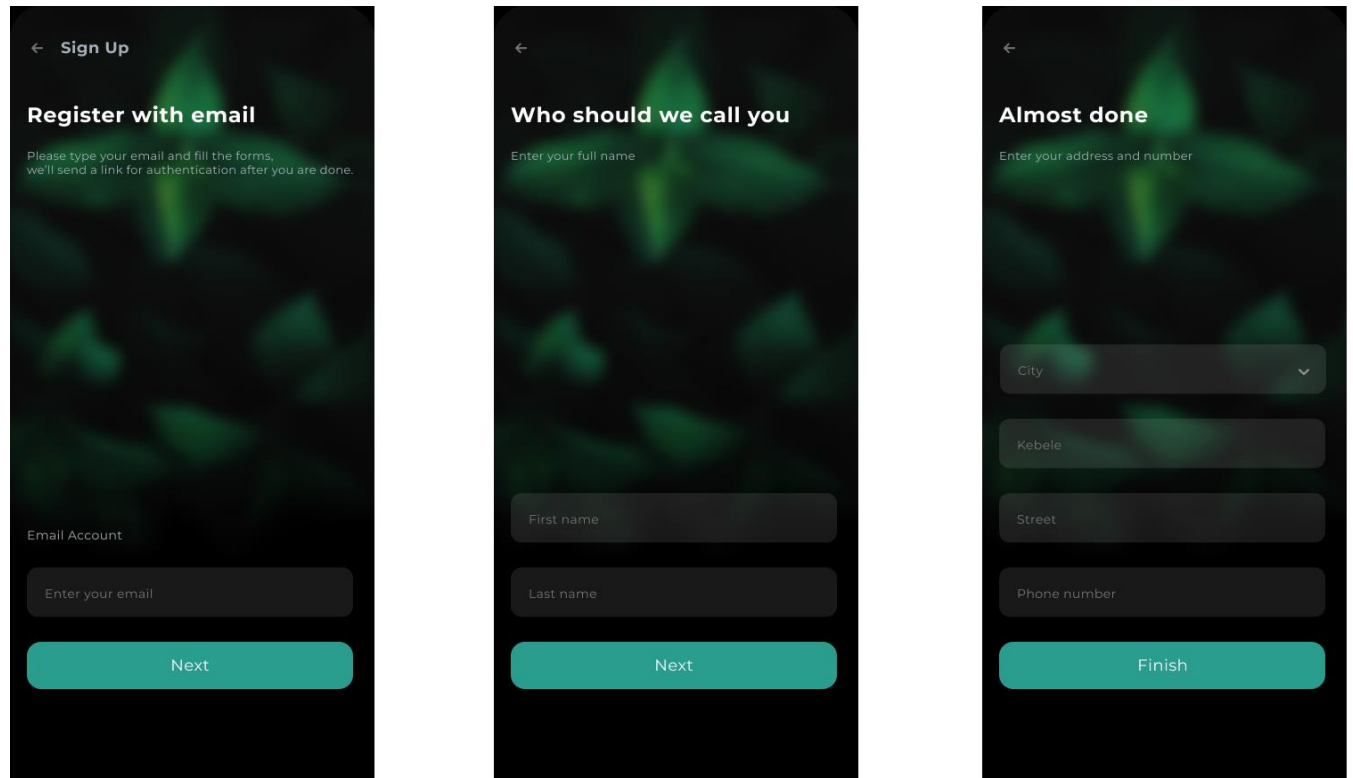


Fig 4.8: High Fidelity Prototype for the sign-up pages.

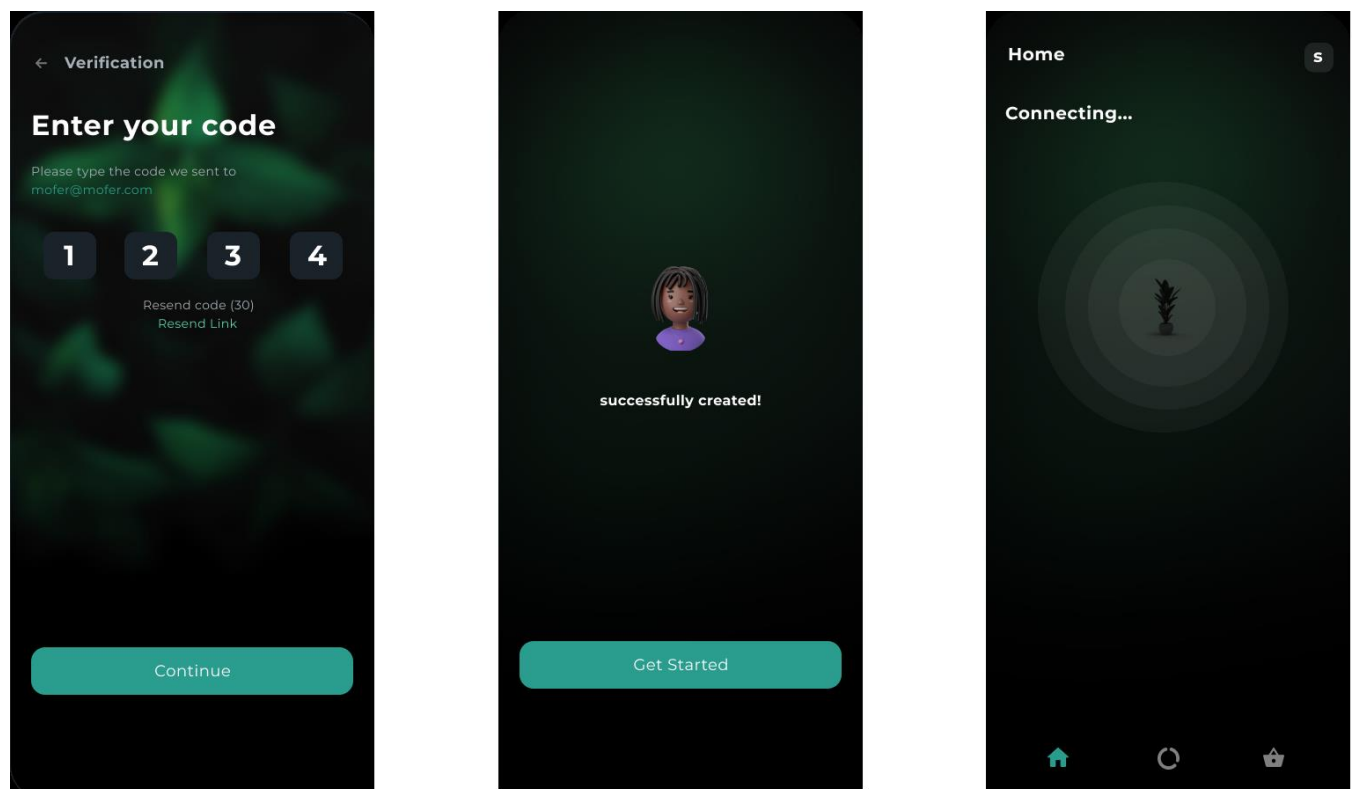


Fig 4.9: High Fidelity Prototype for the verification, loading, successful message shower pages.

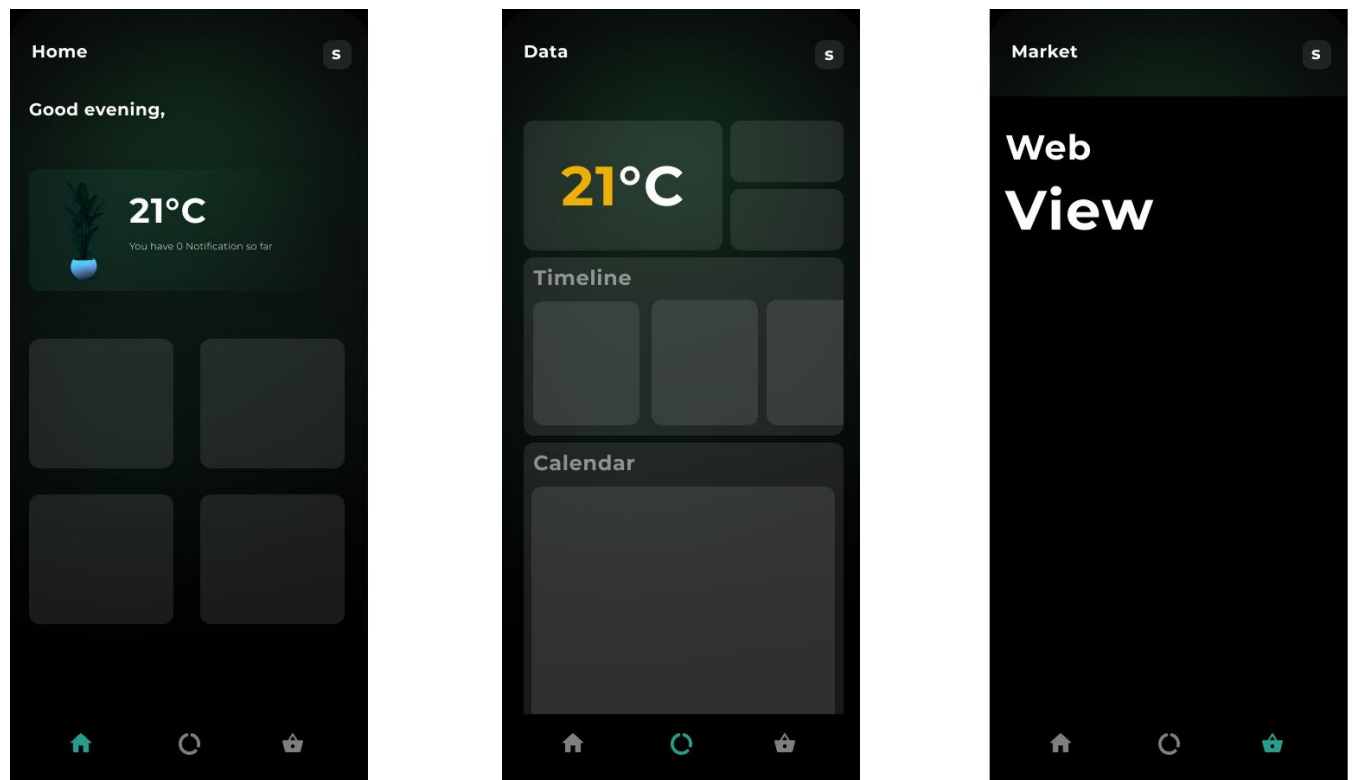


Fig 4.10: High Fidelity Prototype for the Home, data and marketplace pages.

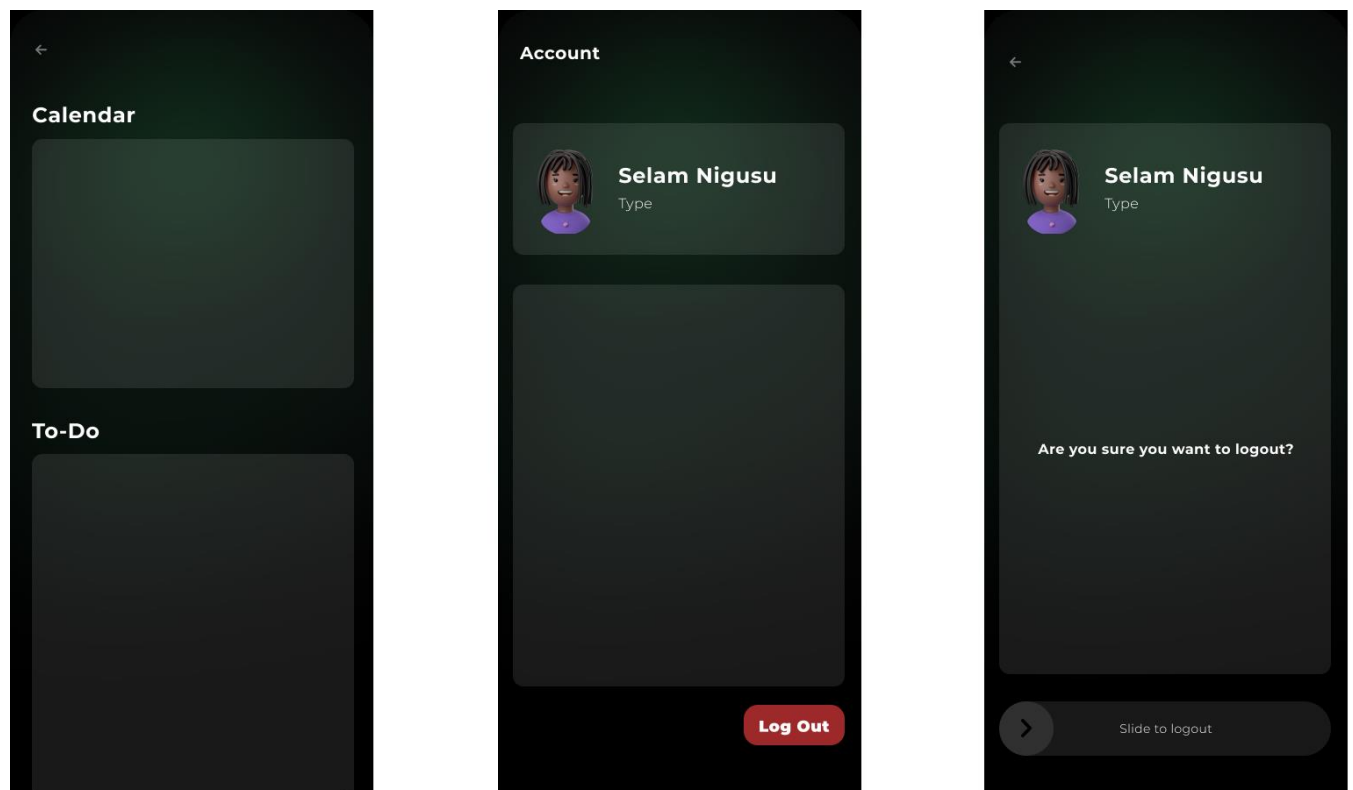


Fig 4.11: High Fidelity Prototype for the calendar and manage account pages.

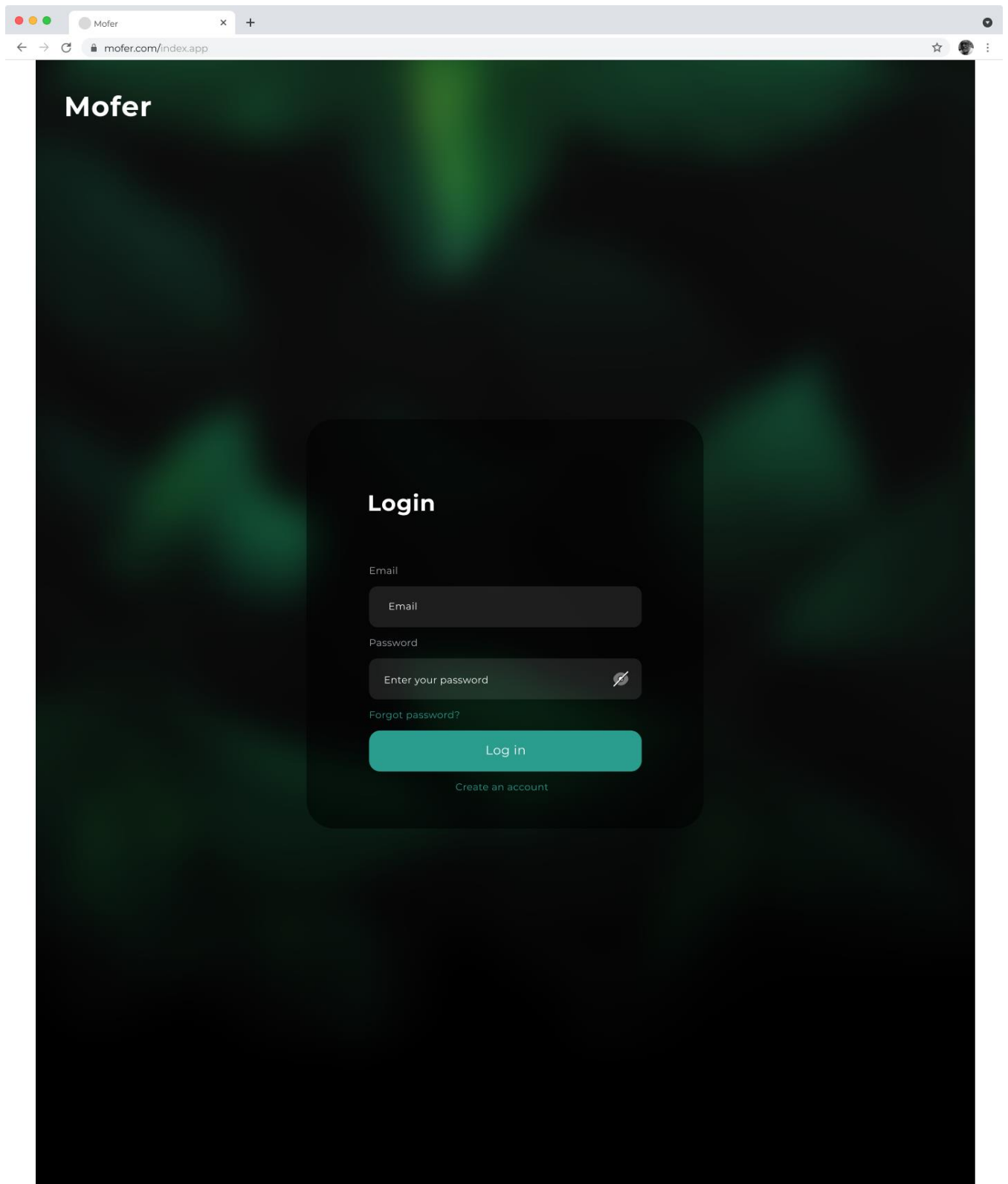


Fig 4.12: Marketplace login page

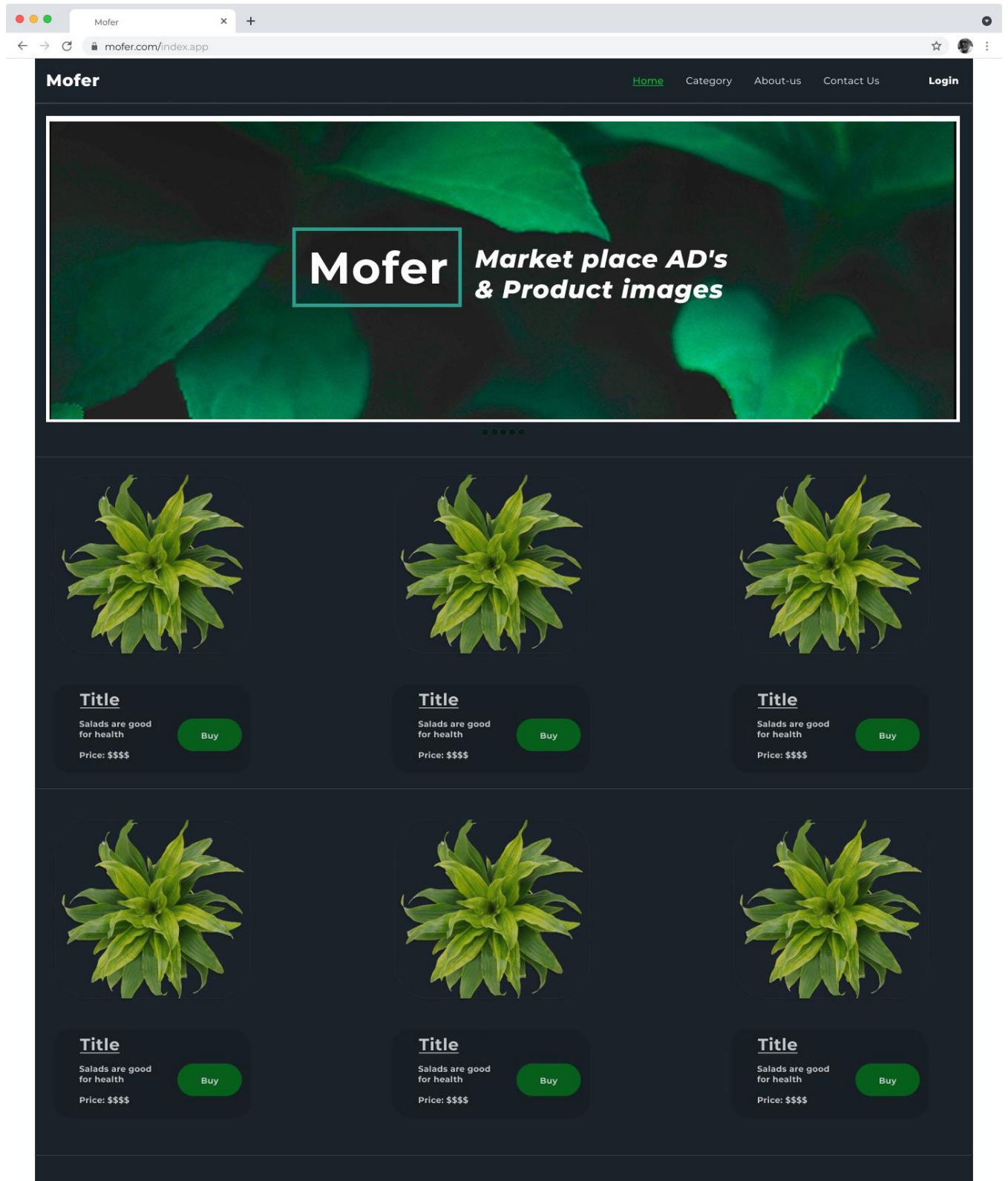


Fig 4.13: Marketplace Homepage

Mofer



Selam Nigusu
Online

Home

User

Admin

Product

Setting

Sign out

User Control

User / Home

Search

☐ All	Full Name	Full Name	
☐	 Selam Nigusu	Selamnigusu123@gmail.com	Ban
☐	 Selam Nigusu	Selamnigusu123@gmail.com	Ban
☐	 Selam Nigusu	Selamnigusu123@gmail.com	Ban

showing 1 to 3 of 3 entries

1 - 2

2023. Mofer lot based urban agriculture

Fig 4.14: Admin panel

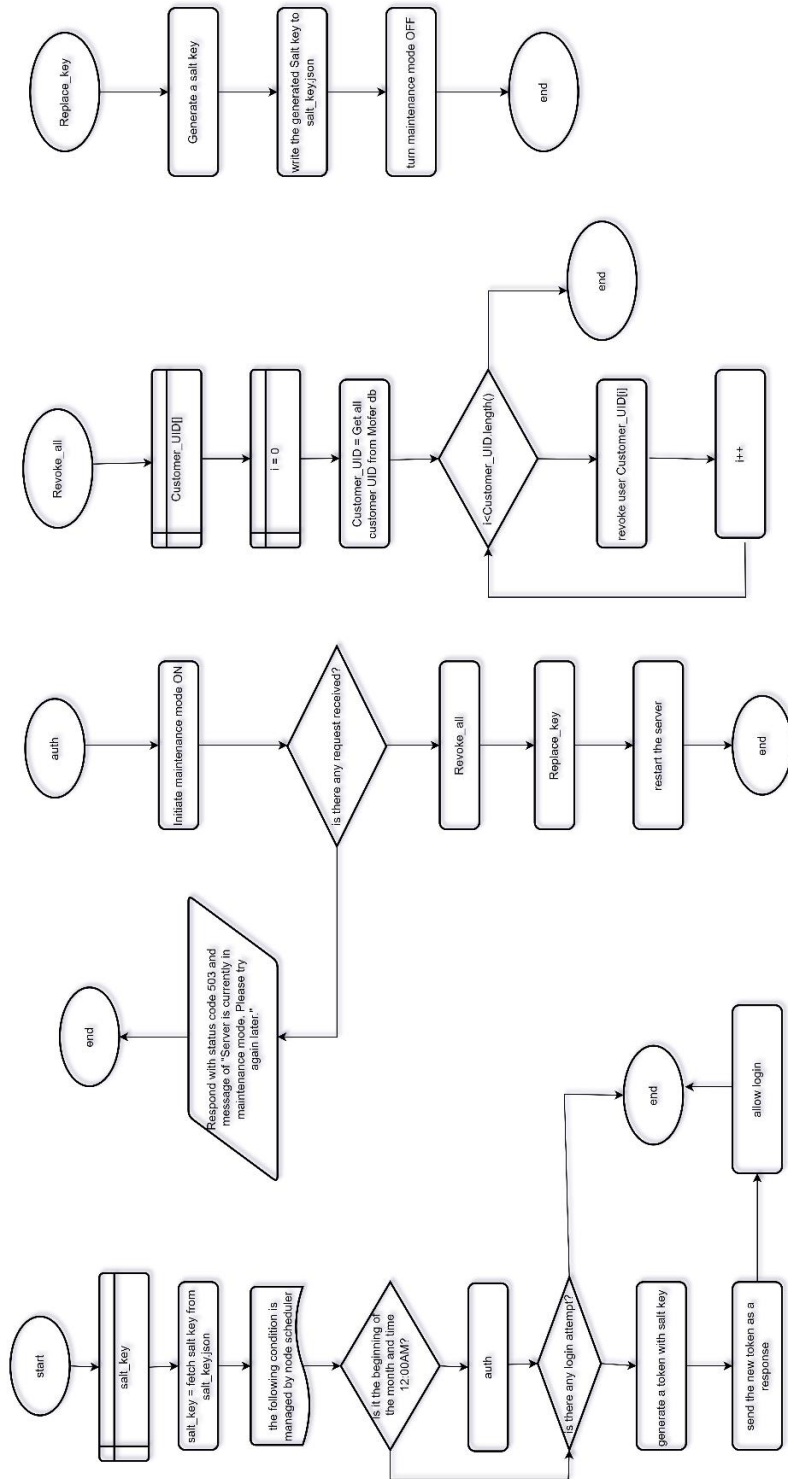
Chapter Five

Implementation

5.1.Introduction

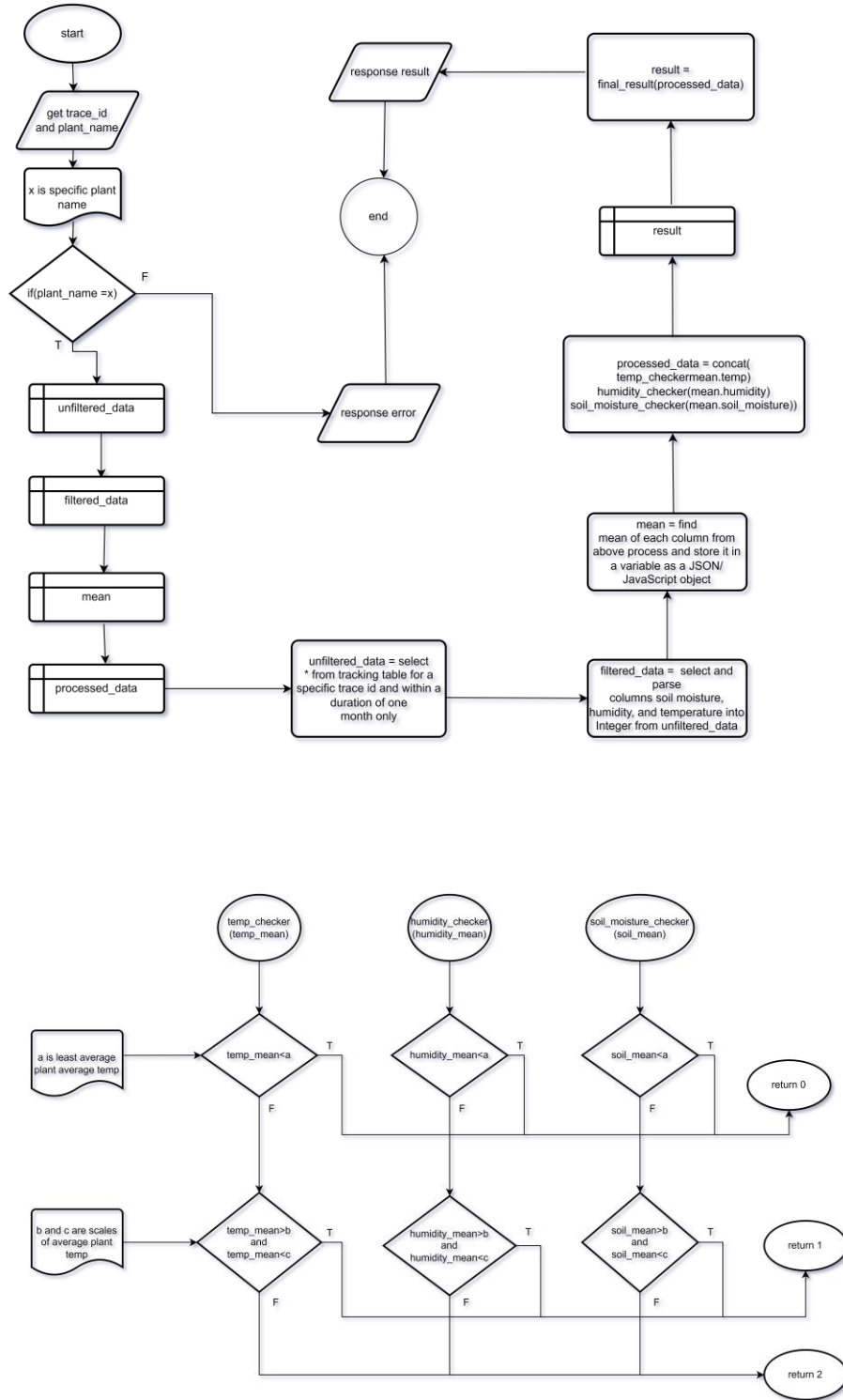
In the implementation phase, we incorporated an artificial intelligence plant image detector into the MOFER API to filter valid and prohibited posts. This AI facilitates the detection and removal of prohibited posts on MOFER Marketplace, which was designed and constructed for Plant-only postings. In prior chapters, we mentioned that we utilized Firebase. However, firebase servers are located at a bit of a distance from Ethiopia, resulting in latency issues that negatively impacted user experience efficiency. We therefore implemented a caching system to resolve the issue. This cache resolved the latency problem caused by firebase. Based on our analysis, it revealed a nearly 80% improvement.

Every system lacking robust security is useless. Therefore, we concentrated heavily on developing a secure system. Firebase auth served us our authentication system. Additionally, we develop our own authorization system that operates on top of the firebase system to assure a high level of security. Because of the above features, we can have the ability to ban users from our platforms in runtime. Besides that, we add SQL injection prevention mechanisms. We also developed an online banking system that helps us to simulate the subscription feature between the user on MOFER status tracking app. We tried to simulate the real banking system. Some of the algorithms that we created are:

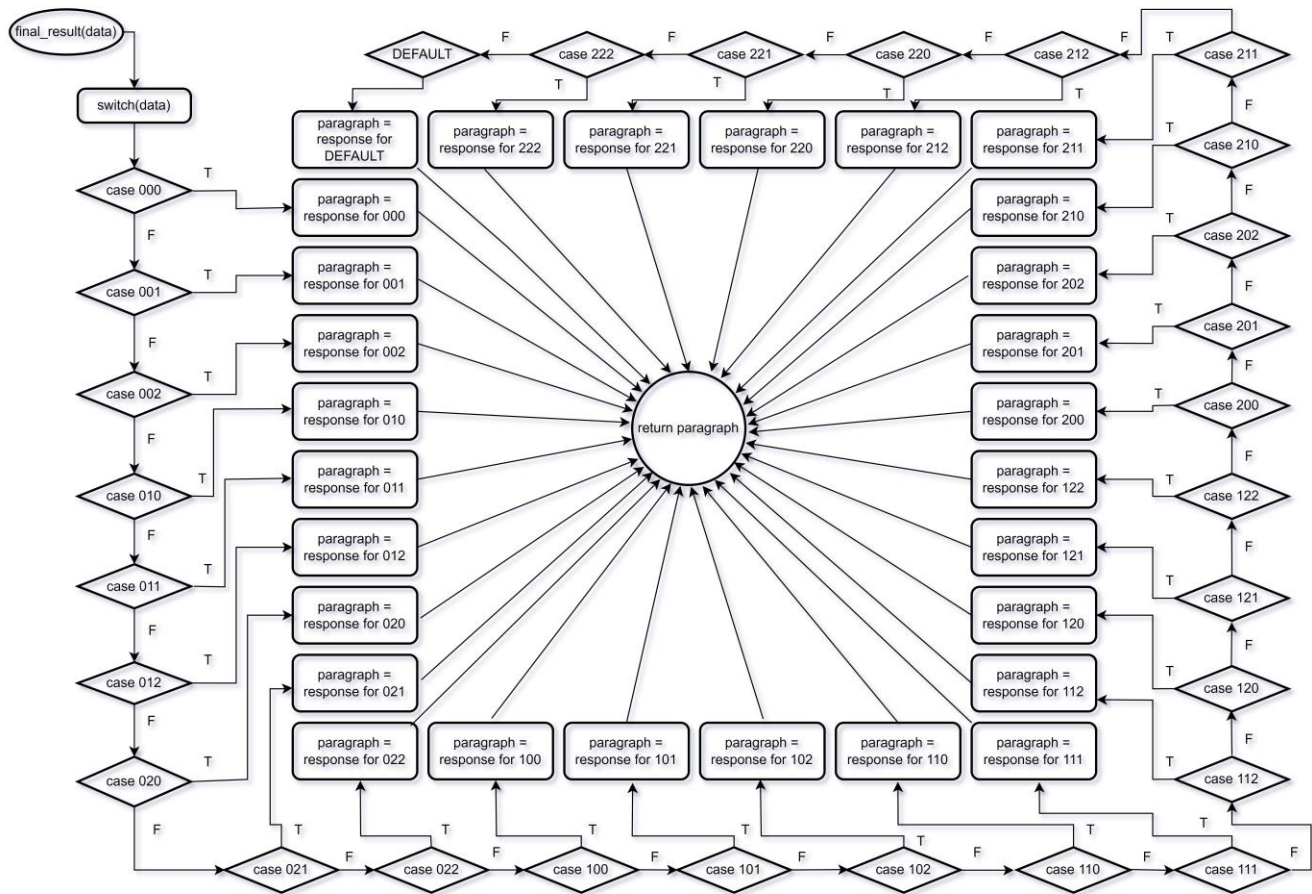


Algorithm 1: dynamic token generation algorithm

The above algorithm shows how new tokens are generated within the iteration of a 1 month.



Algorithm 2: data analytics read from monthly data and plant state conversion.



Algorithm 3: plant recommendation based on the data analytics.

The data analytics algorithms illustrate how raw data was obtained from the database, filtered, and then used to generate reports.

5.2 Development tools

We will try to describe the tools used to develop the MOFER System. Here is the list of tools and the programming language used to develop this project.

Microsoft Visual Studio a popular and incredibly flexible code editor called VS Code is frequently used to create a wide range of applications, including mobile and web site in our case MOFER. VS Code offers a number of plugins and extensions that streamline and increase the effectiveness of the development process while creating a mobile and web application like

MOFER. For instance, VS Code offers extensions for mobile development frameworks like Flutter as well as plugins for well-known web frameworks like React. Additionally, the integrated debugging tools and code navigation features in VS Code can aid programmers in finding and fixing mistakes in their code rapidly.

Insomnia API Tester is a powerful tool for testing and debugging APIs. It offers a user-friendly interface that makes it simple and quick for developers to create requests, examine answers, and troubleshoot problems. Insomnia API Tester is a fantastic option for developers looking for a dependable and effective way to test their APIs because it includes features like automatic request generation, support for numerous authentication methods, and a built-in code editor.

MYSQL Server allows developers to store, manage and retrieve data in a structured and efficient manner. In this case, the MYSQL Server provides a powerful and flexible platform for developing and managing databases.

Firebase is a scalable cloud solution provided by Google, where data is saved on a collection into Cloud Storage or Real Time database.

Express JS is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It is used on the backend. It uses a programming language called JavaScript. We have used it to build our APIs for both mobile and web app. Flutter is an open-source framework by Google for building beautiful, natively compiled, multi-platform applications from a single codebase by using a programming language called Dart.

React JS is an open-source JavaScript library developed by Facebook. It is mainly used for building user interfaces for single-page Web applications and mobile applications. In our case it is used to build the admin and marketplace panels.

5.3 prototype of the system

In this part we try to show the activities done by the actors of the system. We try to show the three actors' workflow with diagrams and descriptions.

5.3.1 Farmers

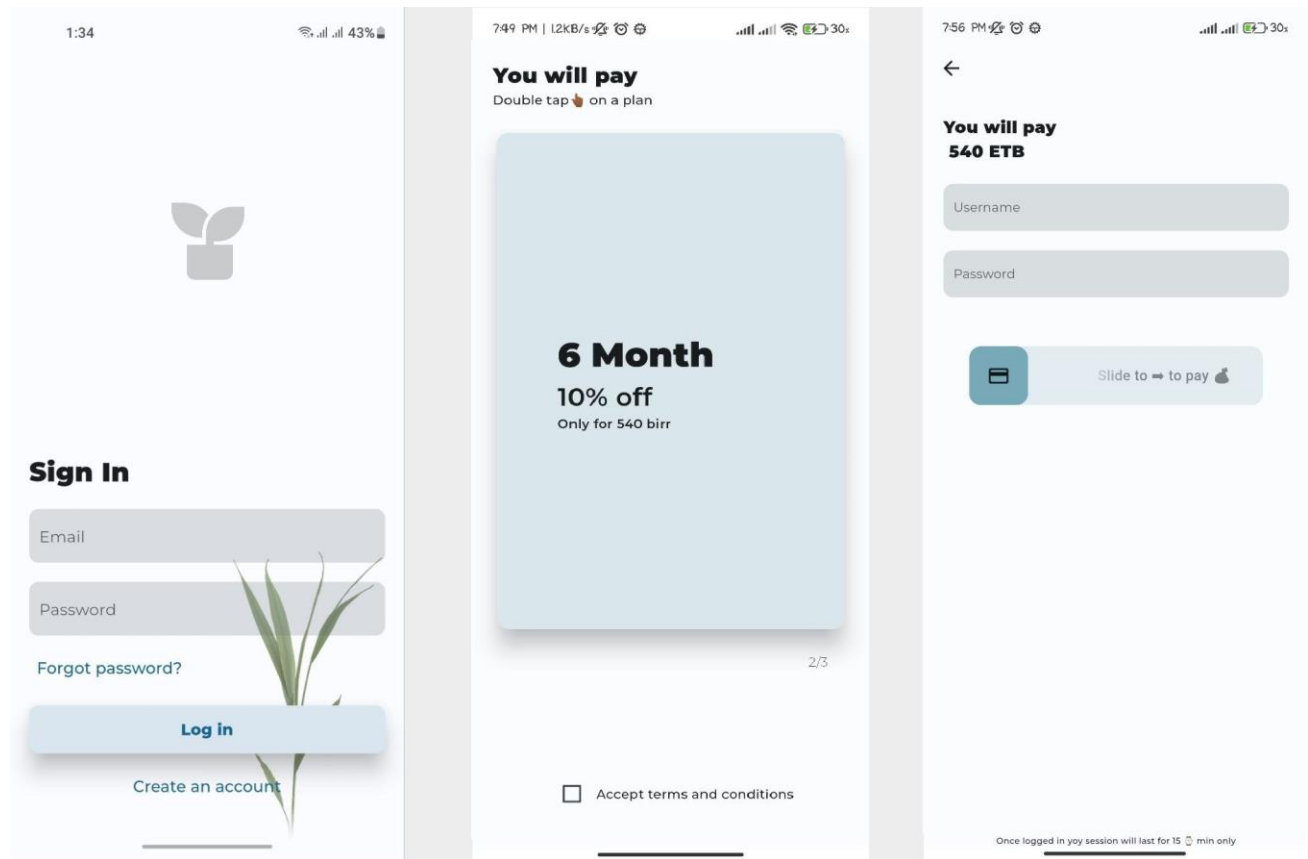


Fig 5.1 sign in, list all subscription packages, bank login pages screenshot.

This are the first three pages you will encounter for the application the first page is log in where the user enters his/her username and password after clicking the login button the system will authenticate the user and pass to the second page that will list the available packages under the system after that the user will double tap on the package that is fitted for him/her. If the user forgets his/her password, he/she will click on the forgot password link to go to the password recovery page where they will proceed to recover their password. The third and last page will be the bank login page for payment where the user is expected to enter his/her username and password and slides to pay for the subscription.

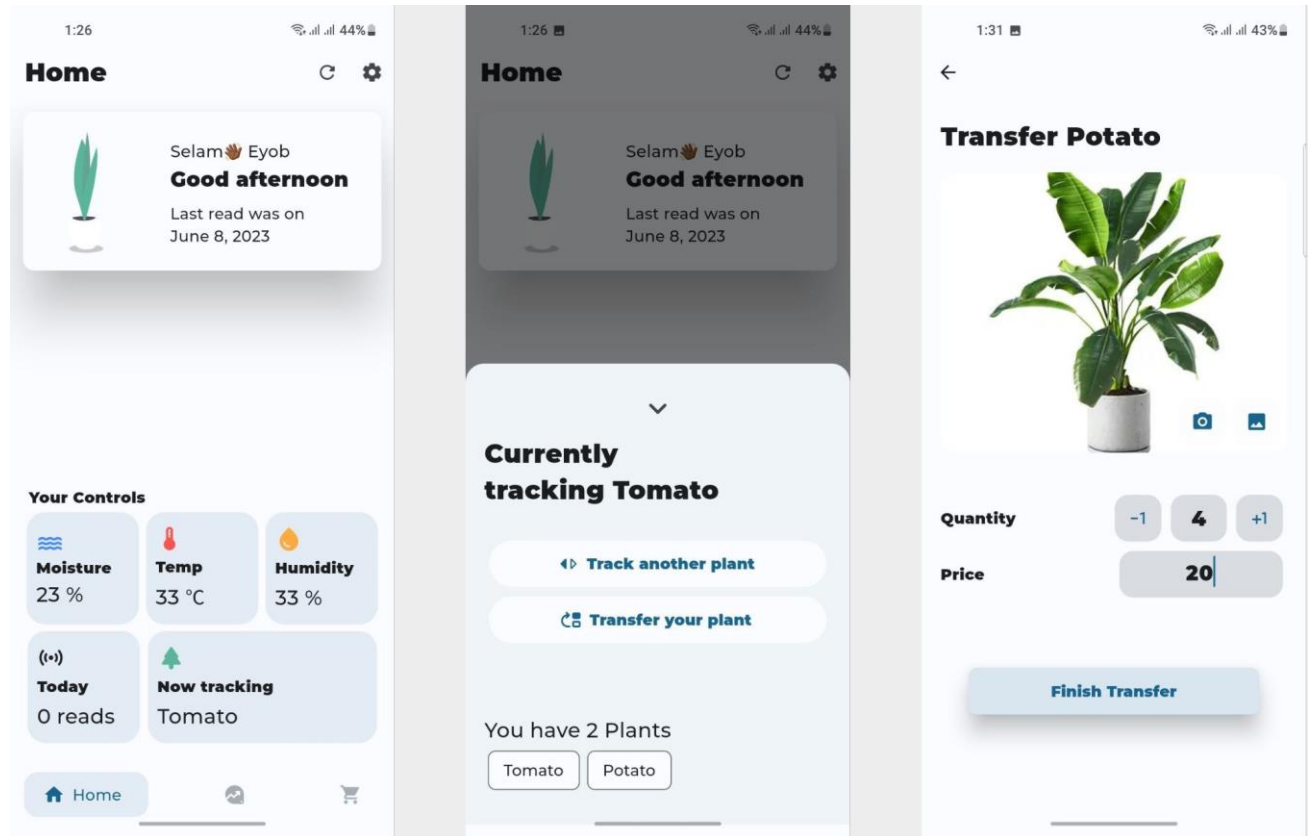


Fig 5.2: Home, view plants and transfer plant pages screenshot.

The first image is the homepage of the system where the user will see all the information about his/her plant where it displays the moisture, temperature, and humidity of the plant other than that the user can shake the phone to get a new update. When the user clicks on the tracking button the second page will be shown with the list of all the plants the user is tracking at the time other than that the user can transfer their plant leading to the third image of the above figure where the user can enter all the information needed and transfer the plant to the marketplace.

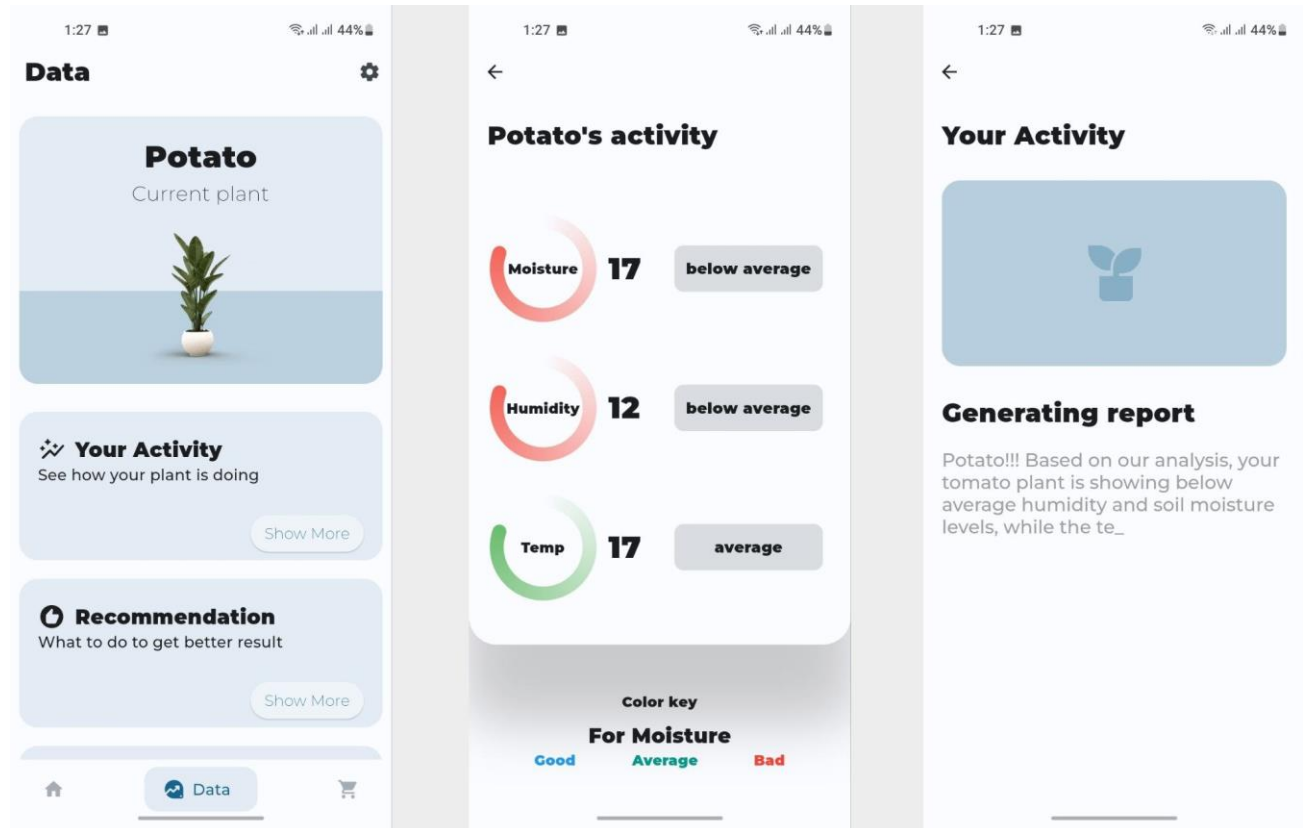


Fig 5.3: Data, plant activity and plant recommendation pages screenshot.

The data page is the second page of the application where the user is provided with three choices to know the details of the plant, he/she is growing. your activity showing the plant status compared to the normal status of the plant they are growing which is shown on the second image. The third image shows the recommendation page of the application where the system suggests a report that is useful to the plant based on the state it is found in.

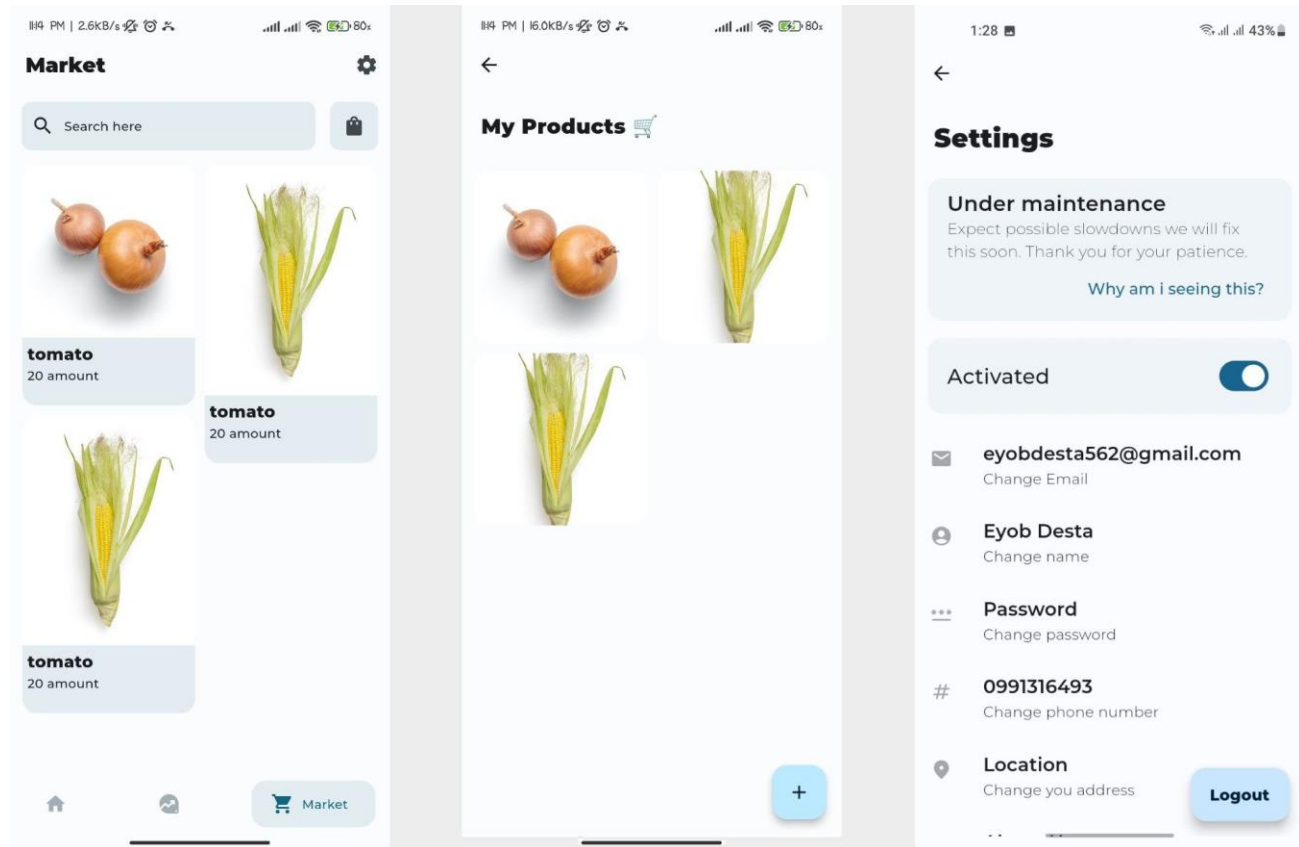


Fig 5.4: App Marketplace, my products, and settings pages screenshot.

The third and final page of the system is shown in the first image where the marketplace is found, and all the available plants are listed on the application. next to that we will find my products page where it will display all the available list of products that is posted by the current user. The last but not the least page on the application is the settings page where the user can change and customize to their liking.

5.3.2 New sellers

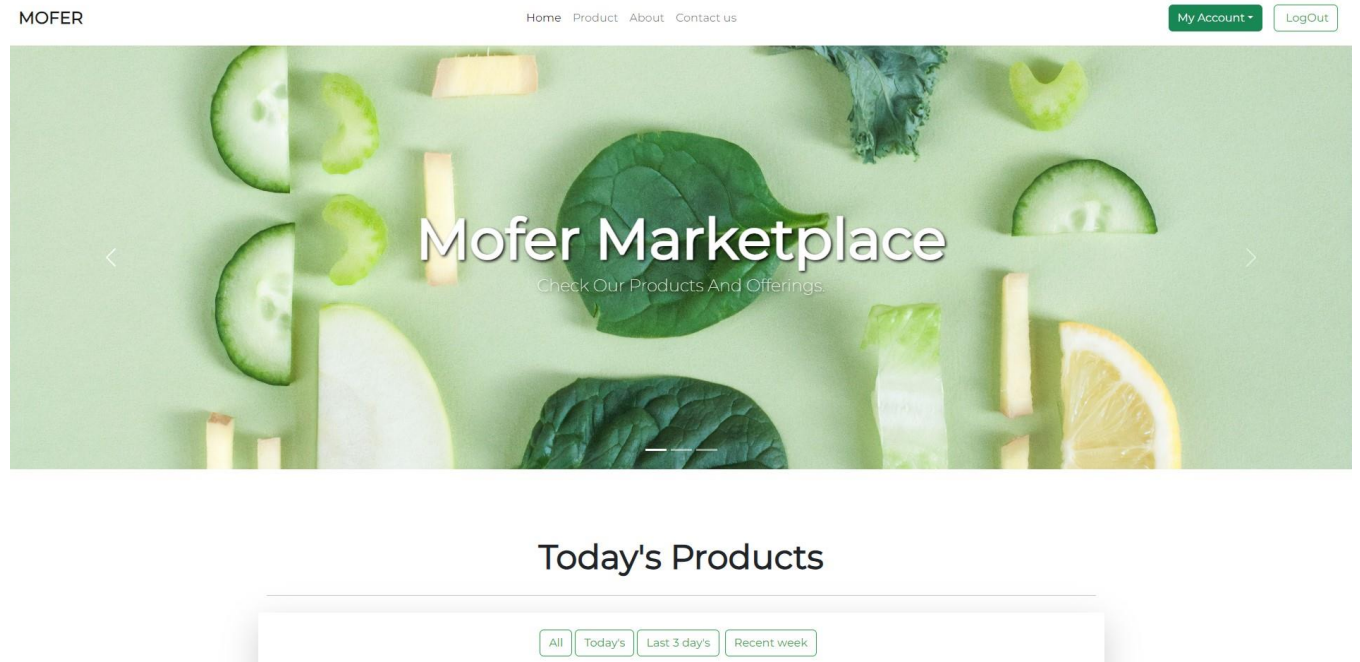


Fig 5.5: MOFER Marketplace web screenshot

This is the homepage of the system that welcomes the user to the MOFER system.

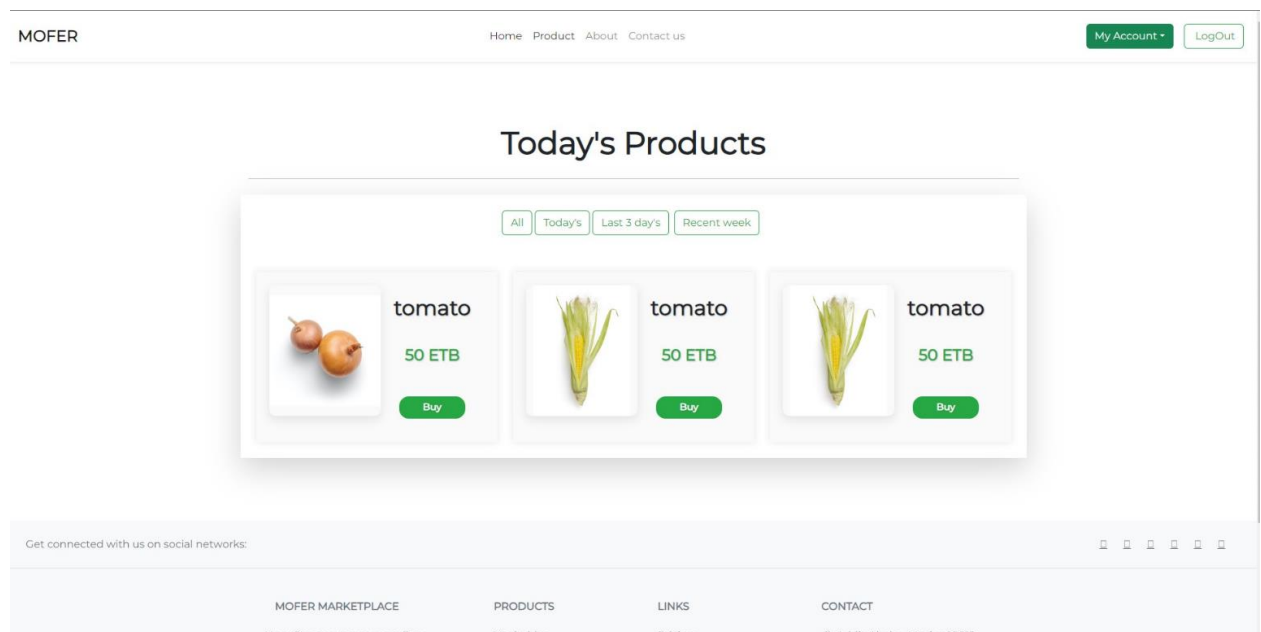


Fig 5.6: MOFER Marketplace product page screenshot.

This is the second page of the marketplace where the user will see all the products available by different mechanisms of filtering by date.

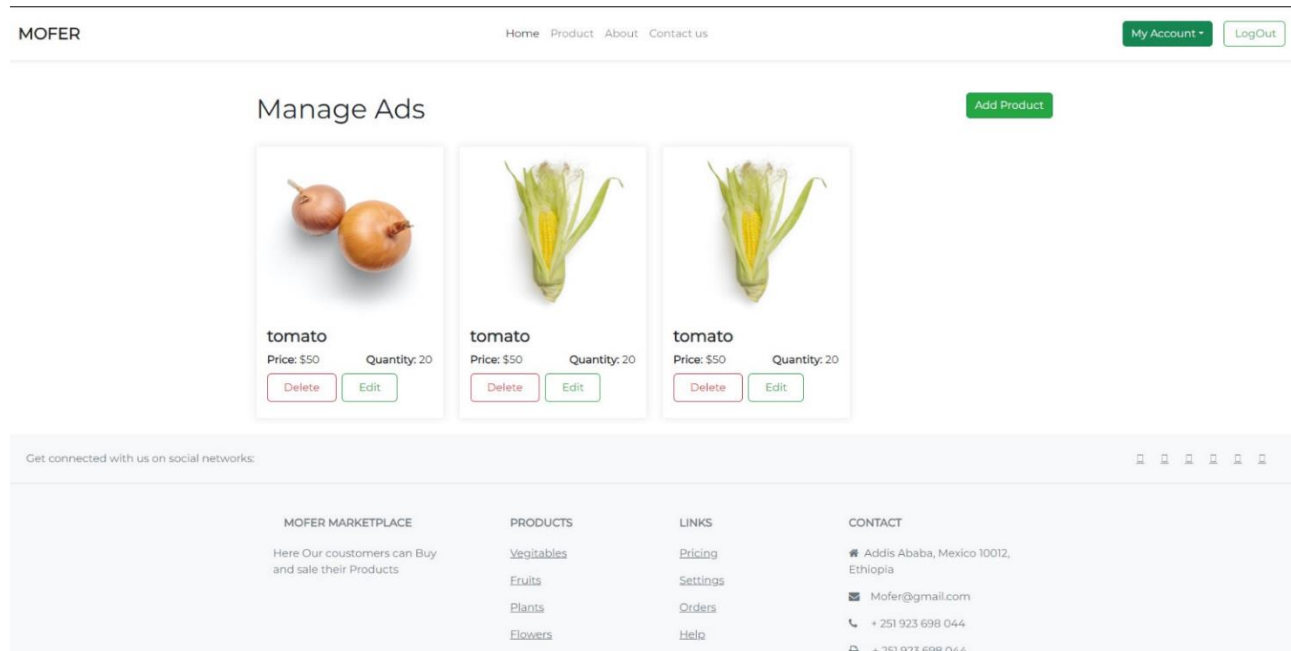


Fig 5.7: MOFER Marketplace Manage my Ads page screenshot.

The page above refers to the marketplace for a seller where the user can view, add, delete, and edit the products that are posted by him/her.

5.3.3 Admin

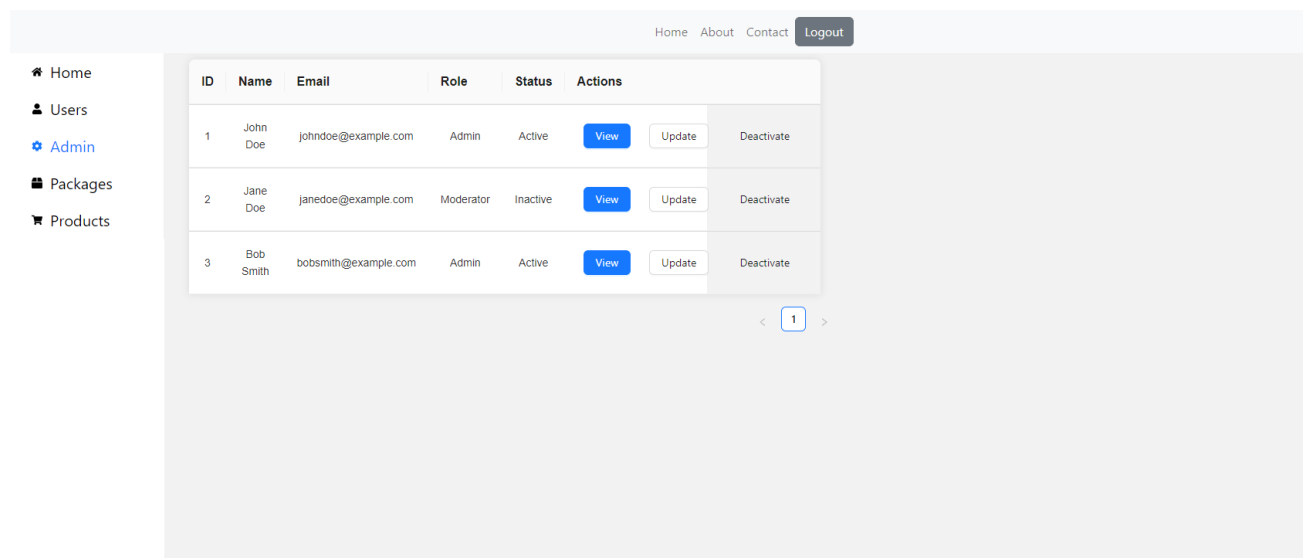


Fig 5.8 Admin panel manages my admin page screenshot.

The image above is found on the admin portal which is used to view the admins of the system.

5.4 Testing

We try to test the system to verify if the software developed meets its intended requirement and functionalities.

5.4.1 User Accepting Testing

This test is done by the end user to test the system before moving the application to deployment environment.

5.4.1.1 Blackbox testing

In the black box test, we will test the functionality with input and see the outputs meet the functional requirement of the document. Sometimes we call it 'Behavioral Test'. The testers guess the error and check if the system is working properly. The testers first test every functionality written in the system test dividing them with every actor. After that they try to see if there are validation errors in the inputs and if the system accepts null values. They test every input in the system, to name a few add patient in the reception side if the input validation works properly, doctor prescribing medicine, physician inserting test results, cashier updating bill and pharmacist inserting prescribed medicine detail.

5.4.1.1.1 System test

- ❖ User (farmers)
 - Registration.
 - Login.
 - Password recovery.
 - View Tracking plant.
 - Generate plant data analytics.
 - Push product to marketplace.
 - List subscription package.
 - Pay for service.
 - Manage profile.
 - View products.
- ❖ User (new sellers)
 - Registration.

- Login.
- Password recovery.
- Manage profile.
- View all products.
- View a specific product.
- Add and Manage product.
- Deactivate account.
- ❖ User (buyers)
 - View all products.
 - View a specific product.
- ❖ Admin panel
 - View product.
 - Remove product.
 - Add subscription package.
 - View subscription package.
 - Login

We will see the two functionalities in detail.

Table 5.1: sign up unit test.

Unit to test – sign up process on MOFER STATUS TRACKING APP		
Assumption – user registering successfully		
Test data – first name, last name, email, phone number, city, kebele, street, password		
Steps of expected	Data	Expected result
When the user has not inserted first name and last name but clicks the next button	empty	“Enter a first name” “Enter a last name”
When the user has not inserted email and phone number but clicks the next button		“Enter your email” “Enter your phone number”

When the user has not inserted city, street or kebele but clicked the next button		“Enter a city”, “Enter a street”, “Enter a kebele”
When the user has not inserted password and confirm password but clicks the next button		“Password is too short” “confirm your password”
When the user has inserted password and confirm password mismatches	Mismatched input	“Password does not match.”
When the user has typed a wrong email and phone number	invalid input	“Enter a proper email” “enter a proper number”
When the user has typed all fields with valid information	Email and password	Email will be sent to the user email address for confirmation
When the user has confirmed his/her email and clicks the check email button	all fields with Valid data	The API checks whether the user confirmed his/her email and save the data to the database and return a successful message

Table 5.2: password recovery unit test.

Unit to test – password recovery on MOFER STATUS TRACKING APP		
Assumption – user password recovered successfully		
Test data – email, password		
Steps of expected	Data	Expected result
When the user has not inserted email but clicks reset	empty	“Enter your email”
When the user has not inserted password		“Enter your password”
When the user types in an invalid email	Invalid input	“Enter a proper email”

When the user inserted a valid email but does not exist in the database	Email that does not exist on the database	“This email does not exist, please insert the correct email”
When the user password is less than 6 characters long	invalid input	“The password must be at least 6 characters long”
When the user has typed all fields (new password) with valid information	New password	The password will be reset. Then the user can login

5.4.1.2 Whitebox testing

We test the code and try to describe some of the code used in the system to see how the system is working perfectly fine. In this test we will test the flow of the specific inputs through the code and expected output. We can try this using the receptionist patient registry. The receptionist inserts the data in the User Interface.

5.4.1.2.1 Unit test

5.4.1.2.2 Integration test

Table 5.3 Integration test

Integration test	
Test condition	Expected condition
When the database is disconnected	It displays an error message “Opps 🙄 No route to host”.
When the API connection timeout	It displays an error message “Opps 🙄 Connection time out”.
When the connection is lost	It displays an error message “Opps 🙄 Software caused connection abort”.
When the server is on maintenance mode	It displays an error message “Opps 🙄 currently the server is on maintenance mode. Please try it again later”.
When the user is banned from the platform	The user will be forcefully logged out then when the user tries to login again it will show an error message “user account is disabled. Please contact the MOFER team”.

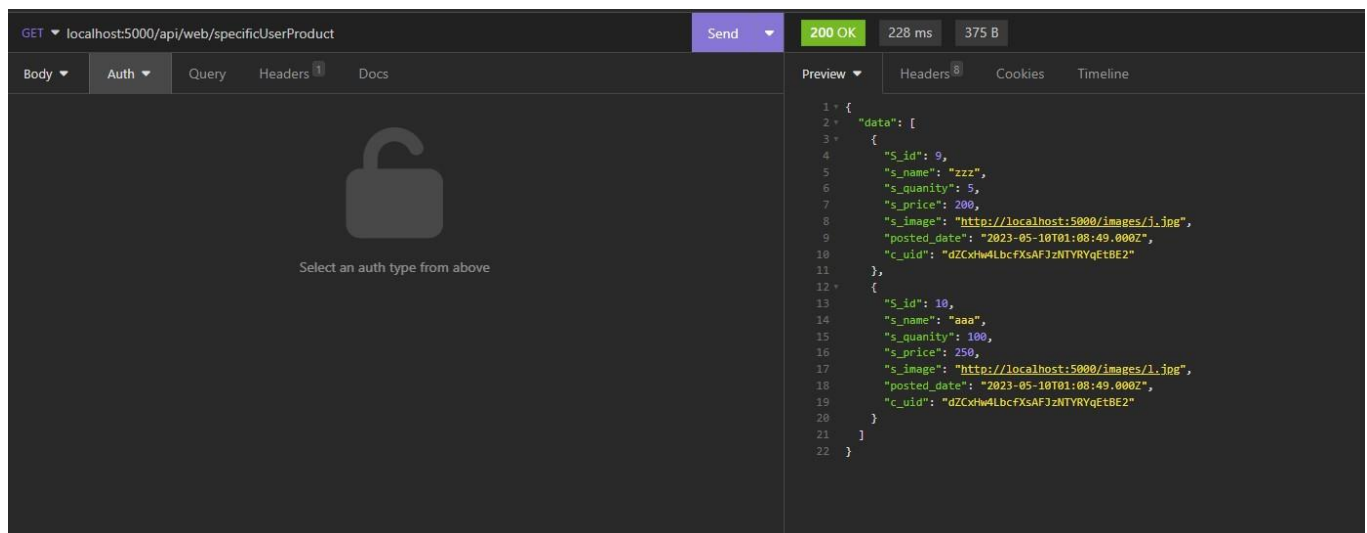


Fig 5.9: specific user product endpoint API testing

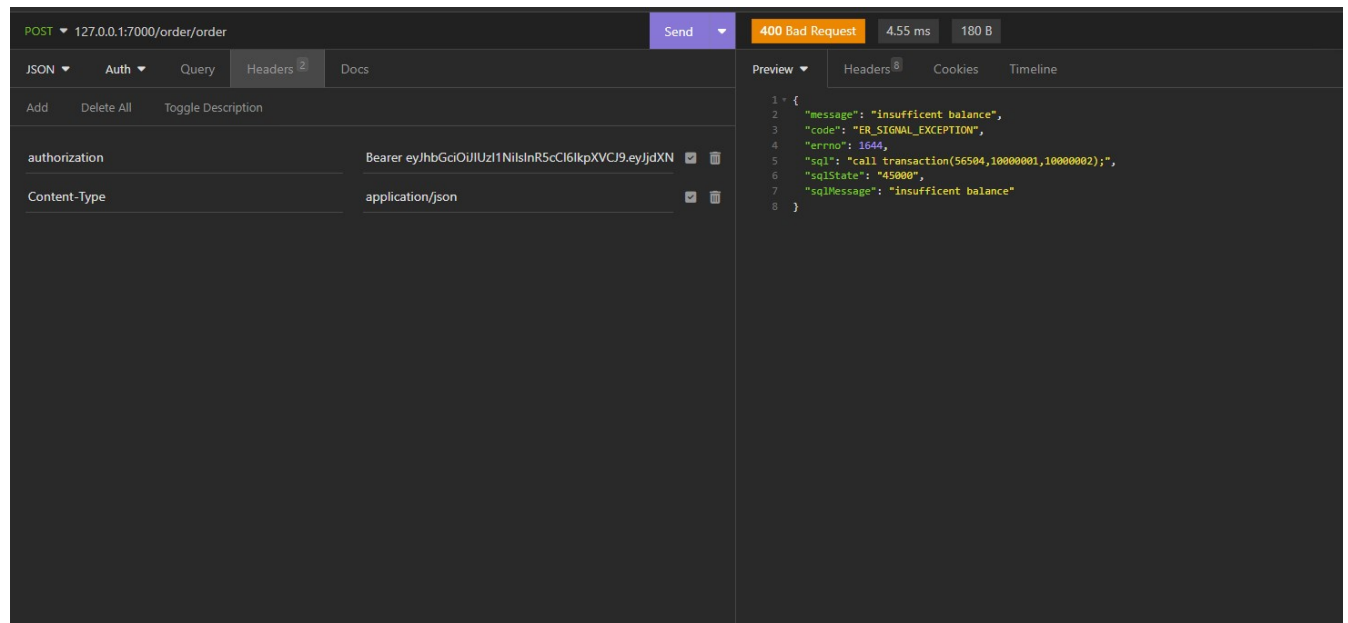


Fig 5.10: Bank order endpoint API testing

Chapter Six

Conclusion and Recommendation

6.1. Conclusion

In conclusion, the IoT-based urban agricultural service provider application MOFER intends to address the existing issues urban farmers in Ethiopia are facing. Since the global population is projected to reach 9 billion by 2050, it is essential to boost agricultural output while minimizing environmental impact. The Internet of Things has the potential to revolutionize conventional agricultural practices by enabling the use of contemporary technologies to improve crop productivity and quality. The suggested system provides a platform for urban farmers to interact with potential clients, monitor their cultivation, and obtain data analysis in order to create high-quality goods. The proposed system aims to increase the productivity and profitability of small-scale urban farmers by boosting awareness of the use of technology in urban farming and offering a plant status tracking service and a marketplace platform.

Overall, by integrating software and hardware components, the system enables farmers to collect and analyze real-time data on plant conditions, improve crop yield and soil quality, and maximize the use of small spaces while also creating a user- and publicly-accessible marketplace that facilitates the trading of goods and reduces the involvement of middlemen in the transaction. So, the proposed system represents a significant step toward the digitization of agricultural methods and techniques, and it has the potential to make a valuable contribution to the agricultural sector in Ethiopia.

6.2. Recommendation

Based on what we learned from the research, we strongly suggest that urban farmers in Ethiopia use MOFER, which is an IoT-based application for providing services to farmers in cities. The proposed system has the potential to change the way agriculture is done in Ethiopia by making small-scale urban farming more productive, profitable, and sustainable.

The application may even incorporate AI's and powerful machine learning algorithms to analyze the data collected by sensors and provide real-time recommendations to farmers.

The marketplace can be implemented on application and not just web, plant disease detection systems and camera sensors can be used to monitor and identify the health of plants.

The integrated hardware may be modified to cover more range and cover large parts of areas so that it can increase data input and analysis.

Therefore, by providing a way to track the status of plants and a marketplace, our application could make a big difference in the agricultural sector in Ethiopia, especially in helping to solve the problems that urban farmers are already facing. Overall, the proposed system could make urban farmers more competitive, help make sure there is enough food, and encourage sustainable farming practices.

Appendices

Appendix A: Code Sample

```
void setup() {
  pinMode(A0, INPUT);
  Serial.begin(9600);
  dht.begin();
  WiFi.begin(ssid, password);
  Serial.println("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.print("Connected to WiFi network with IP Address: ");
  Serial.println(WiFi.localIP());
}

int getSoilMoisture() {
  int soilMoistureValue = analogRead(A0);
  int soilMoisturePercentage = map(soilMoistureValue, 0, 4095, 0, 100);
  return soilMoisturePercentage;
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    http.begin(client, serverName);
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();
    float soil_moisture_percentage;
    digitalWrite(Ledpin, HIGH);
    delay(500);
    digitalWrite(Ledpin, LOW);
    delay(500);

    http.addHeader("Content-Type", "application/x-www-form-urlencoded");
    http.addHeader("authorization", "token")
    String httpRequestData = "temprature=" + temperature + "&humidity=" + humidity + "&soil_moisture=" +
    soil_moisture_percentage + "";
    Serial.print("httpRequestData: ");
    Serial.println(httpRequestData);
    int httpResponseCode = http.POST(httpRequestData);
    if (httpResponseCode == 200) {
      Serial.print("HTTP Response code: ");
      Serial.println(httpResponseCode);
    } else {
      Serial.print("Error code: ");
      Serial.println(httpResponseCode);
    }
    http.end();
  } else {
    Serial.println("WiFi Disconnected");
  }
  //Send an HTTP POST request every 15 seconds
  delay(15000);
}
```

Fig A.1: Arduino C code snippet

```

const filePath3 = path.join(__dirname, "..", "views", "reset_password.ejs");
function password_resetter(name, link, email) {
  return new Promise((resolve, reject) => {
    var transporter = nodemailer.createTransport({
      host: "mail.meinab.com",
      secure: true,
      port: 465,
      secure: true,
      auth: {
        user: "noreply-mofer@meinab.com",
        pass: "*****",
      },
    });
    ejs.renderFile(filePath3, { name: name, link: link }, function (err, html) {
      if (err) {
        console.error(err);
      } else {
        const mailOptions = {
          from: "noreply-mofer@meinab.com",
          to: email,
          subject: "Password Reset",
          html: html,
        };

        transporter.sendMail(mailOptions, function (err, info) {
          if (err) {
            console.error(err);
          } else {
            console.log("Email sent: " + info.response);
          }
        });
      }
    });
  });
}

const forget_password = async (req, res) => {
  const userEmail = req.query.customer_email;
  console.log("test");
  f_admin
    .auth()
    .generatePasswordResetLink(userEmail)
    .then((link) => {
      res.status(200).json("email sent " + link);
      password_resetter("Customer ", link, userEmail);
    })
    .catch((error) => {
      console.log(error);
      res.status(400).json(error);
    });
};

```

Fig A.2: MOFER API Node.js Code snippet.

```

1  sendData() async {
2      debugPrint("Function called");
3      final data = {'customer_email': emailController.text.toString()};
4      final uri =
5          Uri.http('192.168.1.2:5000', '/api/android/forget_password', data);
6      final response = await http.get(uri);
7      if (response.statusCode == 200) {
8          debugPrint(
9              "Sent email to ${emailController.text} the request data is as followed : $data");
10         if (context.mounted) {
11             ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
12                 content: Text('Check your email'),
13             ));
14         }
15         if (context.mounted) {
16             Navigator.pop(context);
17         }
18     } else {
19         if (context.mounted) {
20             ScaffoldMessenger.of(context).showSnackBar(SnackBar(
21                 content: Text('${emailController.text} does not exist'),
22             ));
23         }
24     }
25 }

```

Fig A.3: MOFER Plant status tracking App Dart code snippet.

```

const Products = () => {
  auth()
  const [data, setData] = useState([]);
  const [filter, setFilter] = useState(data);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    let componentMounted = true;
    const getProducts = async () => {
      setLoading(true);
      try {
        const response = await axios.get('http://localhost:5000/api/web/allProducts');
        if (componentMounted) {
          setData(response.data.data);
          setFilter(response.data.data);
          setLoading(false);
        }
      } catch (error) {
        console.error(error);
      }
    };
    getProducts();
    return () => {
      componentMounted = false;
    };
  }, []);
}

```

Fig A.4: MOFER Marketplace React.js code snippet

Appendix B: User questions

Google form survey charts

Demography

96 responses

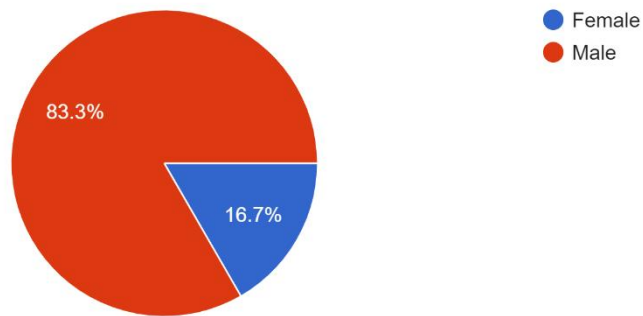


Fig B.1: Questionnaire 1

How do you currently track the status of your plants?

96 responses

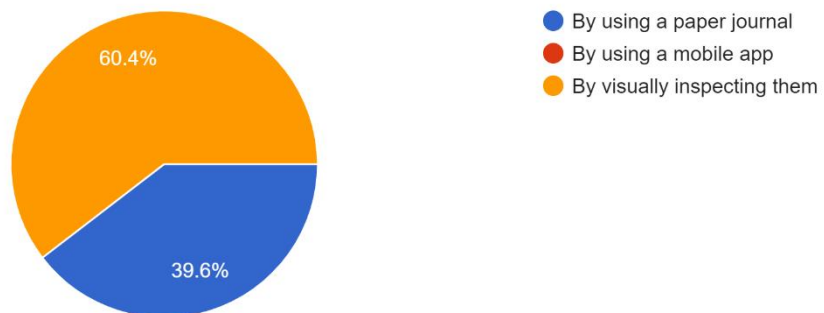


Fig B.2: Questionnaire 2

How do you currently track the status of your plants?

96 responses

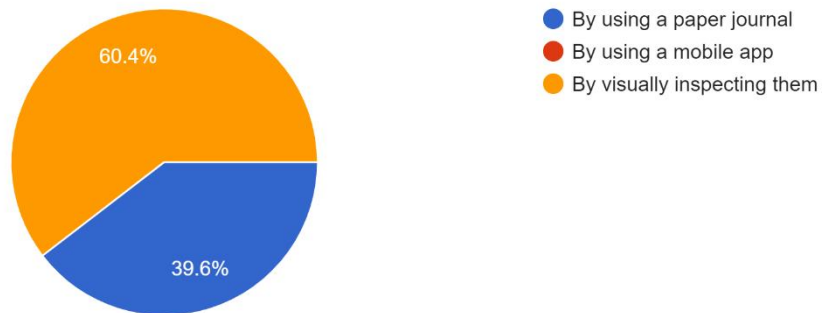


Fig B.3: Questionnaire 3

What types of information would you like to track about your plants?

96 responses

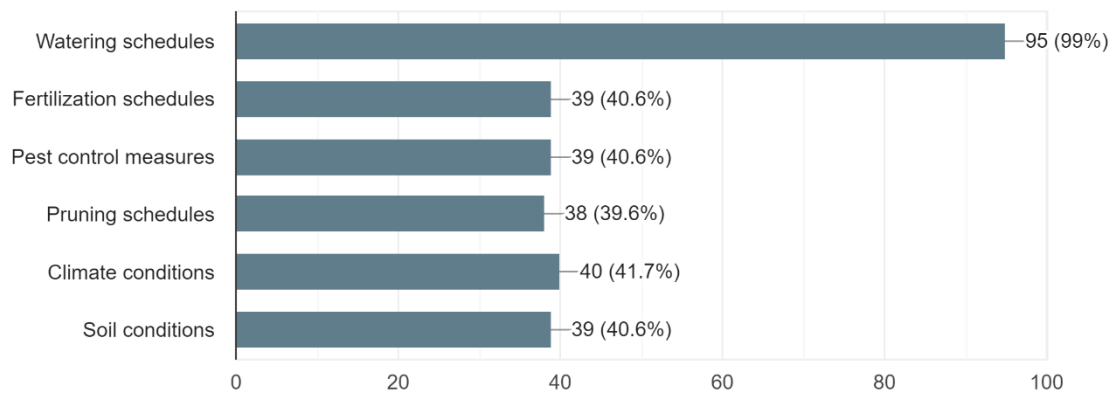


Fig B.4: Questionnaire 4

Are you interested in buying or selling plants or gardening supplies through the app?

96 responses

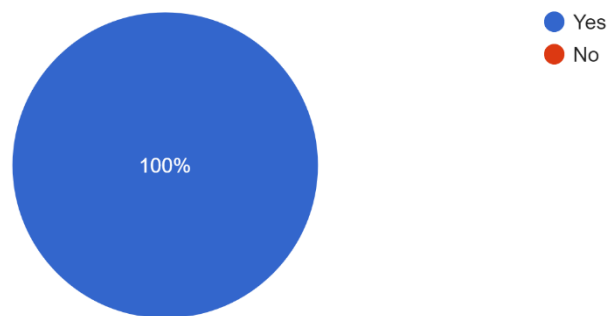


Fig B.5: Questionnaire 5

What is your primary motivation for purchasing plants?Decorative purposesEdible purposesEnvironmental reasonsRelaxationOther (please specify)

96 responses

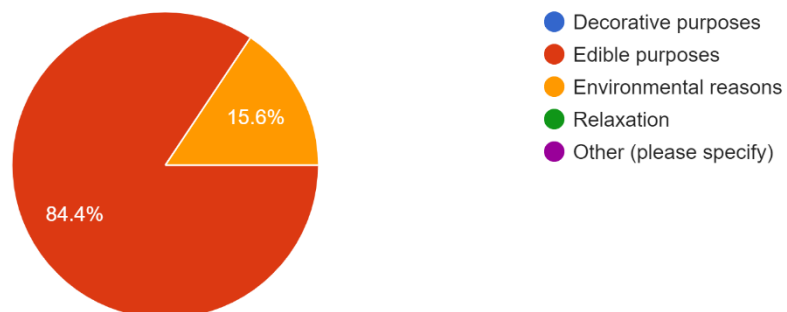


Fig B.6: Questionnaire 6

What type of plants/products do you primarily buy?

96 responses

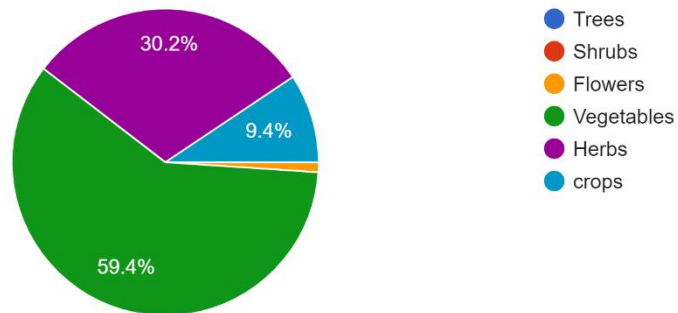


Fig B.7: Questionnaire 7

How many plants do you currently have in your garden or space?

96 responses

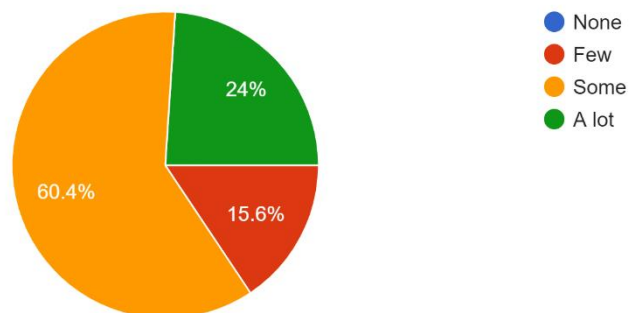


Fig B.8: Questionnaire 8

What features would you like to see in a plant cultivation status tracking app?

96 responses

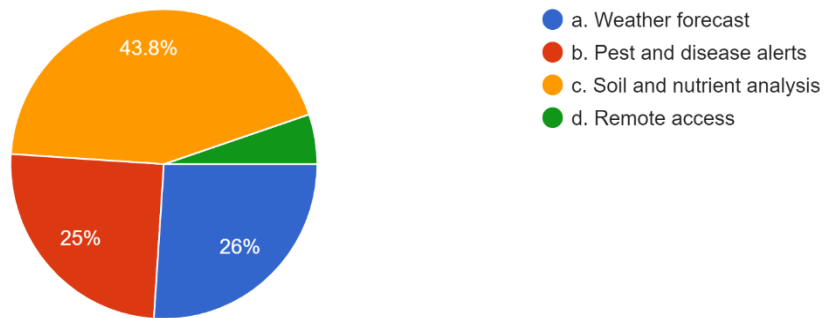


Fig B.9: Questionnaire 9

How often would you want to check the app for updates on your plants?

96 responses

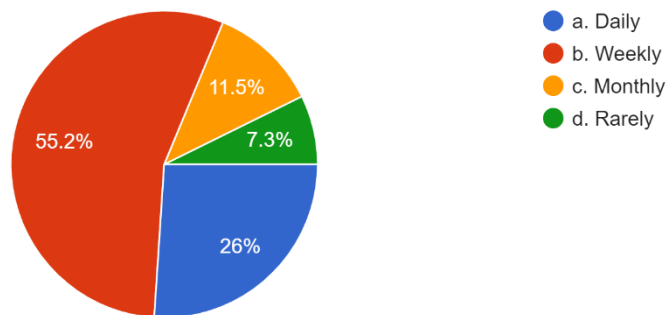


Fig B.10: Questionnaire 10

Are there any specific challenges you face in terms of tracking the status of your plants currently?

96 responses

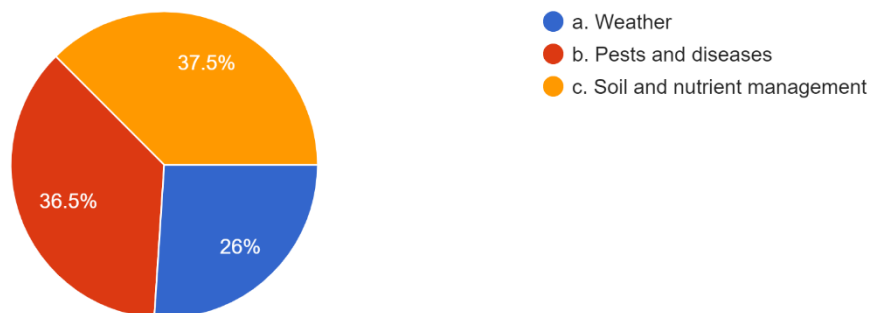


Fig B.11: Questionnaire 11

Appendix C: Interview

- 1) What is the impact of Digital green Ethiopia on the agriculture sector?
- 2) Does Digital green Ethiopia involve the agriculture market?
- 3) What type of technologies does Digital green Ethiopia adopt?
- 4) Do you involve in urban agriculture if so, how?
- 5) What are the challenges you have faced so far in this business?
- 6) As we explained how our app works how relevant would it be in the agriculture sector

References

1. FAO 2017 The future of food and agriculture trends and challenges FAO Agri.services Bulletin (Rome)
2. Tekinerdogan, B. Strategies for Technological Innovation in Agriculture 4.0. Reports; Wageningen University: Wageningen, The Netherlands, 2018.
3. Ferrandez-Pastor, F.J.; Garcia-Chamizo, J.M.; Nieto-Hidalgo, M.; Mora-Pascual, J.; MoraMartinez, J. Developing ubiquitous sensor network platform using Internet of Things: Application in precision agriculture. *Sensors* 2016, 16, 1141. [CrossRef]
4. Wolfert, S.; Ge, L.; Verdouw, C.; Bogaardt, M.J. Big data in smart farming—A review. *Agric. Syst.* 2017, 153, 69–80. [CrossRef]
5. Liakos, K.G.; Busato, P.; Moshou, D.; Pearson, S.; Bochtis, D. Machine learning in agriculture: A review. *Sensors* 2018, 18, 2674. [CrossRef]
6. O’Grady, M.J.; O’Hare, G.M.P. Modelling the smart farm. *Inf. Process. Agric.* 2017, 4, 179–187. [CrossRef]
7. Quy, V.K.; Hau, N.V.; Anh, D.V.; Quy, N.M.; Ban, N.T.; Lanza, S.; Randazzo, G.; Muzirafuti, A. IoT-Enabled Smart Agriculture: Architecture, Applications, and Challenges. *Appl. Sci.* 2022, 12, 3396. [CrossRef]
8. Tay, L. (2008, December 10). Agile Methodology: A Software Development Process Framework. Retrieved March 20, 2013, from Yahoo! Voices
:<http://voices.yahoo.com/agile-methodology-software-development-process-framework-2263234.html>
9. Agile Methodology. (2008, October 23). Retrieved March 27, 2013, from Agile Methodology:<http://agilemethodology.org>
10. Y. Tesema, “Urban farming in Addis Ababa: Residents find ways to feed an expanding city,” *Medium*, 18-Sep-2020. [Online]. Available:<https://medium.com/enabling-sustainability/urban-farming-in-addis-abeba-residents-find-ways-to-feed-an-expanding-city-da3ff6dce64d>. [Accessed: 14-Dec-2022].